



UNIVERSITÀ
DI TORINO



gPLUTO

v 0.91 (June 2026)

User's Guide

Main Developer: A. Mignone^{1,2} [andrea.mignone@unito.it]

Contributors:

M. Rossazza ¹	[marco.rossazza@unito.it]
V. Berta ¹	[vittoria.bera@unito.it]
A. Suriano ¹	[alessio.suriano@unito.it]
M. Bugli ¹	[matteo.bugli@unito.it]
S. Truzzi ¹	[stefano.truzzi@unito.it]
G. Mattia ³	[mattia@mpia.de]
J. D. Melon Fuksman ⁴	[fuksman@mpia.de]

¹ Dipartimento di Fisica, Turin University, Via P. Giuria 1 - 10125 Torino (TO), Italy

² INAF Osservatorio Astronomico di Torino, Via Osservatorio, 20 10025 Pino Torinese (TO), Italy

³ Max Planck Institute for Astronomy, Königstuhl 17, 69117 Heidelberg, Germany

Terms & Conditions of Use

The public version of **gPLUTO** is distributed freely under the BSD 3-Clause License. Code's development and support requires a great deal of work and for this reason we expect **gPLUTO** to be referenced and acknowledged by authors who use it for their publications. In particular,

- If you're using the static grid version, please cite:
Mignone et al. ApJS (2007) 170, 228M
<https://ui.adsabs.harvard.edu/abs/2007ApJS..170..228M/abstract>
- If you're using the MHD-PIC module, please cite:
Mignone et al. ApJ (2018) 859, 13M
<https://ui.adsabs.harvard.edu/abs/2018ApJ...859...13M/abstract>
- If you're using the RADIATION module, please cite:
Melon Fuksman & Mignone ApJS (2019) 242, 20
<https://ui.adsabs.harvard.edu/abs/2019ApJS..242...20M/abstract> for **relativistic** HD/MHD
or
Melon Fuksman et al. ApJ (2021) 906, 78
<https://ui.adsabs.harvard.edu/abs/2021ApJ...906...78M/abstract> for **nonrelativistic** HD/MHD.

Co-authorship may be solicited for those publications demanding considerable additional support and/or changes to the code.

Contents

0	Quick Start	5
0.1	Downloading and unpacking gPLUTO	5
0.2	Running a simple shock-tube problem	5
0.3	Running the Orszag-Tang MHD vortex test	6
0.4	Setting up your own test problem	7
0.5	Supplied test problems	8
0.6	Migrating from PLUTO to gPLUTO	9
1	Introduction	11
1.1	System Requirements	12
1.2	Directory Structure	12
1.3	Configuring gPLUTO	13
1.4	Compiling & Running the Code	14
1.4.1	Output Log file	14
1.4.2	Command line options	15
1.5	Modifying the Distribution Source Files	15
1.6	GPU Programming Models	16
1.6.1	OpenACC Features	16
1.6.2	OpenMP Features	17
2	Problem Header File: definitions.hpp	18
2.1	Basic Options	18
2.1.1	<u>PHYSICS</u>	18
2.1.2	<u>DIMENSIONS</u>	18
2.1.3	<u>GEOMETRY</u>	19
2.1.4	<u>MAPPED_GRID</u>	19
2.1.5	<u>BODY_FORCE</u>	22
2.1.6	<u>COOLING</u>	23
2.1.7	<u>RECONSTRUCTION</u>	23
2.1.8	<u>TIME_STEPPING</u>	23
2.1.9	<u>NTRACER</u>	24
2.1.10	<u>PARTICLES</u>	24
2.1.11	<u>USER_DEF_PARAMETERS</u>	24
2.2	Physics-Dependent Options	25
2.2.1	<u>BACKGROUND_FIELD</u> [MHD]	25
2.2.2	<u>DIVB_CONTROL</u> [MHD, RMHD, ResRMHD]	25
2.2.3	<u>EOS</u>	25
2.2.4	<u>ENTROPY_SWITCH</u>	26
2.2.5	<u>ROTATING_FRAME</u> [HD, MHD]	26
2.2.6	<u>RADIATION</u> [HD, MHD, RHD, RMHD]	27
2.3	User-defined Constants	27
2.4	Fallback Strategies: Making Your Simulation More Robust	28

3	Makefile Selection: makefile	30
3.1	GPU Support	30
3.1.1	OpenACC Support	30
3.1.2	OpenMP Support	30
3.2	MPI Library (Parallel) Support	31
3.3	HDF5 Library Support	31
3.3.1	Installing HDF5	31
3.4	Including Additional Files: local_make	32
4	Runtime initialization file: pluto.ini	33
4.1	The <i>[Grid]</i> block	34
4.2	The <i>[Time]</i> Block	34
4.3	The <i>[Radiation Parameters]</i> Block	35
4.4	The <i>[Solver]</i> Block	35
4.5	The <i>[Boundary]</i> Block	36
4.6	The <i>[Static Grid Output]</i> Block	37
4.7	The <i>[Particles]</i> Block	39
4.8	The <i>[Parameters]</i> Block	39
5	Initial and Boundary Conditions	41
5.1	Initial Conditions: the <code>Init()</code> function	41
5.1.1	Units and Dimensions	42
5.1.2	Assigning Initial Conditions from Input Files	44
5.2	User-defined Boundary Conditions	46
5.2.1	Internal Boundary	48
5.3	Body Forces	50
5.4	The <code>Analysis()</code> function	52
6	Basic Physics Modules	54
6.1	The HD Module	54
6.2	The MHD Module	55
6.2.1	Assigning Magnetic Field Components	55
6.2.2	Controlling the $\nabla \cdot \mathbf{B} = 0$ Condition	56
6.2.3	Background Field Splitting	59
6.3	The RHD Module	61
6.4	The RMHD Module	62
6.5	The Resistive RMHD (<i>ResRMHD</i>) Module	63
6.5.1	Controlling Resistivity	64
7	Equation of State	65
7.1	The <i>ISOTHERMAL</i> Equation of State	65
7.2	The <i>IDEAL</i> Equation of State	66
7.3	The <i>TAUB</i> Equation of state	66
8	Optically Thin Cooling	67
8.1	Tabulated Cooling	68
8.2	Simplified Non-Equilibrium Cooling: <i>SNEq</i>	69
9	Additional Modules	70
9.1	The FARGO Module	70
9.1.1	Using the Module	70
9.1.2	A Note on Parallelization	71
9.2	The RADIATION Module	72
9.2.1	Using the module	73

10	Particles	76
10.0.1	Particle Storage and Memory Management	76
10.1	Initial, Boundary and Injection conditions	77
10.1.1	Initial Conditions	77
10.1.2	Boundary Conditions	78
10.1.3	Injection	78
10.2	Lagrangian Particles	79
10.2.1	Radiative Losses - LP Model	79
10.2.2	Spectral Distribution Initialisation	80
10.3	Particles Output and Visualization	83
10.3.1	Visualization	84
11	I/O and Visualization	87
11.1	Output Data Formats	87
11.1.1	Binary Output: dbl or flt data formats	88
11.1.2	HDF5 Output: dbl.h5 or flt.h5 data formats	88
11.1.3	Restart	88
11.1.4	VTK Output: legacy vtk data format	89
11.1.5	ASCII Output: tab Data format	89
11.1.6	The grid.out output file	89
11.2	Customizing your output	90
11.3	Visualization	91
11.3.1	Visualization with Gnuplot	91
11.3.2	Visualization with IDL	92
11.3.3	Visualization with PyPLUTO	95
11.3.4	Visualization with VisIt or ParaView	96
A	Equations in Different Geometries	98
A.1	MHD Equations	98
A.1.1	Cartesian Coordinates	98
A.1.2	Cylindrical Coordinates	99
A.1.3	Spherical Coordinates	100
A.2	(Special) Relativistic MHD Equations	101
A.2.1	Cartesian Coordinates	101
A.2.2	Cylindrical Coordinates	101
A.2.3	Spherical Coordinates	102
B	Predefined Constants and Macros	103
B.1	Predefined Physical Constants	103
B.2	Predefined Function-Like Macros	103
B.3	Advanced Options	104

0. Quick Start

0.1 Downloading and unpacking gPLUTO

gPLUTO can be download from <https://gitlab.com/PLUTO-code/gPLUTO> using

```
> git clone https://gitlab.com/PLUTO-code/gPLUTO.git
```

This will create the folder **gPLUTO/** in your home directory. At this point, we advise to set the environment variable **PLUTO_DIR** to point to your code directory. Depending on your shell (e.g. **tcsh** or **bash**) use either one of

```
> export PLUTO_DIR=/home/user/gPLUTO # If you're using the bash shell;
> setenv PLUTO_DIR /home/user/gPLUTO # If you're using the tcsh shell;
```

0.2 Running a simple shock-tube problem

gPLUTO can be quickly configured to run one of the several test problems provided with the distribution. Assuming that your system satisfies all the requirements described in the next chapter (i.e. C++ compiler, Python, etc..) you can quickly setup **gPLUTO** in the following way:

1. Change directory to any of the test problems under **gPLUTO/Test.Problems**, e.g.

```
> cd $PLUTO_DIR/Test.Problems/HD/Sod
```

2. Copy the header and initialization files from a configuration of our choice (e.g. #01):

```
> cp definitions_01.hpp definitions.hpp
> cp pluto_01.ini pluto.ini
```

3. Run the Python script using

```
> python3 $PLUTO_DIR/setup.py
```

and select “Setup problem” from the main menu, see Fig. 1.2. You can choose (by pressing Enter) or modify the default setting using the arrow keys.

4. Once you return to the main menu, select “Change makefile”, choose a suitable makefile (e.g. **Linux.gcc.defs**) and press enter.

All the information relevant to the specific problem should now be stored in the four files **init.cpp** (assigns initial condition and user-supplied boundary conditions), **pluto.ini** (sets the number of grid zones, Riemann solver, output frequency, etc.), **definitions.hpp** (specifies the geometry, number of dimensions, reconstruction, time stepping scheme, and so forth) and the makefile.

5. Exit from the main menu (“Quit” or press ‘q’) and type

```
> make
```

to compile the code.

6. You can now run the code by typing

```
> ./pluto
```

At this point, **gPLUTO** reads the initialization file **pluto.ini** and starts integrating. The run should take a few seconds (or less) and the integration log should be dumped to screen.

Data can be displayed in a number of different ways. If you have, for example, Gnuplot (v 4.2 or higher) you can display the density output from the last written file using

```
gnuplot> plot "data.0001.dbl" bin array=400:400:400 form="%double" ind 0
```

where ind 0,1,2 may be used to select density, velocity or pressure. If you have IDL installed on your system, you can easily plot the density by¹:

```
IDL> pload,1
IDL> plot,x1,rho
```

The IDL procedure pload is provided along with the code distribution.

0.3 Running the Orszag-Tang MHD vortex test

1. Change directory to gPLUTO/Test.Problems/MHD/Orszag_Tang.
2. Choose a configuration (e.g. #02) and copy the corresponding configuration files, i.e.,

```
> cp definitions_02.hpp definitions.hpp
> cp pluto_02.ini pluto.ini
```

3. Run the Python script:

```
> python3 $PLUTO_DIR/setup.py
```

select “Setup problem” and choose the default setting by pressing enter;

4. Once you return to the main menu, select “Change makefile” and choose a suitable makefile (e.g. Linux.gcc.defs) and press enter.
5. Exit from the main menu (“Quit” or press ‘q’). Edit pluto.ini and, under the *[Grid]* block, lower the resolution from 512 to 200 in both directions (X1-grid and X2-grid). Change *single_file*, in the “dbl” output under the *[Static Grid Output]* block, to *multiple_files*.
6. Compile the code:

```
> make
```

7. If compilation was successful, you can now run the code by typing

```
> ./pluto
```

At this point, **gPLUTO** reads the initialization file pluto.ini and starts integrating. The run should take a few minutes (depending on the machine you’re running on) and the integration log should be dumped to screen.

You can display data (e.g. density and x velocity) with Gnuplot (v 4.2 or higher) from the last written file using

```
gnuplot> set pm3d map      # set map style drawing
gnuplot> set palette gray # set color to black and white
gnuplot> splot "rho.0001.dbl" bin array=200x200 format="%double" # density
gnuplot> splot "vx1.0001.dbl" bin array=200x200 format="%double" # density
```

If you have IDL installed, you can easily display pressure from the last written output files with

```
IDL> pload,1
IDL> display,x1=x1,x2=x2,prs
```

Several other visualization options are described in more details in §11.3.

¹You need to include gPLUTO/Tools/IDL into your IDL search path, §11.3.2

0.4 Setting up your own test problem

As an illustrative example, we show how **gPLUTO** can be configured to run a 2D Cartesian hydrodynamic blast wave from scratch. We assume that you have already followed the steps in §0.1.

1. First, in your home or work directory, you need to create a folder which will contain the necessary files for the test. For instance,

```
> mkdir Blastwave
> cd Blastwave
```

2. You can now start the setup process by invoking the Python script to set dimensions, geometry, numerical scheme and so on:

```
> python3 $PLUTO_DIR/setup.py
```

and select “Setup problem” from the main menu.

Using the arrows keys make the following changes: set “DIMENSIONS” to 2, “USER_DEF_PARAMETERS” to 3 and leave the other fields as they are. User-defined parameters will be used later in the initial condition routine. Press enter to confirm the changes and proceed to the following screen menu. Since we don’t have to change anything here you can press enter once more.

3. We now set the names of the 3 auxiliary parameters previously introduced. To do so, use the arrow keys to select each of them and explicitly write their names: P_IN, P_OUT and GAMMA and press enter to confirm.
4. Finally, we complete the python session by setting the architecture for the makefile. In the makefile menu choose your system configuration (e.g. Linux.gcc.defs for Linux). Press enter to confirm.

You are now done with the Python script and can exit by pressing either “q” or selecting quit. At this point you should find the following four files inside your Blastwave folder: definitions.hpp, init.cpp, makefile, pluto.ini, sysconf.out

Next, we need to edit the two files pluto.ini and init.cpp. The first one defines the computational domain and certain properties of the run (i.e. time of integration, first timestep etc). The second one sets the initial conditions for the blast wave problem: a circular region of high pressure in a lower pressure ambient.

Edit pluto.ini to make the following changes:

- The domain should span from -1 to 1 in both dimensions with 200 points in each direction.

```
X1-grid    -1.0    200    1.0
X2-grid    -1.0    200    1.0
```

- The simulation should stop when time reaches 0.04:

```
tstop      0.04
```

with the first timestep being

```
first_dt    1.e-6
```

Save the files every $t=0.004$, in double precision and in multiple_files format.

```
dbl         0.004  -1  multiple_files
```

- At the end of the file, set the numerical values for the 3 parameters P_IN (the high pressure of a region yet to be specified), P_OUT (the ambient pressure) and GAMMA (polytropic index):

```
P_IN       8.e2
P_OUT      8.0
GAMMA      1.6666666666666667
```

Save and exit the editor.

Next, you need to edit init.cpp.

- Define inside the function Init() the radius r, a floating point value which we will be used to set a circular region of high pressure.


```
double r;
```

- Set the global variable `g_gamma` (polytropic index) and the radius `r`. Define the initial ambient pressure (`P_OUT`) and put an IF statement to specify the high pressure region inside a circle of `r=0.3` (`P_IN`):

```
g_gamma = run_config.inputParam[GAMMA]; /* calls the auxiliary parameter GAMMA */
r = x1*x1 + x2*x2;
r = sqrt(r);

v[RHO] = 1.0;          /* initial density array */
v[VX1] = 0.0;          /* initial Vx array */
v[VX2] = 0.0;          /* initial Vy array */
v[VX3] = 0.0;          /* initial Vz array */
v[PRS] = run_config.inputParam[P_OUT]; /* calls the auxiliary parameter P_OUT */

if (r <= 0.3) v[PRS] = run_config.inputParam[P_IN]; /* calls the input parameter P_IN */
```

Save and exit the editor. Compile the code and run **gPLUTO** with a the following set of commands:

```
> make
> ./pluto
```

In order to visualize the results follow the instructions described in the two previous sections.

0.5 Supplied test problems

The official distribution of **gPLUTO** comes with several examples and test problems that can be found under the `Test_Problems/` folder. Documentation is extracted from comments at the beginning of `init.cpp` sources files using the Doxygen documentation system and an on-line documentation browser can be found in `Doc/test_problems.html`. Test problem documentation is still being added and more examples will be available in future releases.

0.6 Migrating from PLUTO to gPLUTO

gPLUTO does not introduce major changes with respect to its predecessor and it retains a similar (albeit not identical) structure in the syntax of the basic functions defined in `init.cpp` (former `init.c`). Some optimizations and improvements have been introduced when switching to C++. Here you may find some guidelines helpful in migrating your setup from the legacy code (**PLUTO**) to **gPLUTO**. Before migrating, here's a list of a few important changes you need to know:

1. Compiler: **gPLUTO** is written in C++ and it requires a C++ compiler to run on CPU or GPU, depending on the target architecture and compiler choice.
2. Main header file: "`gpluto.hpp`" replaces "`pluto.h`".
3. Problem setup: your scientific case is still defined in terms of the standard set of 3 files, which are now named:
 - `init.cpp`: assign initial and boundary conditions, gravity and so on;
 - `definitions.hpp`: set compile-time problem specifications;
 - `pluto.ini`: set runtime problem specifications.
4. Argument passing: the new structure `RunConfig` has been introduced in order to replace the employment of global variables, see the API structure reference. This structure is often employed in the code to pass various problem-dependent configuration parameters:

```
run_config.gamma;           /**< The adiabtic index for the ideal EoS */
run_config.isoSoundSpeed = 1.0; /**< The isothermal sound speed (for isothermal EOS) */
run_config.time;           /**< Simulation time */
run_config.domBeg[3];      /**< Lower limits of the computational domain. */
run_config.domEnd[3];      /**< Upper limits of the computational domain. */
run_config.OmegaZ;         /**< Rotating frame angular frequency */
run_config.small_prs = 1.e-12;
run_config.small_rho = 1.e-12;
run_config.unit_density = CONST_mp; /**< Unit density (in gr/cm^3) */
run_config.unit_velocity = CONST_c; /**< Unit velocity (in cm/sec) */
run_config.unit_length = CONST_au; /**< Unit length (in cm) */
```

5. 3D Array indexing: **gPLUTO** employs classes to deal with 3D or 4D data arrays. The spatial ordering is always the same and given by (i, j, k) :

PLUTO (Legacy)

```
d->Vc[nv][k][j][i] = 0.0; // Prim. var array
d->Vs[nv][k][j][i] = 0.0; // Staggered array
d->Uc[k][j][i][nv] = 0.0; // Cons. var array
```

gPLUTO

```
d->Vc(i,j,k,nv) = 0.0; // Prim. var array
d->Vs[nv](i,j,k) = 0.0; // Staggered array
d->Uc(i,j,k,nv) = 0.0; // Cons. var array
```

6. Interface indexing: when referring to an interface, now i means $i - \frac{1}{2}$. Example:

PLUTO (Legacy)

```
// Build rhs = -dt/dx * (F[i+1/2] - F[i-1/2])
NVAR_LOOP(nv) {
    rhs[i][nv] = -dtdx*( flux[i][nv]
                        - flux[i-1][nv]);
}
```

gPLUTO

```
// Build rhs = -dt/dx * (F[i+1/2] - F[i-1/2])
NVAR_LOOP(nv) {
    rhs(i,nv) = -dtdx*( flux(i+1,nv)
                      - flux(i,nv));
}
```

7. Domain Looping Macros: general loop macros like `DOM_LOOP()` or `TOT_LOOP()` have been replaced by the more convenient and general-purpose `BOX_LOOP()` macro, e.g.,

PLUTO (Legacy)

```
DOM_LOOP(k,j,i){
    A[i][j][k] = 0.0;
}
```

gPLUTO

```
RBox box;
RBoxDefine(grid->lbeg, grid->lend, &box);

BOX_LOOP(&box,i,j,k){
    A(i,j,k) = 0.0;
}
```

CURRENT RELEASE: gPLUTO 0.89 (Dec 2025)

This is the 2nd beta release of the gPLUTO code, a full C++ rewrite of the PLUTO code, and capable of running on both multiple CPUs and GPUs.

Algorithms not supported or in development stage:

- Adaptive Mesh refinement (development)
- Cooling (development)
- CHARACTERISTIC_TRACING (not supported)
- HANCOCK (not supported)
- SHEARINGBOX (not supported)
- FINITE DIFFERENCE (not supported)

Main notation differences:

- Staggered arrays start from zero ("i" means "i-1/2" instead of "i+1/2");
- Reduced usage of global variables, replaced with the RunConfig structure;
- Grid definition in [pluto.ini] has changed;
- Global coordinate indices IBEG, IEND, JBEG, JEND have been removed and replaced by extensive usage of RBox structures;
- Limiter is unique for all variables;
- CYLINDRICAL Geometry can be used for both (r,phi) or (r,z).
For the latter simply set DIMENSIONS == 3 and set INCLUDE_JDIR to NO.

1. Introduction

gPLUTO is the newest version of **PLUTO** – a finite-volume / finite-difference, shock-capturing code designed to integrate a system of conservation laws.

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot \mathbf{T}_h = \nabla \cdot \mathbf{T}_p + \mathbf{S}(\mathbf{U}) \quad (1.1)$$

where $\mathbf{U}$ represents a set of conservative quantities, $\mathbf{T}_h(\mathbf{U})$ is the (hyperbolic) flux tensor, $\mathbf{T}_p$ is the diffusion (parabolic) flux tensor and $\mathbf{S}(\mathbf{U})$ defines the source terms [MBM⁺07, MZT⁺12]. An equivalent set of primitive variables $\mathbf{V}$ is more conveniently used for assigning initial and boundary conditions. The explicit form of $\mathbf{U}$, $\mathbf{V}$, $\mathbf{T}(\mathbf{U})$ and $\mathbf{S}(\mathbf{U})$ depends on the particular physics module selected:

- *HD*: Newtonian (classical) hydrodynamics, §6.1;
- *MHD*: ideal/resistive magnetohydrodynamics, §6.2;
- *RHD*: special relativistic hydrodynamics, §6.3;
- *RMHD*: special (ideal) relativistic magnetohydrodynamics, §6.4;
- *ResRMHD*: special (ideal) relativistic magnetohydrodynamics with resistivity §6.5;

gPLUTO adopts a structured mesh approach for the solution of the system of conservation laws (1.1). Flow quantities are discretized on a logically rectangular computational grid enclosed by a boundary and augmented with guard cells or ghost points in order to implement boundary conditions on a given computational stencil. Computations are done using double precision arithmetic.

gPLUTO is a standalone package entirely written in C++. It can run on parallel architectures with the help of MPI and , it can exploit the parallel computing of GPUs with the help of OpenACC directives.

PLUTO has been successfully ported to several parallel platforms including Linux, Windows/Cygwin, Mac OS X, Beowulf clusters, IBM power4 / power5 / power6, SGI Irix, IBM BluGene/P and several others. Figure 1.1 shows the strong scaling on a BlueGene/P machine up to 32,768 processors on a periodic domain with 512^3 computational grid zones.

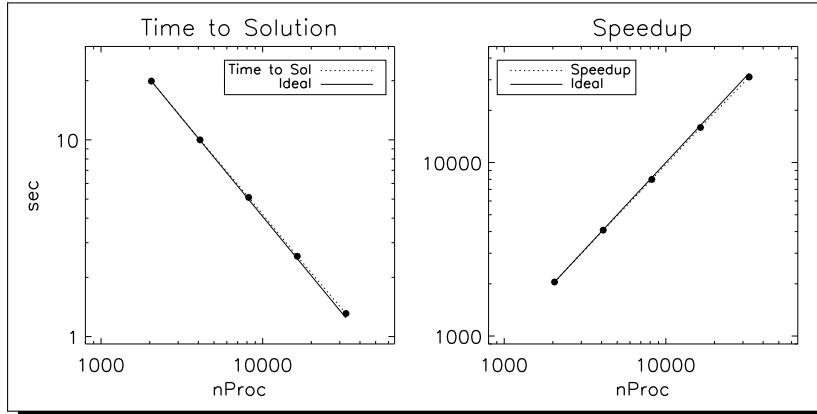


Figure 1.1: Strong scaling of **gPLUTO** on a periodic domain problem with 512^3 grid zones. Left panel: average execution time (in seconds) per step vs. number of processors. Right panel: speedup factor computed as T_1/T_N where T_1 is the (inferred) execution time of the sequential algorithm and T_N is the execution time achieved with N processors. Code execution time is given by black circles (+ dotted line) while the solid line shows the ideal scaling.

gPLUTO scaling tests were performed on many HPC systems: Leonardo, Marenstrum, Meluxina, Karolina. This work is still in progress and will be updated in future versions.

1.1 System Requirements

gPLUTO can run on most platforms but some software prerequisites must be met, depending on the specific configuration you intend to use. The minimal set to get **gPLUTO** running on a workstation with a static grid (no AMR) requires Python3.xx, a C++ compiler and the make utility. These are usually installed by default on most Linux/Unix platforms. A comprehensive list is shown in Table 1.1.

	Static Grid	
	<i>serial</i>	<i>parallel</i>
Python (> 3.0)	yes	yes
C++ compiler	yes	yes
GNU make	yes	yes
MPI library	–	yes
HDF5 library (v 1.6 or 1.8)	opt	opt
PNG library	opt	opt

Table 1.1: Software requirements for different applications of **gPLUTO**. Here “opt” stands for optional, “serial” refers to single-processor runs and “parallel” to multiple-processor architectures.

Parallelization is handled internally (no external library is required). The HDF5 library can be used with the static grid version of the code.

1.2 Directory Structure

Once unpacked, your **gPLUTO**/ root directory should contain the following folders:

- Config/: contains machine architecture dependent files, such as information about C++ compiler, flags, library paths and so on. Important for creating the makefile;
- Doc/: documentation directory;
- Lib/: repository for additional libraries;
- Src/: main repository for all *.cpp source files with the exception of the init.cpp file, which is left to the user. The physics module source files are located in their respective sub-directories: HD/ (classical hydrodynamics), RHD/ (special relativistic hydrodynamics), MHD/ (magnetohydrodynamics), RMHD/ (relativistic magnetohydrodynamics), ResRMHD/ (resistive relativistic magnetohydrodynamics). Cooling, viscosity, thermal conduction and additional physics models are located under the folders with similar names (e.g. Cooling/, Viscosity/, Thermal_Conduction). The Templates/ directory contains templates for the user-dependent files such as init.cpp, pluto.ini, makefile and definitions.hpp;
- Tools/: Collection of useful tools, such as Python scripts, IDL visualization routines, pyPLUTO, etc...;
- Test_Problems/: a directory containing several test-problems used for code verification.

gPLUTO should be compiled and executed in a separate working directory which may be anywhere on your local hard drive.

Although most of the current algorithms can be considered in their final stable version, the code is under constant development and updates are released once or twice per year. When upgrading to a newer version of the code, it is recommended that the entire **gPLUTO**/ directory tree be deleted. Syntax changes are usually listed in the file CHANGES, in the **gPLUTO**/ root directory.

Option	Description
<code>--auto-update</code>	automatically update the configuration files without explicitly running the python menu;
<code>--no-curses</code>	disables the curses terminal control feature of the Python script. Instead a shell-based setup will be used. This switch can be used to circumvent problems with the ncurses library present on some systems (e.g. Snow Leopard 10.6);

Table 1.2: Command line options available when running the Python setup script.

1.3 Configuring gPLUTO

In order to configure and setup **gPLUTO** for a particular problem, *four* main steps have to be followed; the resulting configuration will then be stored in 4 different files, part of your local working directory:

- `definitions.hpp`: header file containing all problem-dependent preprocessor directives required at compilation time (physics module, geometry, dimensions, etc.). This is the subject of Chapter 2.
- `makefile`: needed to compile **gPLUTO** and it depends on your system architecture. This is described in Chapter 3.
- `pluto.ini`: startup initialization file containing run-time parameters (grid size, CFL,...). This is the subject of Chapter 4;
- `init.cpp`: implements initial, boundary conditions, etc.... See Chapter 5.

The Python script `setup.py` is used for the first two steps while the remaining files (`pluto.ini` and `init.cpp`) should be appropriately edited by the user. Templates for all four files can be found in the `Src/Templates/` directory. Several examples are located in the test directories under the `Test.Problems/` directory.

In order to run the Python script anywhere from your hard disk we recommend to set the shell variable `PLUTO_DIR` to point to your **gPLUTO** distribution. Depending on your environment shell, use either one of

```
~> setenv PLUTO_DIR /home/user/gPLUTO # if you're using tcsh shell
~> export PLUTO_DIR=/home/user/gPLUTO # if you're using bash shell
```

The `setup.py` script can now be invoked with

```
~/MyWorkDir > python3 $PLUTO_DIR/setup.py [options]
```

Command line options are listed in Table 1.2 or can be briefly described by invoking `setup.py` with `-help`. By default the Python script uses the `ncurses` library for enhanced terminal control. However, this option may be turned off by invoking the setup script with the `--no-curses` switch. You should then¹ see the menu shown in Fig. 1.2. Additional menus, depending on the physics module, will display later.

¹Python will first create an architecture-dependent file named `sysconf.out` containing system-related information: this file does not have any specific purpose but may be helpful for the user.

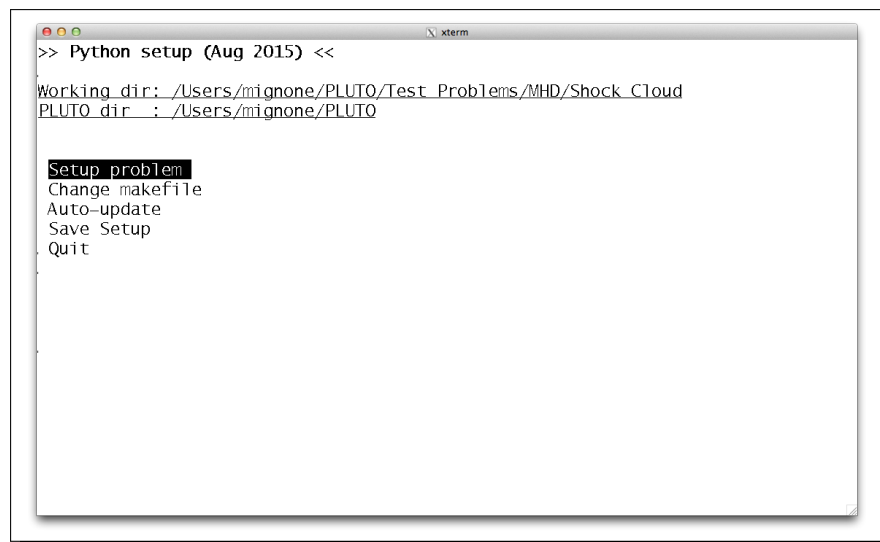


Figure 1.2: Python script main menu.

1.4 Compiling & Running the Code

After the four basic configuration files (init.cpp, definitions.hpp, makefile and pluto.ini) have been created, **gPLUTO** can be compiled from your local working directory by typing

```
~/MyWorkDir> make    # 'gmake' is also fine
```

It is important to remember that the makefile created by Python (Chapter 3) guarantees that your working directory is always searched before **gPLUTO**/Src. This turns out to be useful when modifying **gPLUTO** source files (§1.5).

If compilation is successful, type

```
~/MyWorkDir> ./pluto [flags]
```

for a single processor run, or

```
~/MyWorkDir> mpirun [...] ./pluto [args]
```

for a parallel run; [...] are options given to MPI, such as number of processors, etc, while [args] are command line options specific to **gPLUTO**, see Table 1.3. For example,

```
~/MyWorkDir> ./pluto -restart 5 -maxsteps 840
```

will restart from the 5-th double precision output file and stop computation after 840 steps.

1.4.1 Output Log file

Serial mode output log. In serial mode, the output is displayed on screen and is looks something like:

```
...
> nstep = 0; t = 0.000000e+00; dt = 1.000000e-04;    0.0 %
Max(MACH) = 2.630800
Max(vc) = 1.500000e+00  1.206000e+00  1.206000e+00  0.000000e+00  1.500000e+00
Min(vc) = 1.380000e-01 -3.489616e-06 -3.489616e-06  0.000000e+00  2.900000e-02
> nstep = 1; t = 1.000000e-04; dt = 1.100000e-04;    0.0 %
Max(MACH) = 2.630800
Max(vc) = 1.500000e+00  1.206000e+00  1.206000e+00  0.000000e+00  1.500000e+00
Min(vc) = 1.380000e-01 -1.680204e-05 -1.680204e-05  0.000000e+00  2.900000e-02
> nstep = 2; t = 2.100000e-04; dt = 1.210000e-04;    0.0 %
Max(MACH) = 2.630800
Max(vc) = 1.500000e+00  1.206000e+00  1.206000e+00  0.000000e+00  1.500000e+00
Min(vc) = 1.380000e-01 -4.643808e-05 -4.643808e-05  0.000000e+00  2.900000e-02
...
```

where `nstep` gives the current integration step, `t` is the current integration time, `dt` is the current time step, `x.x%` is the percentage of integration. The `Max(Mach)` is the maximum Mach number during the current step and is it very sensitive function of the numerical method it may be used as a “robustness”

indicator. Very large Mach numbers or rapid variations usually indicate problems and/or fixes during the computation. The $\text{Max}(\text{vc})$ (and Min) are the maximum (and minimum) primitive cell center variables (density, v_x , v_y , v_z , pressure for the HD example shown) values of the whole grid for the current step.

Parallel mode output log. In parallel mode, no output is dumped on screen. Instead, each processor can write to a dedicated log file, `pluto.n.log`, where n is the processor number. If the macro `MULTIPLE_LOG_FILES` is set to `NO` (default) only `pluto.0.log` will be created by processor #0 on a regular execution using the `print()` function. Other processors output is suppressed unless an error occurs (in this case the `printLog()` function is used instead). Conversely, setting `MULTIPLE_LOG_FILES` to `YES` will cause all processors to write individual log files on a regular execution. Since computations on many cores produce the same number of log files, it is possible to write them into a dedicated directory if desired (use the `log_dir` in `pluto.ini`, see §4.6);

Notice that if a log file does not exist it will be created; if the file exists but integration starts from initial conditions, it will be over written. Finally, if you restart from a previously saved file, the output will be appended.

Note: When `MULTIPLE_LOG_FILES` is disabled (default), the `print()` function works only for processor #0. If output is required from any other processor, then the `printLog()` function should be used instead.

1.4.2 Command line options

When running **gPLUTO**, a number of command-line switches can be given to enable or disable certain features at run time. See Table 1.3 for a description of the available flags.

1.5 Modifying the Distribution Source Files

gPLUTO source files are compiled directly from the `gPLUTO/Src` directory. Should you need to modify a C++ source file other than your `init.cpp`, we strongly advise to copy the file to your local working directory and then edit it, since the latter is always searched before `PLUTO/Src` during the compilation phase. In other words, if you want to modify say, `boundary.cpp`, copy the file to your working area and introduce the appropriate changes. When `make` is invoked, your local copy of `boundary.cpp` is compiled since it has priority over `PLUTO/Src/boundary.cpp` which is actually ignored. In such a way, you can keep track of the problem dependent modification, without affecting the original distribution.

Note: Header files (`*.hpp`) do not follow the same convention and *must not* be copied to the local working directory. Modifications to header files must therefore be done in the original directory.

Option	Description
<code>-dec n1 [n2] [n3]</code>	Enable user-defined parallel decomposition mode. The integers <code>n1</code> , <code>n2</code> , and <code>n3</code> specify the number of processors along the <code>x1</code> , <code>x2</code> , and <code>x3</code> directions. The number of integers must match the number of dimensions, and their product must equal the total number of processors used by ‘ <code>mpirun</code> ’, or an error will occur.
<code>-frestart n</code>	Same as <code>restart</code> (see below) but for the fluid only (particle restart is suppressed).
<code>-i fname</code>	Use <code>fname</code> as the initialization file instead of <code>pluto.ini</code> .
<code>-h5restart n</code>	Restart computations from the <code>n</code> -th output file in HDF5 double precision format (<code>.dbl.h5</code> , only for static grids). The input data files are read from the directory specified by the <code>output_dir</code> variable in <code>pluto.ini</code> (default is the current working directory).
<code>-makegrid</code>	Generate grid only; do not start computations.
<code>-maxsteps n</code>	Stop computations after ‘ <code>n</code> ’ steps.
<code>-no-write</code>	Do not write data to disk.
<code>-no-x1par,</code> <code>-no-x2par,</code> <code>-no-x3par</code>	Do not perform parallel domain decomposition along the <code>x1</code> , <code>x2</code> , or <code>x3</code> direction, respectively.
<code>-restart n</code>	Restart computations from the <code>n</code> -th output file in double precision format (<code>.dbl</code> , for static grids). Input data files are read from the directory specified by the <code>output_dir</code> variable in <code>pluto.ini</code> (default is the current working directory).
<code>-x1jet,</code> <code>-x2jet, -x3jet</code>	Exclude from integration regions of zero pressure gradient that extends up to the end of the domain in the <code>x1</code> , <code>x2</code> , or <code>x3</code> direction, respectively. This option is specifically designed for jets propagating along one of the coordinate axes. In parallel mode, parallel decomposition is not performed along the selected direction.
<code>-xres n1</code>	Set the grid resolution in the <code>x1</code> direction to <code>n1</code> zones, overriding ‘ <code>pluto.ini</code> ’. The cell aspect ratio is preserved by modifying the grid resolution in the other coordinate directions accordingly.

Table 1.3: Command line options available when running **gPLUTO**. †: applicable only on parallel architectures.

1.6 GPU Programming Models

GPU acceleration on **gPLUTO** is based on programming models heavily relying on `#pragma` directives. Both OpenACC and OpenMP programming models are supported.

1.6.1 OpenACC Features

OpenACC is the programming model that **gPLUTO** uses to run on NVIDIA GPUs. It relies on information provided by the programmer to the compiler, mainly through directives that apply to the code blocks following them.

OpenACC directives, also called pragmas, are specified using the keyword `pragma`, followed by the `acc` sentinel, the directive itself, and optional clauses:

```
#pragma acc <directive> <clause,clause>
```

In **gPLUTO**, the standard pragma used to accelerate a loop is `parallel loop`. It tells the compiler that the loop is both safe and beneficial to parallelise. The `loop` directive, when used on its own, assures the compiler that the iterations are independent, meaning that no iteration modifies data used by another. This directive is typically applied to inner loops inside a `parallel loop`.

For tightly nested loops, the `collapse` clause is useful. It merges a nested loop structure into a single loop, which can improve the efficiency of parallelisation.

When an OpenACC compiler parallelises a region of code, it analyses the data required inside that region and transfers it between host and device memory as needed. The `present` clause can be added to a loop construct to specify that a variable is already in GPU memory, avoiding unnecessary data transfers.

```
#pragma acc parallel loop collapse(3) present(A, box)
BOX_LOOP(&box,i,j,k){
    A(i,j,k) = ...
}
```

Other useful clauses of the `parallel loop` pragma include:

- `reduction`, which performs reduction operations on variables across parallel threads;
- `private`, which declares variables private to each parallel thread;
- `async`, which launches the loop on a specific instruction stream, enabling asynchronous execution with respect to other kernels. Asynchronous kernels are typically paired with the `wait` directive, which synchronises the host with ongoing asynchronous operations on the device.

When a function call appears inside a kernel, the compiler needs instructions on how to handle it. This is provided by the `routine` directive, used with clauses such as `vector` or `seq`.

OpenACC with Unified Memory (see §3.1) is able to handle may data copy implicitly but we provide the compiler with additional information using data directives to manage memory more efficiently, keeping data local to the accelerator as long as possible and only moving it when absolutely necessary. OpenACC provides two types of data directives: structured and unstructured.

```
#pragma acc data copy(A)
{
    ...
}
```

The pragma above defines a structured data region, where the lifetime of the data is bounded by curly brackets. Within this region, the variable `A` is available on the device. The `copy` clause copies `A` from host to device at the start of the region and back to the host at the end.

Unstructured data directives can achieve the same effect using pairs of pragmas:

```
#pragma acc enter data copyin(A)
...
#pragma acc exit data copyout(A)
```

Here, `copyin` transfers `A` to the GPU, while `copyout` brings it back to the CPU. Together, they act like `copy`. In addition to `copy`, `copyin`, and `copyout`, the clauses `create` and `delete` are available to allocate and free device memory for a variable without implying any data transfer.

The update directives synchronise host (CPU) and device (GPU) memory explicitly:

```
#pragma acc update device(A)
```

and

```
#pragma acc update host(A)
```

The first updates the variable `A` in GPU memory by copying it from the host, while the second updates host memory by copying it back from the device.

Make it clear when you updated a structure member ofr instance

1.6.2 OpenMP Features

...

2. Problem Header File: *definitions.hpp*

This chapter explains how to create the configuration header file `definitions.hpp` for a specific problem.

2.1 Basic Options

The header file `definitions.hpp` is created by the Python script `setup.py` by selecting *Setup problem* (see Fig. 2.1). If you do not have an existing `definitions.hpp`, a new one will be created for you, otherwise the Python script will try to read your current setup from the existing one.

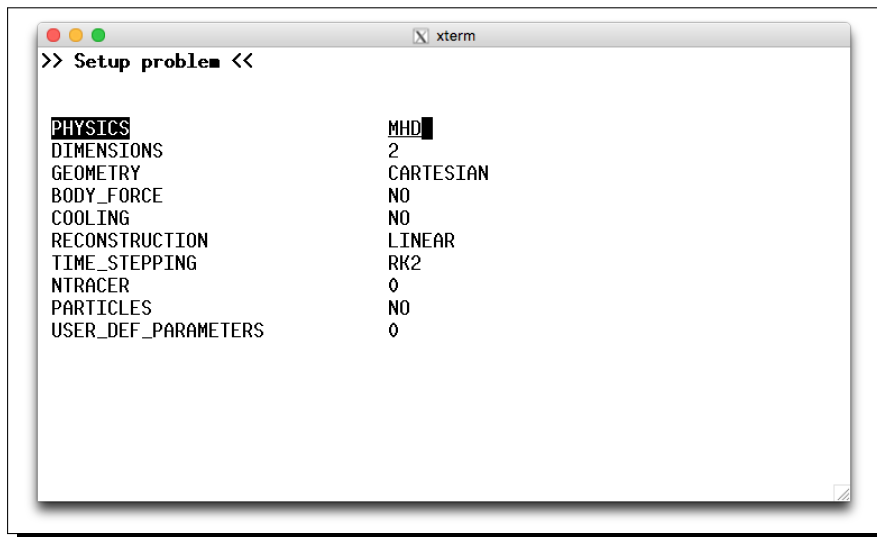


Figure 2.1: The Setup problem menu, needed for your `definitions.hpp` and makefile creation; by moving the arrow keys you should be able to browse through different options.

The header file `definitions.hpp` also contains other more advanced options (*user-defined constants*) that are not accessible via the Python script and should be changed manually (§2.3). We now describe the options accessible through the Python script.

2.1.1 PHYSICS

Specifies the fluid equations to be solved. The available options are:

- *HD*: classical hydrodynamics described by the Euler equations, §6.1;
- *MHD*: single fluid, ideal/resistive magnetohydrodynamics, §6.2;
- *RHD*: special relativistic hydrodynamics, §6.3;
- *RMHD*: special relativistic magnetohydrodynamics, §6.4.
- *ResRMHD*: special relativistic magnetohydrodynamics with resistivity, §6.5.

2.1.2 DIMENSIONS

`DIMENSIONS` sets the number of spatial dimensions of your problem. The number of vector components is always 3 (e.g. even in 1D, we evolve v_x , v_y and v_z). In addition, it is now possible to exclude the intermediate dimension (x_2) by setting the macro `INCLUDE_JDIR` to *NO* in your `definitions.hpp`. This feature

can be used, for instance, in axisymmetric calculations in the (r, z) coordinates in polar geometry by excluding the ϕ coordinate, or for 2D Cartesian simulation in the $x - z$ Cartesian plane.

2.1.3 GEOMETRY

Sets the geometry of the problem. Spatial coordinates are generically labeled with x_1 , x_2 and x_3 and their physical meaning depends on the value assigned to `GEOMETRY`:

- *CARTESIAN*: Cartesian coordinates $\{x_1, x_2, x_3\} = \{x, y, z\}$;
- *CYLINDRICAL*: Cylindrical coordinates $\{x_1, x_2, x_3\} = \{r, \phi, z\}$;

Note: Axisymmetric simulations can be carried out by selecting 3 dimensions and excluding integration in the ϕ -direction by setting `INCLUDE_JDIR` to `NO` in your definitions.hpp.

- *SPHERICAL*: Spherical coordinates $\{x_1, x_2, x_3\} = \{r, \theta, \phi\}$.

Note that when `DIMENSIONS = 2`, the third coordinate x_3 is meaningless and will be set to zero (similarly in 1-D x_2 and x_3 do not play any role). The corresponding vector component is, however, integrated regularly (e.g. v_z in 2D Cartesian coordinates).

Singularity Treatment. The clustering of cells in the azimuthal direction near the pole/axis imposes a severe time-step restriction. This takes places at singular point: $R = 0$ in cylindrical coordinates or $\theta = 0, \pi$ in spherical coordinates. In polar or spherical coordinates, in fact, the CFL condition requires

$$\Delta t \lesssim \frac{r \Delta \phi}{\lambda_{\max}},$$

so that near the axis, where $r \sim \Delta r$, one obtains

$$\Delta t \sim \Delta r \Delta \phi \sim \mathcal{O}(N^{-2}),$$

instead of the usual hyperbolic scaling $\Delta t \sim \mathcal{O}(N^{-1})$ (here N denotes the characteristic number of zones per direction).

To overcome this limitation, **gPLUTO** implements the ring average technique [ZSL⁺19] which employs a conservative averaging–reconstruction strategy on cylindrical or spherical meshes. It can be activated by defining the macro `RING_AVERAGE` to a power of two. By default, `RING_AVERAGE` specifies the de-refinement ratio of the innermost azimuthal ring. Consecutive rings are then progressively refined by a factor of two until the last ring matches the resolution of the original grid. The ring-average module also supports a number of tuning macros, see §B.3.

The correct boundary condition to be applied is “*singular_axis*” (§4.5). The ring average technique works with either cell-center or staggered mesh algorithms. At present, parallelization in the azimuthal direction is not possible.

2.1.4 MAPPED_GRID

Mapped grids provide a convenient way to transform the physical domain onto a simpler computational (or logical) domain. The logical coordinates ξ_1 , ξ_2 and ξ_3 are always defined in $[0, 1]$ and the transformation between physical and logical coordinates is assumed to be univariate, that is, $x_1 = x(\xi_1)$, $x_2 = x(\xi_2)$ and $x_3 = x(\xi_3)$. Using mapped grid has the advantage, for example, that the reconstruction step is carried on the logical grid where the spacing is uniform.

With `MAPPED_GRID` enabled, computational zones may be distributed non-uniformly by incorporating the `MappedGrid()` function into your `init.cpp` and defining an appropriate grid transformation. The leftmost and rightmost points of the physical grid as well as the total number of computational zones are still usually defined by `pluto.ini` (see §4.1) and the transformation $x_d = x_d(\xi_d)$ with $d = 1, 2, 3$ is specified through this function:

```
double MappedGrid (Grid *grid, int dir, double xi)
{
    double x;

    if (dir == IDIR) return MappedGrid_<TYPE1>(grid, dir, xi, <PAR>);
    if (dir == JDIR) return MappedGrid_<TYPE2>(grid, dir, xi, <PAR>);
    if (dir == KDIR) return MappedGrid_<TYPE3>(grid, dir, xi, <PAR>);
}
```

For each direction $\langle DIR \rangle$, **gPLUTO** provides a number of pre-defined functions `MappedGrid_<TYPE $n$ >()` with different number of arguments, depending on the map. While the first 3 arguments are always the same, some grid mappings require additional parameters ($\langle PAR \rangle$). Let x_b and x_e be the leftmost and rightmost domain interface (as defined in §4.1). Note also that *all dimensions should return a valid map*. The pre-defined transformations are:

- `MappedGrid_Uniform (grid, dir, xi) // No free parameters`



This function has no free-parameters and it creates a (constant-spacing) grid transformation,

$$x(\xi) = x_b + \xi(x_e - x_b). \quad (2.1)$$

- `MappedGrid_Sin (grid, dir, xi) // No free parameters`



The sinusoidal map has no free parameters and it is used mainly for testing:

$$x(\xi) = x_b + \frac{x_e - x_b}{2} \left(2\xi + \frac{\sin(2\pi\xi)}{2\pi} \right). \quad (2.2)$$

- `MappedGrid_Sinh (grid, dir, xi, a) // 1 parameter (a)`



The hyperbolic sine transformation is a classic choice for grid clustering, especially near some central region while keeping smooth variations. It is similar to the previous transformation, but more flexible.

The centering of clustering is $x_c = (x_b + x_e)/2$ while the parameter controlling the clustering strength is $a > 0$. Larger values of a mean tighter clustering:

$$x(\xi) = x_c + \frac{x_e - x_b}{2} \frac{\sinh[a(2\xi - 1)]}{\sinh(a)}. \quad (2.3)$$

- `MappedGrid_Stretched (grid, dir, xi, r) // 1 parameter (r)`



Constructs a geometrically stretched grid with constant stretching ratio $r = \Delta x_{i+1}/\Delta x_i$:

$$x(\xi) = x_b + (x_e - x_b) \frac{r^{N\xi} - 1}{r^N - 1}. \quad (2.4)$$

This transformation requires one additional parameter (the stretching ratio r).

- `MappedGrid_Exp (grid, dir, xi) // No free parameters`



Constructs an logarithmic grid such that $\Delta r/r = C$ where C is constant.

$$x(\xi) = Ae^{C\xi} \implies x(\xi) = x_b \left(\frac{x_e}{x_b} \right)^\xi \quad (2.5)$$

In *CYLINDRICAL* or *SPHERICAL* coordinates, this has the advantage of preserving the cell aspect ratio at any distance from the origin.



- `MappedGrid_US (grid, dir, xi, q, xq) // 2 parameters (q, xq)`
Creates a composite grid with a uniform patch followed by geometrically stretched one:

$$x(\xi) = \begin{cases} x_b + (x_q - x_b) \frac{\xi}{q} & \text{for } 0 \leq \xi \leq q \\ x_q + (x_e - x_q) \frac{r^{(\xi-q)/(1-q)} - 1}{r - 1} & \text{for } q \leq \xi \leq 1 \end{cases} \quad (2.6)$$

The stretching factor (function of r) is determined by imposing continuity of the transformation at $\xi = q$. This map has two free parameters: q (the fraction of total number of points that goes into the uniform patch) and $x_q = x(q)$ (the physical coordinate at which the grid changes).

- `MappedGrid_SUS (grid, dir, xi, qL, xqL, qR, xqR) // 4 parameters (q1, xq1, q2, xq2)`



Similarly to the `MappedGrid_US()`, this transformation allows the user to frame the grid using 3 patches: a stretched one, followed by a uniform one and then again by a stretched one. The transformation is given in terms of four parameters: q_1, x_{q1} (for the left patch) and q_2, x_{q2} (for the right patch):

$$x(\xi) = \begin{cases} x_{q1} - (x_{q1} - x_b) \frac{r_1^{(q1-\xi)/q1} - 1}{r_1 - 1} & \text{for } 0 < \xi < q_1, \\ x_{q1} + (x_{q2} - x_{q1}) \frac{\xi - q_1}{q_2 - q_1} & \text{for } q_1 < \xi < q_2, \\ x_{q2} + (x_e - x_{q2}) \frac{r_2^{(\xi-q2)/(1-q2)} - 1}{r_2 - 1} & \text{for } q_2 < \xi < 1. \end{cases} \quad (2.7)$$

The two stretching factors r_1 and r_2 are determined by imposing continuity of the map in q_1 and q_2 . Notice that $q_2 - q_1$ gives the fraction of total number of points falling in the central uniform grid.

Examples of mapped grids are found in configurations #8 of HD/Istentropic.Vortex/ and #8 of MHD/Rotor/. Beware that non-uniform grids may introduce extra dissipation in the algorithm. Also, for periodic domain, it is recommended to enforce periodicity by enabling the macro `MAPPED_GRID_FORCE_PERIODS` (see §B.3).

2.1.5 BODY_FORCE

Include a body force in the momentum and energy equations. Possible values are:

- *POTENTIAL*: body force is derived from a scalar potential, $\rho \mathbf{a} = -\rho \nabla \Phi$;
- *VECTOR*: body force is expressed as a three-component vector $\rho \mathbf{a} = \rho \mathbf{g}$.
- (*VECTOR+POTENTIAL*): body force is prescribed using both, $\rho \mathbf{a} = \rho(-\nabla \Phi + \mathbf{g})$.

More details can be found in §5.3.

2.1.6 COOLING

Optically thin thermal losses can be included by appropriately setting this flag to one of the following:

- *TABULATED*: radiative losses are computed as $n^2\Lambda(T)$, where $\Lambda(T)$ is a user-supplied tabulated function of temperature, see §8.1. Alternatively, this module can be used to provide user-defined cooling functions;
- *SNEq*: simplified non-equilibrium cooling function for atomic hydrogen. See §8.2 for more details;

2.1.7 RECONSTRUCTION

Sets the spatial order of the finite volume method. The following options are available:

- *FLAT*: first order reconstruction. The stencil is 1 point.
- *LINEAR*: Piecewise TVD linear reconstruction is applied to primitive variables. It is 2nd order accurate in space. Stencil is 3 point wide.
- *PARABOLIC*: Piecewise parabolic reconstruction as implemented by [Mig14]. The stencil requires either 3 or 5 zones (see PPM_ORDER in §B.3).
- *Lim03*: 3rd-order reconstruction limiter function [ČT09] based on a 3-point stencil. This reconstruction avoids clipping at extrema.
- *WENO3*: 3rd-order weighted essentially non-oscillatory reconstruction [YC09] inside a cell using is 3-point stencil.
- *WENOZ*: 5rd-order weighted essentially non-oscillatory reconstruction [BCCD08] inside a cell with a 5-point stencil.
- *MP5*: 5th-order monotonicity preserving scheme of [SH97].

The default is *LINEAR*. Note that *LINEAR* and *PARABOLIC* reconstruction suffer from clipping at extrema. On the contrary *WENO3* and *Lim03* employ a local three-point stencil to achieve piecewise-quadratic reconstruction for smooth data and preserves their accuracy at local extrema thus avoiding clipping of classical second-order TVD limiters.

We point out that, although 3rd-order reconstructions are available, the code retains a global 2nd-order accuracy as fluxes are computed at the interface midpoint (unless the high-order methods are selected).

2.1.8 TIME STEPPING

If $\Delta t^n = t^{n+1} - t^n$ is the time increment between two consecutive steps and $\mathcal{L}$ denotes the discretized spatial operator on the right hand side of Eq. (1.1), the possible options are:

- *EULER*: 1st (explicit) Euler algorithm is used to evolve from $\mathbf{U}^n$ to $\mathbf{U}^{n+1}$:

$$\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t^n \mathcal{L}^n$$

- *RK2*: 2nd-order TVD Runge Kutta is used to advance the solution to the next time level:

$$\begin{aligned} \mathbf{U}^{(1)} &= \mathbf{U}^n + \Delta t^n \mathcal{L}^n \\ \mathbf{U}^{n+1} &= \frac{1}{2} \left(\mathbf{U}^n + \mathbf{U}^{(1)} + \Delta t^n \mathcal{L}^{(1)} \right) \end{aligned} \quad (2.8)$$

- *RK3*: 3rd-order TVD Runge Kutta is used to advance the solution to the next time level:

$$\begin{aligned} \mathbf{U}^{(1)} &= \mathbf{U}^n + \Delta t^n \mathcal{L}^n \\ \mathbf{U}^{(2)} &= \frac{1}{4} \left(3\mathbf{U}^n + \mathbf{U}^{(1)} + \Delta t^n \mathcal{L}^{(1)} \right) \\ \mathbf{U}^{n+1} &= \frac{1}{3} \left(\mathbf{U}^n + 2\mathbf{U}^{(2)} + 2\Delta t^n \mathcal{L}^{(2)} \right) \end{aligned} \quad (2.9)$$

The right hand side include contributions from all directions simultaneously.

Time Step Determination. The time step Δt^n is computed using the information available from the previous integration step and it can be controlled by the Courant-Friedrichs-Lewy (CFL) number C_a within the limits, see [Bec92]. If Δl is the cell physical length, the time step roughly scales as $\sim \Delta l$ for hyperbolic problems and as $\sim \Delta l^2$ when parabolic terms are included via a standard explicit scheme (§??).

$$\frac{\Delta t^n}{N_{\text{dim}}} \left[\max_{ijk} \left(\sum_d \frac{|\lambda_d|}{\Delta l_d} \right) + \max_{ijk} \left(\sum_d \frac{2\tau_d}{\Delta l_d^2} \right) \right] = C_a \leq \frac{1}{N_{\text{dim}}} \quad (2.10)$$

For a given direction d , Δl_d represents the cell physical length in that direction, λ_d provides the largest signal speed while τ_d accounts for diffusion processes. These limits are based on a stability analysis for a 1st-order scheme on the constant coefficient advection-diffusion equation by Beckers (1992), [Bec92].

2.1.9 NTRACER

The number of passive scalars or “colors” (denoted with Q_k) obeying simple advection equations of the form:

$$\frac{\partial Q_k}{\partial t} + v \cdot \nabla Q_k = 0 \quad \Longleftrightarrow \quad \frac{\partial(\rho Q_k)}{\partial t} + \nabla \cdot (\rho Q_k v) = 0 \quad (2.11)$$

The second form gives the conservative equation and it is the one actually being discretized by the code. The array index used to access tracer variables (§5.1, §5.2) is TRC for the first tracer, TRC+1 for the second one and so on. The maximum number is 8.

2.1.10 PARTICLES

Used to enable one of the available particle modules (Ch. 10). Available options are:

- *NO*: particles are not enabled
- *PARTICLES_LP*: enable the lagrangian particle module, §10.2;

Note that this module works only with the static version of the code.

2.1.11 USER_DEF_PARAMETERS

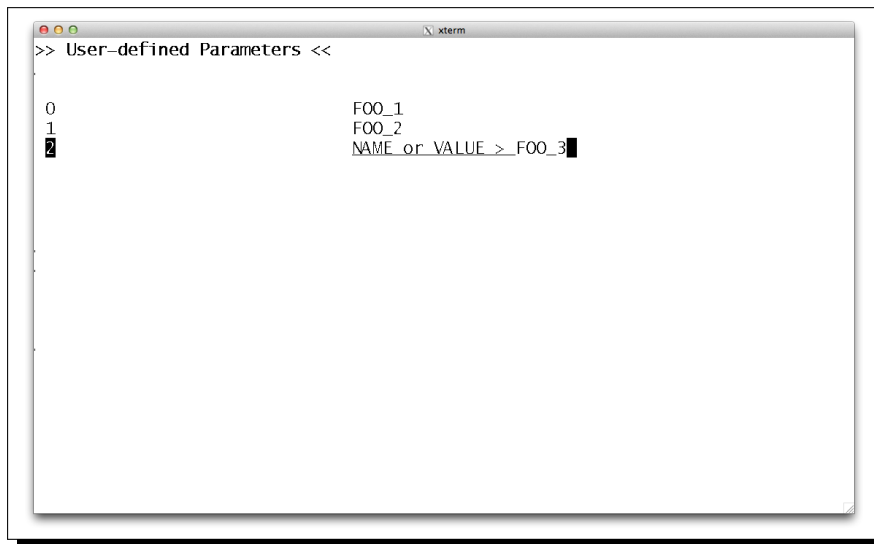


Figure 2.2: User-defined parameter names are chosen in this sub-menu.

Sets the number of user-defined parameters that can be changed at runtime and accessed from anywhere in the code. The explicit numerical value is read at runtime from `pluto.ini` and can be changed before execution without re-compiling the code. The parameters are identified by means of a label corresponding to an index of the structure member array `run_config.inputParam` available almost anywhere in the program. If, for instance, `USER_DEF_PARAMETERS` has been set equal to 3, you will be

prompted to define 3 different “labels”, say F00_1, F00_2 and F00_3, as in Fig. 2.2. These names are the integer indexes of the `run_config.inputParam` array: `run_config.inputParam[F00_1]` will contain the actual value of the first user-defined parameter, `run_config.inputParam[F00_2]` the second one and so forth.

The maximum number is 31 and parameter names should be chosen with care in order to avoid overlapping conflicts with names that are already defined in the code. Although there are no strict rules, we strongly advise to use capital letters, to avoid short labels such as “V0” or “VX” and to choose a more representative name that explains the use of the variable on its own, e.g., `PAR_INFLOW_VEL`.

Parameter names (and values) are automatically inserted inside `pluto.ini` in the correct order after the execution of the python script. However, if you use a different initialization file than `pluto.ini`, you may have to set the parameter names together with their values manually.

2.2 Physics-Dependent Options

After the physics module has been selected, further options become available in a following menu. Depending on the selected physics module, different options may appear. They are described in the following.

2.2.1 BACKGROUND_FIELD [MHD]

If set to *YES*, the magnetic field is split into a static curl-free contribution and a time dependent deviation. This option is only available for the *MHD* module and it is described in detail in §6.2.3.

2.2.2 DIVB_CONTROL [MHD, RMHD, ResRMHD]

Numerical methods do not naturally preserve the condition $\nabla \cdot \mathbf{B} = 0$. For this reason, this option allows the user to select a control strategy to enforce the $\nabla \cdot \mathbf{B} = 0$ constraint. Possible values are

- *NO*
divergence constraint is not enforced. Recommended for one-dimensional problems or 2D configurations with purely azimuthal fields.
- *EIGHT_WAVES*
magnetic fields retain a cell average representation and the eight wave formulation introduced by Powell [Pow94] is used, see §6.2.2.1;
- *DIV_CLEANING*
magnetic fields retain a cell average representation and the mixed hyperbolic/parabolic divergence cleaning technique of [DKK⁺02, MTB10] is used, see §6.2.2.2. A new scalar variable, the generalized Lagrange multiplier ψ (`PSI_GLM`) is introduced.
- *CONSTRAINED_TRANSPORT*
the magnetic field has a staggered representation and the constrained transport is used, see §6.2.2.3.

2.2.3 EOS

Select the equation of state (EOS):

- *IDEAL*
use the perfect gas law with constant specific heat ratio. This EOS is available for all physics module and it is described in §7.2.
- *ISOTHERMAL*
use an isothermal equation of state (§7.1). In this case, the energy equation is not included.
- *TAUB*
use the Taub-Matthews equation of state (only for relativistic modules), see §7.3.

Please refer to Chapter 7 for a detailed description.

2.2.4 ENTROPY_SWITCH

By enabling this switch (only for the *IDEAL* or *TAUB* EOS), the equation of entropy is added to the system of conservation laws:

$$\frac{\partial \sigma_c}{\partial t} + \nabla \cdot (\sigma_c \mathbf{v}) = S_\sigma \quad (2.12)$$

where $\sigma_c = \rho \sigma$ (for *HD* or *MHD*) or $\sigma_c = D\sigma$ (for relativistic flows), σ is the entropy (e.g., $\sigma = p/\rho^\Gamma$ for the *IDEAL* EOS) and S_σ is a source term accounting for dissipative terms in the HD or MHD modules (see Eq. [3.27] of [BS03]):

$$S_\sigma = (\Gamma - 1)\rho^{1-\Gamma} \left[\nabla \cdot (\kappa \nabla T) + \eta \mathbf{J}^2 + \mu \Pi_{ij} \frac{\partial v_i}{\partial x_j} \right]. \quad (2.13)$$

Equation (2.12) is solved in a conservative way and entropy is essentially treated as a passive scalar. However, the source term (2.13) is never actually included since the total energy equation is *always* employed when adding parabolic terms in a conservative fashion, see the documentation of `ParabolicUpdate()`. This allows the entropy switch to be used with any of the nonideal terms described in Chapter ??.

The entropy switch acts in a selective way allowing gas pressure to be recovered from either the total energy or the entropy:

$$\begin{cases} p = p(E) & \text{if } \mathcal{F}_\sigma = 0 \\ p = p(\sigma_c) & \text{if } \mathcal{F}_\sigma = 1 \end{cases} \quad (2.14)$$

Here $\mathcal{F}_\sigma$ tells if a zone has been flagged with `FLAG_ENTROPY` at the beginning of the step and it depends on the value assigned to `ENTROPY_SWITCH`:

- *NO*
the entropy equation is not included ($\mathcal{F}_\sigma = 0$ everywhere);
- *SELECTIVE*
a selective update is employed. By default, all zones are flagged to be updated using the entropy equation unless they lie in proximity of a shock wave. Thus for each zone, we evaluate

$$\mathcal{F}_\sigma = \begin{cases} 0 & \text{if } \tilde{\nabla} \cdot \mathbf{v} < 0 \text{ and } \|\tilde{\nabla} p\|/p > \epsilon_p \\ 1 & \text{otherwise,} \end{cases} \quad (2.15)$$

where $\tilde{\nabla}$ is a three-point undivided difference operator and ϵ_p sets the shock strength threshold (see `EPS_PSHOCK_ENTROPY` in Appendix B.3). The first criterion acts as shock detector. The implementation can be found in the source file `Src/flag_shock.c`.

Note that by enabling this selective update, neither the total energy nor the entropy will generally be conserved at the numerical level.

- *ALWAYS*
the entropy equation is used everywhere in the computational domain to update the solution array, i.e., $\mathcal{F}_\sigma = 1$ always in Eq. (2.14). This choice is consistent only with smooth flows (no shocks).

The `ENTROPY_SWITCH` is also compatible with super-time-stepping although it will only be used during the hydro steps.

2.2.5 ROTATING_FRAME [HD, MHD]

When set to *YES*, it solves the equations in a frame of reference rotating with constant angular velocity Ω_z around the vertical polar axis z . This feature should be enabled only when `GEOMETRY` is either *CYLINDRICAL* or *SPHERICAL*. The value of Ω_z is specified using the structure member `run_config.OmegaZ` inside your `Init()` function. The discretization of the angular momentum and energy equations is then

done in a conservative fashion [Kle98, MFS⁺12]. For example, in cylindrical geometry, we solve

$$\begin{aligned}
 \frac{\partial}{\partial t}(\rho v_R) + \nabla \cdot (\rho v_R \mathbf{v}) + \frac{\partial p}{\partial R} &= \frac{\rho(v_\phi + w_\phi)^2}{R} \\
 \frac{\partial}{\partial t}[R\rho(v_\phi + w_\phi)] + \nabla \cdot [R(\rho v_\phi + \rho w_\phi)\mathbf{v} + R p \hat{\mathbf{e}}_\phi] &= 0 \\
 \frac{\partial}{\partial t}\left(E + \frac{w_\phi^2}{2}\rho + w_\phi \rho v_\phi\right) + \nabla \cdot \left[F_E + p w \hat{\mathbf{e}}_\phi + \frac{w_\phi^2}{2}\rho \mathbf{v} + w_\phi \rho v_\phi \mathbf{v}\right] &= 0
 \end{aligned} \tag{2.16}$$

where $w_\phi = R\Omega_z$, R is the cylindrical radius and F_E is the standard energy flux and body force terms have been omitted only for the sake of exposition. Terms under temporal derivatives are replaced, in the code, with the corresponding flux divergence terms only during the r -sweep (in cylindrical coordinates) or only during the r -sweep and θ -sweep (in spherical coordinates). During the ϕ -sweep, in fact, derivatives cancel out and no additional term is present (the pressure term inside the divergence of the energy equation cancels with the $w\partial_\phi p/R$ term coming from the temporal derivative).

Note that the source term in the radial component of the momentum equation implicitly contains the Coriolis force and centrifugal terms:

$$\frac{\rho(v_\phi + w_\phi)^2}{R} = \frac{\rho v_\phi^2}{R} + 2\rho v_\phi \Omega_z + \rho \Omega_z^2 R \tag{2.17}$$

On the other hand, the azimuthal component of the Coriolis force has been incorporated directly into the fluxes using the conservation form. An example of such a configuration in polar or spherical geometry may be found in the directory `Test_Problems/HD/Disk_Planet`.

2.2.6 RADIATION [HD, MHD, RHD, RMHD]

If set to *YES*, the RADIATION module is used (see §9.2).

2.3 User-defined Constants

In addition to the options described so far, the user can insert an arbitrary number of user-defined symbolic constants (macros) in the header file `definitions.hpp`. This should be done manually in the section delimited by

```
/* [Beg] user-defined constants (do not change this line) */

/* [End] user-defined constants (do not change this line) */
```

of this file. Only lines beginning with `#define` should appear in this section as they will not be changed by the python script. The value of a symbolic constant can be either a number or another symbolic constant previously defined by the code (e.g. *YES* or *NO*) and cannot be changed at runtime.

User-defined symbolic constants are useful in the following circumstances:

1. Conditional compilation: useful when your initial configuration contains computationally expensive code blocks that should be compiled separately. As an example, define (in your `definitions.hpp`) the symbolic constant name as `SETUP_VERSION` and give it the value of 0 or 1:

```
/* [Beg] user-defined constants (do not change this line) */

#define SETUP_VERSION 1

/* [End] user-defined constants (do not change this line) */
```

This symbolic macro name can then be used inside `init.c` (or any other source file) for conditional compilation:

```
#if SETUP_VERSION == 0
{
    /* implements version 0... */
}
#elif SETUP_VERSION == 1
```

```
{
    /* implements version 1... */
}
#endif
```

2. **Advanced options** (expert users): override the default value of special constant macros used throughout the code, a comprehensive list of which is given in Appendix B.3. This gives the user more control on the code and it avoids copying and modifying source files in the local working directory.

A simple example is given by configuration #04 in the Test_Problems/MHD/Rayleigh_Taylor/ problem where the symbolic constants

```
/* [Beg] user-defined constants (do not change this line) */
#define USE_RANDOM_PERTURBATION NO
/* [End] user-defined constants (do not change this line) */
```

are be used in init.c to enable/disable a random perturbation. Another typical example is provided by the CGS physical units described §5.1.1.

2.4 Fallback Strategies: Making Your Simulation More Robust

gPLUTO provides a number of workarounds in order to prevent or cure situations in which the numerical integration may produce unphysical states (e.g. negative density or pressure, or failure of the conservative-to-primitive variable conversion). A typical situation is encountered, for instance, in strongly supersonic and/or magnetized flows since the pressure is recovered by subtracting kinetic and magnetic contributions from the total energy density, e.g.,

$$p = E - E_{\text{kin}} - E_{\text{mag}}.$$

Discretization errors can then become large enough to produce negative values during this subtraction.

Default Behaviour. By default, **gPLUTO** resets the pressure to `run_config.small_prs` (which can be set in your `Init()` function) and redefines the total energy afterwards. While this is a practical fix, it breaks total energy conservation. Likewise, the code redefines the conserved density to `run_config.small_rho` when density positivity is violated locally. However, this may easily lead to the occurrence of high velocities and severe code crashes due to negative pressures afterwards.

Increasing Numerical Diffusion. A better workarounds is choosing a more diffusive numerical scheme (employing, for instance the HLL or TVDLF Riemann solvers and lowering the spatial accuracy): this turns out to be quite effective at the cost of introducing more numerical dissipation indiscriminately.

An adaptive strategy that may prevent the occurrence of unphysical values can be enables `SHOCK_FLATTENING` to `MULTID`, which relies on a multi-dimensional shock detection algorithm to: i) lower the reconstruction by reverting to the `MINMOD` limiter and ii) replace the Riemann solver at all zone interfaces with the HLL Riemann solver. The shock detection and flag strategy is set in `flag_shock.cpp`. The `MULTID` shock flattening has proven to be an effective adaptation strategy that can noticeably increase the code robustness. It is highly suggested for complex flow structures involving strong shocks. By making of local copy of `flag_shock.cpp`, you can easily add other conditions for order reduction (e.g. regions with small plasma β or geometrically-based adaptivity).

Solving the Entropy Equation. For kinetically- or magnetically-dominated flows containing no shocks, one may also resort to the entropy equation, as described in §2.2.4. This option, however, does not conserve total energy.

Fallback Mode. The Fallback approach is designed to cure situations in which unphysical states may be produced during numerical integration. They act a posteriori, i.e. after a candidate update has been computed, and are triggered whenever an error is detected. The fallback behavior is selected at compile time by introducing the macro `FALLBACK_MODE` in your `definitions.hpp` and set its value to one of the following:

- **FALLBACK_RECOMPUTE**: Whenever one or more computational zones are flagged as *FLAG_FALLBACK*, the current time step is *discarded* and recomputed using a more robust (and diffusive) numerical setup in proximity of the flagged zones. In particular, the integration is repeated starting from the previous time level, typically enforcing:
 - linear (minmod) or 1st-order (piecewise constant) spatial reconstruction,
 - a more diffusive Riemann solver (e.g. HLL),
 - local buffering around the flagged zones.

This strategy preserves conservation properties by modifying the numerical fluxes used in the update, at the price of increased numerical diffusion in the affected regions. It is the most robust fallback option and is recommended when strict conservation is required. This strategy permits the following tuning parameters:

- **FALLBACK_BUFFER**: controls the number of zones to be added on each side of the flagged zones in all spatial directions. The size of the resulting block is $(1 + 2n_b)^D$, where n_b is the buffer size and D is the number of spatial dimensions. For example, if **FALLBACK_BUFFER** is set to 2, order reduction is applied in a block of 5^D zones centered on the flagged one (the default value is $n_b = 1$).
- **FALLBACK_TTL_MAX**: sets, in number of time steps, how long the fallback policy persists across steps (in other words, its “time to live” or TTL). Default is 2.
- **FALLBACK_SUBSTEPS**: allows the solution to be advanced using smaller time steps (the default value is 1).

Notice that this strategy can become significantly time consuming, typically by a factor that depends on how frequently zones are flagged and if substepping is enabled. A factor ~ 2 is typically expected. Configurations using fallback may be found in the configuration #3 of MHD/Current_Sheet/ and #4 of RHD/Riemann_2D.

3. Makefile Selection: makefile

The makefile contains instructions to compile and link C++ source code files and produce the executable pluto. The Python script creates a new makefile every time you choose *Change makefile* from the menu; otherwise, it automatically updates the existing one after you have finished the problem setup. If you choose to create a new makefile, Python will ask you to select an appropriate .defs file containing architecture-dependent flags from the Config/ directory.

The simplest example is a definition file for a single-processor without any additional library. In this case it suffices to set:

```
CC      = g++
CFLAGS  = -c -O3 -std=c++17
LDFLAGS = -lm

PARALLEL = FALSE # TRUE/FALSE: enable/disable parallel mode
USE_HDF5 = FALSE # TRUE/FALSE: enable/disable support for HDF5 library
USE_PNG  = FALSE # TRUE/FLASE: enable/disable support ofr PNG library
```

where g++ is the name of your C++ compiler (g++, nvc++, mpicxx , etc...), CFLAGS are command line options (such as optimization, search path, etc...) and LDFLAGS contains options to be passed to the linker. The variables PARALLEL can be set to either *TRUE* or *FALSE* to enable or disable parallel mode. When set to *TRUE* the same variable name is passed to **gPLUTO** as a #define directive with value 1.

3.1 GPU Support

3.1.1 OpenACC Support

Executables for GPU execution exploiting OpenACC directives are created selecting .defs file with acc in their name. One is for example the linux.nvc++.acc.defs file:

```
CC      = nvc++
...

CFLAGS  = -c -O -acc -gpu=managed
LDFLAGS = -lm -acc -gpu=managed
...
```

Here -acc enables OpenACC directives and -gpu=managed use CUDA unified memory (managed memory).

3.1.2 OpenMP Support

3.2 MPI Library (Parallel) Support

Parallel executables for running the code on multiple CPUs or GPUs are created by specifying the name of the MPI C++ compiler (e.g. `mpicxx`) and by setting the makefile variable `PARALLEL` to `TRUE` in your `.defs` file:

```
CC      = mpicxx # or similar
...

PARALLEL = TRUE
...

#####
# MPI additional specifications
#####

ifeq ($(strip $(PARALLEL)), TRUE)
endif
```

In this case, you may also modify existing variables or add new ones inside the conditional statement beginning with `ifeq`. When parallel mode is enabled, C++ source code sections that are specific to MPI should be enclosed inside `#ifdef PARALLEL ... #endif` statements.

Note that parallel communication is particularly important to exchange data across processors in the ghost zone layers. This step is done at each Runge-Kutta stage and two implementations are available. The first one (default) employs an asynchronous approach involving non-blocking MPI calls with the 8 (26) neighbours in 2D (3D). The second one relies on a synchronous approach where data is send and received direction-wise using blocking communication. The macro `PARALLEL_COMM_ASYNC` can be used to switch between the two (see B.3).

3.3 HDF5 Library Support

If your system is already configured with serial or parallel HDF5 libraries, you may enable support for HDF5 I/O by turning the makefile variable `USE_HDF5` to `TRUE` inside your `.defs` file. If you do not have HDF5 installed, you may follow the installation guidelines given below.

Once `USE_HDF5` has been set to `TRUE`, add the HDF5 library path to the list of directories to be searched for header files as well as the corresponding linker option for HDF5 library files. Note that different pathnames should be given if you are building **gPLUTO** in serial or parallel mode. These information are supplied using the `INCLUDE_DIRS` and `LDLAGS` variables, respectively:

```
...
USE_HDF5 = TRUE
...

#####
# HDF5 library options
#####

ifeq ($(strip $(USE_HDF5)), TRUE)
# https://www.hdfgroup.org/download-hdf5/
HDF5_DIR      = /usr/local/opt/hdf5
HDF5_LIB      = $(HDF5_DIR)/lib
INCLUDE_DIRS += -I$(HDF5_DIR)/include
LDLAGS        += -L$(HDF5_DIR)/lib \
                -lhdf5_cpp -lhdf5
endif
```

Note: **gPLUTO** uses the HDF5 1.6 API although it may be linked with HDF5 1.8.x without any problem since the `H5_USE_16_API` macro (defined in `hdf5.io.cpp`) forces the library to use HDF5 1.6 macro definitions.

3.3.1 Installing HDF5

The HDF5 library is available from the HDF Group website¹ and it can also be installed through common package managers such as Homebrew, apt, Spack, or Conda. **gPLUTO** can be compiled against

¹<https://www.hdfgroup.org/downloads/hdf5/>

either a serial or a parallel HDF5 installation. The serial version is sufficient for single-process execution, whereas the parallel version is required when using MPI parallel I/O.

On Linux and macOS systems, the recommended approach is to use the package manager provided by the operating system. For example:

```
# Ubuntu/Debian:

> sudo apt install libhdf5-dev

# Homebrew (serial version)

> brew install hdf5

# Homebrew (MPI-enabled version)

> brew install hdf5-mpi
```

Users requiring a custom installation may also build HDF5 from source. Detailed instructions are available from the HDF Group documentation.

Serial and Parallel HDF5. A serial HDF5 installation provides the standard HDF5 API and is suitable for all non-MPI applications. A parallel HDF5 installation is built with MPI support and can be used for both serial and parallel applications. When compiling MPI-enabled versions of gPLUTO, a parallel HDF5 installation is required in order to enable collective I/O operations through MPI. The HDF5 compiler wrapper can be used to verify the installation and configuration:

```
> h5cc -showconfig # C interface
> h5c++ -showconfig # C++ interface
```

For MPI-enabled installations, the command

```
> h5cc -showconfig
```

should report

```
Parallel HDF5: yes
```

Library Paths. When HDF5 is installed through a package manager, compiler and linker paths are usually configured automatically. If manual configuration is required, the installation prefix can be determined using

```
> h5cc -show
```

or, on Homebrew systems,

```
> brew --prefix hdf5
> brew --prefix hdf5-mpi
```

which print the include and library locations needed by the build system.

Users building shared libraries from source may still need to ensure that the corresponding runtime library path is visible to the dynamic linker (for example through `LD_LIBRARY_PATH` on Linux).

3.4 Including Additional Files: `local_make`

Additional (e.g. user defined) files may be added to the standard object list created by Python in your makefile. To this end, create a new file named `local_make` like:

```
OBJ      += myfile.o
HEADERS += myheader.hpp
```

This will instruct make that gPLUTO has to be compiled and linked together with the (user-supplied) file `myfile.cpp` which depends on `myheader.hpp`. This is particularly useful when the user wants to compile and link the code together with supplementary routines contained in external files.

4. Runtime initialization file: *pluto.ini*

At start-up, the code checks for the `pluto.ini` input file (or a different one if the `-i` command flag is given) that contains all the runtime information necessary for the computation. The initialization file controls several options used by the code at runtime, such as grid generation, CFL number, boundary conditions, output type and so forth. A template for this file can be found in the `Src/Templates` directory. The initialization file is divided into different “blocks” enclosed by a pair of square brackets `[...]`. Each block contains a set of labels and corresponding mandatory or optional fields:

```
[Block]
label      ...  fields  ...
label      ...  fields  ...
label      ...  fields  ...
```

Tag labels on the left side identify appropriate field(s) following on the same line and must not be changed.

There’re in total 9 blocks:

- **[Grid]**: controls grid generation;
- **[Time]**: time-stepping options;
- **[Solver]**: select the Riemann solver(s);
- **[Boundary]**: used to specify boundary conditions;
- **[Static Grid Output]**: controls output in the static grid version of **gPLUTO** ;
- **[Particles]**: particle-related options (mainly output);
- **[Parameters]**: used to provide user-defined input parameters;

Block ordering is irrelevant. Runtime parameters can be accessed anywhere in the code through the members of the `Runtime` structure using the function `RuntimeGet()`, e.g.

```
cfl = RuntimeGet()->cfl; /* Obtain the cfl number */
char fname[64];
sprintf (fname, "%s/mydata.dat", RuntimeGet()->output_dir); /* Prepend output dir to file name */
```

The quantities (and related data-types) read from the file are now described.

4.1 The *[Grid]* block

```
[Grid]
X1-grid    0.0    100    1.0
X2-grid    0.0    100    1.0
X3-grid    0.0     1     1.0
```

In the static version of **gPLUTO**, it defines the physical domain and generates the whole computation mesh:

- X1-grid (*double*) (*int*) (*double*)
- X2-grid (*double*) (*int*) (*double*)
- X3-grid (*double*) (*int*) (*double*)

For each dimension, the three fields (*double*) (*int*) (*double*) specify, respectively, the leftmost domain interface x_b , the number of points and the rightmost domain interface x_e . The global domain therefore extends, in each direction, from the first (*double*) node coordinate to the last (*double*) node coordinate. If a dimension is ignored, then 1 grid-point only should be assigned to that grid. These values can be accessed from the RunConfig structure.

By default, a uniform grid with constant spacing is generated. Computational cells may not necessarily have the same physical length in each direction (e.g., squares in 2D, cubes in 3D) and can have an aspect ratio different from 1.

Non-uniform grid spacing can be generated by enabling the MAPPED_GRID option, see §2.1.4.

4.2 The *[Time]* Block

```
[Time]
CFL          0.4
CFL_max_var  1.1
CFL_par       0.3      # optional
tstop        1.0
first_dt      1.e-4
```

This section specifies some adjustable time-marching parameters:

- CFL (*double*)
Courant number: it controls the time step length and, in general, it must be less than 1. The actual limit can be inferred from Eq. (2.10). In the case of unsplit Runge-Kutta time stepping, for instance, $CFL \lesssim 1/N_{\text{dim}}$ where N_{dim} is the number of spatial dimensions. Certain combinations of algorithms may have more stringent limitations: a 2nd-order Runge-Kutta algorithm with parabolic reconstruction, for example, requires $CFL \lesssim 0.4$ for stability reasons.
- CFL_max_var (*double*)
Maximum value allowed for $\Delta t^n / \Delta t^{n-1}$ (maximum time step growth between two consecutive steps).
- CFL_par (*double*) [optional]
When parabolic terms are integrated using operator splitting (with Super-Time-Stepping, §??), it controls the diffusion Courant number. The default value is $0.8/N_{\text{dim}}$. This parameter has no effect when parabolic terms are included via standard explicit integration.
- tstop (*double*)
Integration ends when $t = t_{\text{stop}}$, unless a maximum number of steps (§1.4) is given. tstop has to be > 0.0 .
- first_dt (*double*)
The initial time step. A typical value is 10^{-6} .

4.3 The *[Radiation Parameters]* Block

```
[Radiation parameters]
```

```
rad_first_dt      1.e-7
rad_CFL           0.4
rad_CFL_max_var   1.1
```

This optional block is used only when the RADIATION module is enabled. In the nonrelativistic module, it controls the radiation substeps:

- `rad_first_dt` (*double*)
Initial radiation substep. The default is `first_dt/100`.
- `rad_CFL` (*double*)
Courant number used by the explicit radiation transport substep. The default is `CFL`.
- `rad_CFL_max_var` (*double*)
Maximum growth factor between two consecutive radiation substeps. The default is `CFL_max_var`.

4.4 The *[Solver]* Block

```
[Solver]
```

```
Solver           tvdlf
RadSolver        hll
```

	HD	RHD	MHD	RMHD	ResRMHD	RADIATION
<i>two_shock</i>	✓	✓	-	-	-	-
<i>roe</i>	✓	-	✓	-	-	-
<i>hlld</i>	-	-	✓	✓	-	-
<i>mhllc</i>	-	-	-	-	✓	-
<i>hllc</i>	✓	✓	✓	✓	-	✓
<i>gforce</i>	-	-	✓	✓	-	-
<i>mhll</i>	-	-	-	-	✓	-
<i>hll</i>	✓	✓	✓	✓	-	✓
<i>tvdlf</i>	✓	✓	✓	✓	✓	✓

Table 4.1: Available Riemann solvers for the different physics modules.

- `Solver` (*string*)
Sets the Riemann solver for flux computation. From the most accurate (i.e. least diffusive) to the least accurate (i.e. most diffusive):
 - *two_shock*: The Riemann problem is solved exactly or approximately at every interface; this is usually more accurate, but computationally intensive. See [CW84] for the HD module.
 - *roe*: Linearized Roe Riemann solver based on characteristic decomposition of the Roe matrix, [Roe81].
 - *hlld*: The hlld approximate Riemann solver of [MK05] (for the adiabatic case), [Mig07] (for the isothermal case) and [MUB09] for the relativistic MHD equations;
 - *hllc*: Harten, Lax, Van Leer approximate Riemann Solver that restores with the middle contact discontinuity. Similarly, the *mhll(c)* is a five-wave solver is based on the direct combination of two Riemann solvers: one for the outermost EM waves across which only

transverse components of electric and magnetic fields can change, and a second one across the sound waves where only hydrodynamical variables have non-trivial jumps (see §4 in [MMBD19]).

- *gforce*: The Generalize FORCE scheme by [TT06], where the intercell numerical flux is obtained from a weighted average of the Lax-Friedrichs (LF) and Lax-Wendroff (LW) fluxes;

$$\hat{F} = \omega_g F^{LW} + (1 - \omega_g) F^{LF}. \quad (4.1)$$

If not specified $\omega_g = \text{GFORCE_OMEGA} = 1/(1 + \text{CFL})$ by default. Otherwise, you may define a different value of `GFORCE_OMEGA` in your `definitions.hpp`.

- *h11*: Harten, Lax, Van Leer approximate Riemann Solver;
- *tvd1f*: A simple Lax-Friedrichs scheme is used.

Note that not all solvers are available for a given physics module, see Table 4.1. We warn the user that, under some circumstances (high Mach number flows, low density plasmas), more diffusive solvers such as HLL or TVDLF turn out to be more robust than accurate solvers. This may be used to prevent the occurrence of negative pressures / densities, see §2.4. However, hybrid/adaptive strategies can be turned on when `SHOCK_FLATTENING` is set to *MULTID*, §B.3.

- *RadSolver* (*string*)
Sets the Riemann solver for flux computation of radiation fields if the `RADIATION` module is used (see §9.2). From the most accurate (i.e. least diffusive) to the least accurate (i.e. most diffusive):
 - *h11c*: Harten, Lax, Van Leer approximate Riemann Solver by [MM19] including a middle contact discontinuity;
 - *h11*: Harten, Lax, Van Leer approximate Riemann Solver by [GAH07];
 - *tvd1f*: A simple Lax-Friedrichs scheme is used.

4.5 The *[Boundary]* Block

```
[Boundary]
X1-beg      outflow
X1-end      outflow
X2-beg      outflow
X2-end      outflow
X3-beg      outflow
X3-end      outflow
```

Specifies the boundary conditions to be applied in the ghost zones of the computational domain:

- *X1-beg* (*string*)
- *X1-end* (*string*)
- *X2-beg* (*string*)
- *X2-end* (*string*)
- *X3-beg* (*string*)
- *X3-end* (*string*)

Assuming that q is a scalar quantity and n is the coordinate direction orthogonal to the boundary plane, *string* can be one of the following:

- *axisymmetric*
Same as *reflective*, except for the angular component of v_ϕ or B_ϕ which also changes sign:

$$q \rightarrow q, \quad \left\{ \begin{array}{l} v_n \rightarrow -v_n \\ B_n \rightarrow -B_n \end{array} \right\}, \quad \left\{ \begin{array}{l} v_\phi \rightarrow -v_\phi \\ B_\phi \rightarrow -B_\phi \end{array} \right\}, \quad \left\{ \begin{array}{l} v_{axis} \rightarrow v_{axis} \\ B_{axis} \rightarrow B_{axis} \end{array} \right\}$$

where *axis* is $(r = 0, z)$ or $(r, \theta = 0)$ in cylindrical or spherical coordinates.

- *eqtsymmetric*

Sets equatorial symmetry with respect to a given plane. It is similar to *reflective*, but with reversed sign for the magnetic field:

$$q \rightarrow q, \quad \left\{ \begin{array}{l} v_n \rightarrow -v_n \\ B_n \rightarrow B_n \end{array} \right., \quad \left\{ \begin{array}{l} v_t \rightarrow v_t \\ B_t \rightarrow -B_t \end{array} \right.$$

- *outflow*

Sets zero gradient across the boundary: $\frac{\partial q}{\partial n} = 0, \quad \frac{\partial v}{\partial n} = 0, \quad \frac{\partial B}{\partial n} = 0$

- *reflective*

Reflective (rigid walls) boundary conditions. Variables are symmetrized across the boundary and normal components of vector fields flip signs,

$$q \rightarrow q, \quad \left\{ \begin{array}{l} v_n \rightarrow -v_n \\ B_n \rightarrow -B_n \end{array} \right., \quad \left\{ \begin{array}{l} v_t \rightarrow v_t \\ B_t \rightarrow B_t \end{array} \right.$$

where $n(t)$ is normal (tangential) to the interface.

- *periodic*

Sets periodic boundary conditions on both sides of the computational domain.

- *singular_axis*

Used to set boundary conditions at $R = 0$ in cylindrical coordinates or at $\theta \rightarrow 0, \pi$ in spherical coordinates. Fluid variables in the ghost cells across the pole are simply taken from the corresponding active cells after π rotation around the pole: $q(i, j) = q(i', j + n_\phi/2)$ where i' is the symmetric radial index with respect to the inner boundary. Signs are reverse only for inplane vector components:

$$\left\{ \begin{array}{l} (v_r, v_\phi) \rightarrow -(v_r, v_\phi) \\ (B_r, B_\phi) \rightarrow -(B_r, B_\phi) \end{array} \right., \quad \left\{ \begin{array}{l} v_z \rightarrow v_z \\ B_z \rightarrow B_z \end{array} \right.$$

This b.c. can also be used in conjunction with the ring average technique [ZSL⁺19] to remove the severe time step restriction near a singular axis, see “RING_AVERAGE” in table B.1. Note: parallelization is not possible in the azimuthal direction with this b.c.

- *rotated*

This boundary conditions is useful to rotate a 1D problem in a 2D or 3D computational domain. It must be used in conjunction with the `RotateSet()` function. See, e.g., the rotated shock tube configuration in `Test_Problems/MHD/Rotated_Sod`.

- *userdef*

User-supplied boundary conditions (it requires coding your own boundary conditions in the function `UserDefBoundary()` in `init.cpp`, see §5.2).

Like the `[Grid]` block, you should include the x_3 boundaries for 2D runs, even if they will not be considered.

4.6 The `[Static Grid Output]` Block

```
[Static Grid Output]

uservar      0
output_dir   ./                      # optional
log_dir      ./Log_Files             # optional
dbl          1.0 -1 single_file
flt          -1.0 -1 single_file
vtk          -1.0 -1 single_file     # optional
dbl.h5       1.0 2.40h               # optional
flt.h5       1.0 -1                 # optional
tab          -1.0 -1                 # optional
log          1
analysis     -1.0 -1                 # optional
```

This block controls the output options in the static grid version of the code. Output files are written at specific times in a specific directory (local working directory by default) using different file formats, described in Chapter 11. The available fields are:

- `uservar (int) (string1 string2 ...)`
Defines supplementary variables to be written to disk in any of the format described below. The first integer represents the number of supplementary variables. When greater than zero, it must be followed by as many variable names separated by spaces, see Chapter 11.
- `output_dir (string)`
Sets the directory name for writing and reading simulation data. The directory must already exist at runtime or an error will occur. When writing, any of the data formats available below and the corresponding `.out` log files will be written in the directory specified by `output_dir`. In parallel, processor log files will also be written to the same directory unless `log_dir` (see below) is explicitly set to a different value. When reading (during restarts), `.dbl` or `.dbl.h5` files and the corresponding `*.out` must be present in this directory or an error will occur (see §11.1.3).
If `output_dir` is not specified, the directory from which `pluto` is executed is assumed.
- `log_dir (string)`
Sets the directory where log files are being written. This directory must already exist or the local directory will be used instead.
- `dbl (double) (int/string) (string)`
Assigns the output intervals for double precision (8 bytes) binary data. A negative value suppresses output with this format.
 - The first field (*double*) specifies the time interval (in code units) between consecutive outputs.
 - The second field can be an *int* giving the number of steps between consecutive outputs or a *string* giving the wall-clock time between consecutive outputs. A value, for instance, of `2.40h` tells **gPLUTO** to write one `.dbl` file every two hours and 40 minutes. Negative values will be ignored for this control parameter.
 - The last field (*string*) can be either *single_file* (one single output file per time step containing all of the variables) or *multiple_files* (different variables are written to different files).

Double-precision format files can be used to restart the code using the `-restart n` command line argument (see §11.1.3).

- `flt (double) (int/string) (string) [cgs]`
Like `dbl`, but for single-precision (4 bytes) data files. The last value (*cgs*) is optional and can be given to save datafiles directly in *cgs* physical units rather than in code units.
- `vtk (double) (int/string) (string) [cgs]`
Like `flt`, but for VTK legacy file format, see §11.1.4.
- `dbl.h5 (double) (int/string)`
Like `dbl`, but for `hdf5` double-precision format §11.1.2. This format can also be used for restarting the code by supplying the `-h5restart n` command line argument (see §11.1.3).
- `flt.h5 (double) (int/string)`
Like `dbl` but for `hdf5` single-precision format §11.1.2;
- `tab (double) (int/string)`
Sets the time and the number of steps interval for tabulated `ascii` format, §11.1.5;
- `log_dir (string)`
The name of the directory where output log files will be written. The directory must already be present at runtime. If not specified, the value of `output_dir` is used by default.

- `log (int)`
Sets how often (in number of steps) the log file should be dumped to screen (in serial mode) or to disk (in parallel).
- `analysis (double) (int)`
Sets time and number of steps interval between consecutive calls to the function `Analysis()`, see 5.4.

4.7 The *[Particles]* Block

```
[Particles]
Nparticles          1/1
particles_tstart    5.0
particles_dbl       -1.0  5.59h
particles_flt       -0.4  -1
particles_vtk       15.0  -1
particles_tab       -1.0  -1
```

The particles block is used to provide relevant information for the particle module (see Chapter 10) used at runtime. Additional information describing the datafile formats are given in §10.3. The available fields, mostly controlling output, are:

- `Nparticles (int)/(int)`
Specifies the particle number density (particles per cell) at initialization in the form n/m where:
 - n gives the number of particles per block;
 - m gives the number of cells per block.

Example: 1/1 means one particle per cells, 4/1 means 4 particle per cell while 1/8 means one particle every 8 cells. More details can be found in §10.1.1.
- `particles_tstart (double)`
Time at which particles integration starts (default is 0.0).
- `particles_dbl (double) (int/string)`
Controls output intervals for double precision particle datafiles. The two fields have the same meaning used in the *[Static Grid Output]* block:
 - The first field (double) specifies the time interval (in code units) between consecutive outputs.
 - The second field can be an integer giving the number of steps between consecutive outputs or a string giving the wall-clock time between consecutive outputs. A value, for instance, of 5.50h tells PLUTO to write one .dbl file every five hours and 50 minutes.

Negative values will suppress the option.

- `particles_flt (double) (int/string)`
Same as `dbl`, but for single precision data files.
- `particles_vtk (double) (int/string)`
Same as `dbl`, but for VTK legacy format.
- `particles_tab (double) (int/string)`
Same as `dbl`, but for tabulated (ASCII) format (serial only).

4.8 The *[Parameters]* Block

```
[Parameters]
SCRH 0
```


- PAR_NAME_1 (double)
- ...
- PAR_NAME_n (double)

User-defined parameter values are read at runtime in this section. The labels on the left identify the parameter *labels* (i.e. the corresponding indices of the array `g_inputParam`) while the *(double)* values on the right are the actual user-defined parameter values. The number of parameters specified in this section must exactly match the number and the order given in `definitions.hpp`

5. Initial and Boundary Conditions

The source file `init.cpp` provides a set of functions that are used to set the fluid configuration for your own specific problem. These include:

- `Init()`: sets (fluid) initial conditions locally, as functions of the spatial coordinates x_1, x_2, x_3 ;
- `UserDefBoundary()`: sets user-defined boundary conditions at the physical boundary sides of your computational domain if necessary;
- `Analysis()`: run-time data analysis and reduction;
- `BodyForceVector()`, `BodyForcePotential()`: define, respectively, the vector components of the acceleration vector and/or the gravitational potential.

The `init.cpp` must be part of your local working directory and a template can be found in `Src/Templates/init.cpp`. Functions are described in the next sections.

5.1 Initial Conditions: the `Init()` function

The `Init()` function is used to assign the initial condition as a function of the spatial coordinates.

Syntax:

```
void Init (double *v, double x1, double x2, double x2, RunConfig &run_config)
```

Arguments:

- `v`: a pointer to a vector of primitive quantities. A particular variable is located by means of an index: $\rho = v[\text{RH0}]$, $v_x = v[\text{VX1}]$, $v_y = v[\text{VX2}]$... and so forth. Although `VX1`, `VX2` and `VX3` should be used in any coordinate system, in order to avoid confusion, an alternative set may be adopted if the geometry is not Cartesian, see columns 2-4 in Table 5.1.

Temperature (in Kelvin) can also be used to initialize density or pressure, see §??.

- `x1, x2, x3`: coordinates x_1, x_2, x_3 of the computational cell where `v` is initialized;
- `run_config`: a reference to the `RunConfig` structure.

Example #1:

The following code sets a disk with radius 1 centered around the origin in a 2D Cartesian domain. The flow is stationary and the disk has higher density and pressure ($\rho = 10, p = 30$) with respect to the background state ($\rho = 1, p = 1$):

```

void Init (double *v, double x1, double x2, double x3, RunConfig &run_config)
{
    double r;

    r = sqrt(x1*x1 + x2*x2);
    v[VX1] = v[VX2] = 0.0;
    if (r < 1.0){
        v[RHO] = 10.0;
        v[PRS] = 30.0;
    }else{
        v[RHO] = 1.0;
        v[PRS] = 1.0;
    }
}

```

With a small modification, the same initial condition can be written in a dimension-independent way (e.g. a line in 1D, a disk in 2D and a sphere in 3D):

```

void Init (double *v, double x1, double x2, double x3, RunConfig &run_config)
{
    double r2,r;

    r2 = DIM_EXPAND(x1*x1, + x2*x2, + x3*x3);
    r = sqrt(r2);
    v[VX1] = 0.0;
    v[VX2] = 0.0;
    v[VX3] = 0.0;

    if (r < 1.0){
        v[RHO] = 10.0;
        v[PRS] = 30.0;
    }else{
        v[RHO] = 1.0;
        v[PRS] = 1.0;
    }
}

```

The macro `DIM_EXPAND(a,b,c)` is used to write dimension-independent code by conditionally compiling one, two or three comma-separated arguments on the value taken by `DIMENSIONS`. Function-like macro are documented in the file `macro.hpp` of the API reference guide, see `./Doc/-Doxygen/html/macros_8hpp.html`.

Index	Quantity	Physics Module
RHO	density	ALL
VX1	x_1 -velocity	ALL
VX2	x_2 -velocity	ALL
VX3	x_3 -velocity	ALL
PRS	(thermal) pressure	ALL
BX1	x_1 cell-centered magnetic field	MHD, RMHD, ResRMHD
BX2	x_2 cell-centered magnetic field	MHD, RMHD, ResRMHD
BX3	x_3 cell-centered magnetic field	MHD, RMHD, ResRMHD
BX1s	x_1 staggered magnetic field	MHD, RMHD, ResRMHD
BX2s	x_2 staggered magnetic field	MHD, RMHD, ResRMHD
BX3s	x_3 staggered magnetic field	MHD, RMHD, ResRMHD
TRC	tracer (passive scalar, Q)	ALL

Table 5.1: Array indices used for labeling primitive variables. Staggered components (“s” suffix) are used only for magnetic fields in the boundary conditions, see §6.2.2.3.

5.1.1 Units and Dimensions

In general, **gPLUTO** works with non-dimensional (or “code”) units so that flow quantities can be properly scaled to “reasonable” numbers. Although it is possible, in principle, to work directly in c.g.s. units

(i.e. *cm*, *sec* and *gr*), we strongly recommend to scale all quantities to non-dimensional units, in order to avoid occurrences of extremely small ($\lesssim 10^{-9}$) or large ($\gtrsim 10^{12}$) numbers that may be misinterpreted by numerical algorithms.

For simple adiabatic simulations involving no source term, the dimensionalization process can be avoided since the HD or MHD equations are scale invariant. However, dimensionalization is strictly necessary when specific length, time or energy scales are introduced in the problem and they must compare to the dynamical advection scales. For such problems, **gPLUTO** requires three fundamental units to be specified in you `Init()` function by defining appropriate values for the following members of the `run_config` structure:

```
run_config.unit_density  (ρ₀):  sets the reference density in gr/cm³;
run_config.unit_length   (L₀):  sets the reference length in cm;
run_config.unit_velocity (v₀):  sets the reference velocity in cm/s.
```

All other units are derived from a combination of the previous three: time is measured in units of $t_0 = L_0/v_0$, pressure in units of $p_0 = \rho_0 v_0^2$, while magnetic field (for the MHD module only, see §6.2) in units of $B_0 = v_0 \sqrt{4\pi\rho_0}$, i.e.:

$$\rho = \frac{\rho_{\text{cgs}}}{\rho_0} \quad , \quad v = \frac{v_{\text{cgs}}}{v_0} \quad , \quad p = \frac{p_{\text{cgs}}}{\rho_0 v_0^2} \quad , \quad B = \frac{B_{\text{cgs}}}{\sqrt{4\pi\rho_0 v_0^2}} \quad . \quad (5.1)$$

where ρ , v , p and B are now dimensionless.

Note: The dimensionless form of the equations can be derived by factoring out ρ_0 , v_0 , L_0 (and t_0) from the original equations written in c.g.s. units. For instance, starting from the HD equation in presence of thermal conduction and cooling one has to write:

$$\begin{aligned} \frac{\rho_0}{t_0} \frac{\partial \rho}{\partial t} + \frac{\rho_0 v_0}{L_0} \nabla \cdot (\rho \mathbf{v}) &= 0 \\ \frac{\rho_0 v_0}{t_0} \frac{\partial \mathbf{m}}{\partial t} + \frac{\rho_0 v_0^2}{L_0} \nabla \cdot (\mathbf{m} \mathbf{v}) + \frac{\rho_0 v_0^2}{L_0} \nabla p &= \frac{\rho_0}{L_0} (-\rho \nabla \Phi_{\text{cgs}}) + \rho_0 \rho \mathbf{g}_{\text{cgs}} \\ \frac{\rho_0 v_0^2}{t_0} \frac{\partial (E + \rho \Phi)}{\partial t} + \frac{\rho_0 v_0^3}{L_0} \nabla \cdot \left[\left(\frac{\rho v^2}{2} + \rho e + p + \rho \Phi \right) \mathbf{v} \right] - \frac{1}{L_0^2} \nabla \cdot (\kappa_{\text{cgs}} \nabla T_{\text{cgs}}) &= \rho_0 v_0 \mathbf{m} \cdot \mathbf{g}_{\text{cgs}} - \Lambda_{\text{cgs}} \end{aligned} \quad (5.2)$$

Since $t_0 = L_0/v_0$ and $T_{\text{cgs}} = \mathcal{K} \mu p / \rho$ is the temperature express in Kelvin (see §??), straightforward manipulation yields the dimensionless form of the equations:

$$\begin{aligned} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) &= 0 \\ \frac{\partial \mathbf{m}}{\partial t} + \nabla \cdot (\mathbf{m} \mathbf{v}) + \nabla p &= -\rho \nabla \Phi + \rho \mathbf{g} \\ \frac{\partial (E + \rho \Phi)}{\partial t} + \nabla \cdot \left[\left(\frac{\rho v^2}{2} + \rho e + p + \rho \Phi \right) \mathbf{v} \right] - \nabla \cdot \left[\kappa \nabla \left(\frac{\mu p}{\rho} \right) \right] &= \mathbf{m} \cdot \mathbf{g} - \frac{L_0}{\rho_0 v_0^3} \Lambda_{\text{cgs}} \end{aligned} \quad (5.3)$$

where $\Phi = \Phi_{\text{cgs}}/v_0^2$, $\mathbf{g} = \mathbf{g}_{\text{cgs}} L_0/v_0^2$, and

$$\kappa = \kappa_{\text{cgs}} \frac{m_u}{\rho_0 v_0 L_0 k_B} \quad (5.4)$$

is the dimensionless form of the conduction coefficient.

If not specified, the default values are, respectively, $\rho_0 = 1 \text{ } m_p \text{ gr/cm}^3$, $L_0 = 1 \text{ AU}$ and $v_0 = 1 \text{ Km/s}$. The values of the three fundamental units can be changed in your `Init()` function,

```
run_config.unit_density = 1.67e-23;
run_config.unit_length  = 3.1e18;
run_config.unit_velocity = 1.e5;
```

Note that, when the relativistic modules are used, v_0 must be the speed of light.

Output files can be directly saved in cgs units using the .flt or .vtk data format, see §4.6.

Example #2:

Consider a flow with typical number densities of the order of $n \approx 10 \text{ cm}^{-3}$, temperatures $T \approx 10^4 \text{ K}$ (corresponding to typical sound speeds of $c_{s0} \approx 10 \text{ Km/s}$). Suppose, also, that the flow propagates with uniform speed $v \approx 50 \text{ Km/s}$ and the typical scale size of the problem is $L \approx 1 \text{ pc} \approx 3.1 \cdot 10^{18} \text{ cm}$. Then one may choose

$$\rho_0 = n_0 m_p \approx 1.67 \cdot 10^{-23} \text{ gr/cm}^3, \quad v_0 = 1 \text{ Km/s} = 10^5 \text{ cm/s}, \quad L_0 = 3.1 \cdot 10^{18} \text{ cm}$$

From the python script, this is done by including the following lines in your `Init()` function:

```
/* [Beg] user-defined constants (do not change this line) */

run_config.unit_density = 10.0*CONST_mp;
run_config.unit_length  = CONST_pc;
run_config.unit_velocity = 1.e5;

/* [End] user-defined constants (do not change this line) */
```

where `CONST_mp` and `CONST_pc` are pre-defined symbolic constants (proton mass and parsec in c.g.s units) and are defined, together with several other constants, in Appendix B.1. Please remember using parenthesis around a macro expression to avoid incorrect expansion.

With this choice of units, the piece of code describing the initial condition becomes

```
v[RHO] = 1.0;      /* means 1 * [1.67 10^{-23}] gr/cm^3 or 10/cm^3 */
v[VX1] = 50.0;     /* means 50 * [1 Km/sec] */
cs      = 10.0;     /* means 10 * [1 Km/sec] */

/* -- define pressure to that sound speed = 1 * 1.e6 = 10 Km/sec -- */

v[PRS] = v[RHO]*cs*cs/GAMMA_EOS; /* means 100/gmm * [\rho_0*v_0^2] */
```

where `CONST_PI` ($= \pi$) is another pre-defined constant. With this initialization, the sound speed is exactly $c_s = 10 \text{ Km/s}$.

5.1.2 Assigning Initial Conditions from Input Files

It is possible to assign initial conditions from user-supplied binary data by providing i) one or more grid data files and the corresponding ii) raw binary files containing the variable(s) to be read. The size, dimensions and even the geometry of the input grid may be different from the actual grid employed by PLUTO, as long as the coordinate transformation is implemented. Even different input data files with different size and geomery may be used. This provides a flexible and efficient tool to assign initial conditions by mapping data values originally defined on different computational domains. For instance, you can map a 2D spherical grid onto a 2D axisymmetric cylindrical domain, generate a 3D Cartesian domain by rotating any 2D axisymmetric data and so forth.

The module is initialized by calling the function `InputDataOpen()` which reads and stores input grid information such as size, number of variables, geometry and dimensions. This function should be called once per variable from your `InitDomain()` function:

```
id = InputDataOpen (data_fname, grid_fname, endianness, offset, vpos);
```

where the arguments are:

1. `data_fname`: a string giving the name of the input binary file, with the extension .dbl or .flt for double or single precision, respectively.
2. `grid_fname`: the name of the input grid file;
3. `endianness`: a string specifying the endianness of the input data file. Either "big", "little" or an empty string " " to leave things unchanged.

4. `offset`: a long integer giving the offset of the variable (in units of double or float) from the beginning of the file. For a typical double-precision datafile, density and the x_1 -velocity component (RHO and VX1) are stored, respectively, as the 1st and 2nd records. Since each block has $N_b = nx1 \times nx2 \times nx3$ “words”, we will use `offset = 0` for the former and `offset = Nb` for the latter. Here `nx1`, `nx2` and `nx3` are the sizes of the input data file.
5. `vpos`: the variable position with respect to the cell center (`CENTER` / `X1FACE` / `X2FACE` / `X3FACE`).

The return argument (`id`) is a file handle that can be used later.

The input binary data file(s) should be written in binary format using either single or double precision with extensions “.flt” (for the former) or “.dbl” (for the latter). Both single and/or multiple data files are supported. If a single file containing more variables is supplied, variables should be stored sequentially; otherwise you can provide one file per variable. You may provide only some of the variables used by **gPLUTO** and not necessarily all of them. The input grid file should be written using the same format employed by **gPLUTO**, see §11.1.6. Each field must contain as many points as specified by the input grid file.

After initialization, the grid size can be retrieved with the function `InputDataGridSize()` by supplying the file `id` and an array of integer containing, on output, the grid size (see the example below). Interpolation can then be done using, e.g.,

```
d->Vc(i,j,k,RHO) = InputDataInterpolate (id, x1[i], x2[j], x3[k]);
```

which will read input data (in chunks) and interpolate them using bi- or tri-linear interpolation at the desired coordinate location (`x1[i]`, `x2[j]`, `x3[k]`). Input data are read from disk in chunks of size `nx1`, `nx2`, `ID_NZ_MAX` where `ID_NZ_MAX` is an integer number (default is 4) which can be changed in your definitions.hpp (see B.3).

Note: Note that when interpolating between different geometries (e.g. from Cartesian to spherical), data values may not be contiguous and the input data files are read several times slowing down the process. In this case it may be advisable to set the macro `ID_NZ_MAX` equal to the number of zones in the third direction of the input grid.

The function `InputDataClose()` is finally used to free the memory structure when not needed.

In the example below, density, velocity components and pressure are assigned from the input binary file `tmp/data.0010.flt` defined on the computational domain specified in `tmp/grid.out`:

```
void InitDomain (Data *d, Grid *grid)
{
    int i, j, k, id, size[3];
    int *ntot = grid->ntot;
    long int block_size;
    char grid_file[MAX_STRING_SIZE];
    char data_file[MAX_STRING_SIZE];
    double *x1 = grid->xC[IDIR];
    double *x2 = grid->xC[JDIR];
    double *x3 = grid->xC[KDIR];

    RBox box;
    RBoxDefine(0, ntot[IDIR] - 1,
              0, ntot[JDIR] - 1,
              0, ntot[KDIR] - 1, &box);

    /* -----
    0. Set input file names (grid, data), read grid size
    ----- */

    snprintf (grid_file, sizeof(grid_file), "input_grid.out");
    snprintf (data_file, sizeof(data_file), "input_data.dbl");

    /* -----
    1. Read and Interpolate data
    ----- */

    id = InputDataOpen(data_file, grid_file, "\n", 0*block_size, CENTER);
    InputDataGridSize (id, size);
    block_size = (long)size[0]*(long)size[1]*(long)size[2]; /* Offset of a single data block */
    BOX_LOOP(&box, i,j,k) d->Vc(i,j,k,RHO) = InputDataInterpolate (id,x1[i],x2[j],x3[k]);
    InputDataClose(id);

    id = InputDataOpen(data_file, grid_file, "\n", 1*block_size, CENTER);
    BOX_LOOP(&box, i,j,k) d->Vc(i,j,k,VX1) = InputDataInterpolate (id,x1[i],x2[j],x3[k]);
    InputDataClose(id);
}
```

```

id = InputDataOpen(data_file, grid_file, "_", 7*block_size, CENTER);
BOX_LOOP(&box, i, j, k) d->Vc(i, j, k, PRS) = InputDataInterpolate (id, x1[i], x2[j], x3[k]);
InputDataClose(id);
}

```

Note: When the input geometry differs from the one used by **gPLUTO**, vector components are *not* automatically transformed to the current geometry. Staggered magnetic fields may not be assigned in this way since the divergence free condition is not necessarily maintained. Using the vector potential components is more advisable.

5.2 User-defined Boundary Conditions

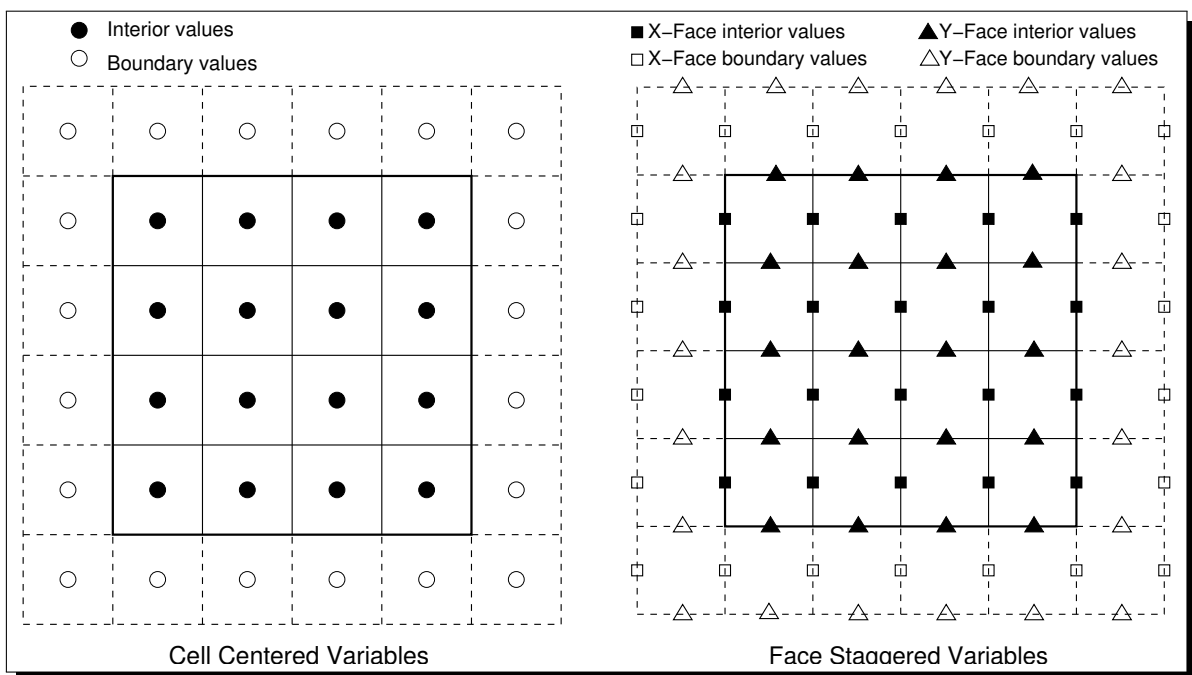


Figure 5.1: Schematic representation of cell-centered (left panel) and face-centered (right panel) collocation of physical variables on a 2D grid. X and Y-face centered staggered quantities are shown by squares and triangles, respectively. Filled symbols (circles, boxes and triangles) are considered interior values part of the solution, whereas boundary values are identified by empty symbols and must be prescribed by the user if the boundary is *userdef*.

The `UserDefBoundary()` function is used to assign user-defined boundary conditions to a particular physical boundary (see Fig 5.1) if this has been tagged *userdef* inside your `pluto.ini`.

Alternatively, this function may also be used to control the solution array at the beginning of every time step inside the computational domain (set floor values, override the solution, etc...) by first enabling `INTERNAL_BOUNDARY` to `YES` inside `definitions.hpp`, see §5.2.1.

Syntax:

```
void UserDefBoundary (const Data *d, RBox *box, int side, Grid *grid)
```

Arguments:

- `*d`: a pointer to the **gPLUTO** data structure

All variables must be assigned at a user-defined boundary with the exception of the staggered component of magnetic field normal to the interface if you are using the Constrained Transport (CT) method, see §6.2.2.3.

- `*box`: a pointer to a `RBox` structure, defining the rectangular portion of the domain over which ghost zone values should be assigned. The loop on the domain can be performed using the macro `BOX_LOOP(box,i, j, k)` which automatically takes care of the index range of definition.
- `side`: an input integer label specifying on which side of the physical domain user-defined values should be prescribed. It can take on the following values:
 - `X1_BEG, X1_END`: boundary conditions can be assigned in the ghost zones at the beginning and end of the physical domain in the x_1 direction
 - `X2_BEG, X2_END`: boundary conditions can be assigned in the ghost zones at the beginning and end of the physical domain in the x_2 direction
 - `X3_BEG, X3_END`: boundary conditions can be assigned in the ghost zones at the beginning and end of the physical domain in the x_3 direction
 - 0 (zero): change/control the solution inside the computational domain. This feature can be used *only* if the macro `INTERNAL_BOUNDARY` has been enabled in your `definitions.hpp`, see §5.2.1.

If, say, `X1-beg` has been tagged `userdef` inside your `pluto.ini`, the user has to specify the boundary values at the beginning of the x_1 direction when `side==X1_BEG`.

- `*g`: a pointer to the `Grid` structure containing all of the relevant grid information. See the code documentation for more details on the members of the `Grid` structure.

Example #1:

As a first example we show how to prescribe a fixed inflow boundary condition for a jet model. The inflow values are specified as

$$\begin{pmatrix} \rho \\ v_x \\ v_y \\ v_z \\ p \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ M \\ 1/\Gamma \end{pmatrix} \quad \text{for } R \leq 1, \quad \begin{pmatrix} \rho(R, -z) \\ v_x(R, -z) \\ v_y(R, -z) \\ v_z(R, -z) \\ p(R, -z) \end{pmatrix} = \begin{pmatrix} \rho(R, z) \\ v_x(R, z) \\ v_y(R, z) \\ -v_z(R, z) \\ p(R, z) \end{pmatrix} \quad \text{for } R > 1$$

where M is the Mach number and $R = \sqrt{x^2 + y^2}$.

```
void UserDefBoundary (const Data *d, RBox *box, int side, Grid *grid)
{
  int i, j, k;
  RunConfig &run_config = *(d->run_config);

  if (side == X2_BEG){ /* -- select the boundary side -- */
    #pragma acc parallel loop collapse(3) present(d, box, run_config)
    BOX_LOOP(box,i, j, k){ /* -- Loop over boundary zones -- */
      double *x1 = grid->xC[IDIR]; /* -- array pointer to x1 coordinate -- */
      double *x2 = grid->xC[JDIR]; /* -- array pointer to x2 coordinate -- */
      if (sqrt(x1[i]*x1[i] + x2[j]*x2[j]) <= 1.0){ /* -- set jet values for r <= 1 -- */
        d->Vc(i,j,k,RH0) = 1.0;
        d->Vc(i,j,k,VX1) = 0.0;
        d->Vc(i,j,k,VX2) = 0.0;
        d->Vc(i,j,k,VX3) = run_config.inputParam[MACH];
        d->Vc(i,j,k,PRS) = 1.0/run_config.gamma;
      }else{ /* -- reflective boundary for r > 1 -- */
        int jbeg = grid->lbeg[JDIR];
        d->Vc(i,j,k,RH0) = d->Vc(i,2*jbeg - j - 1,k,RH0);
        d->Vc(i,j,k,VX1) = d->Vc(i,2*jbeg - j - 1,k,VX1);
        d->Vc(i,j,k,VX2) = d->Vc(i,2*jbeg - j - 1,k,VX2);
        d->Vc(i,j,k,VX3) = -d->Vc(i,2*jbeg - j - 1,k,VX3);
        d->Vc(i,j,k,PRS) = d->Vc(i,2*jbeg - j - 1,k,PRS);
      }
    }
  }
}
```


Example #1:

```

void UserDefBoundary (const Data *d, RBox *box, int side, Grid *grid)
{
    int i, j, k;
    double ri,rip;
    int ibeg = grid->lbeg[IDIR], iend = grid->lend[IDIR];
    int jbeg = grid->lbeg[JDIR], jend = grid->lend[JDIR];
    int kbeg = grid->lbeg[KDIR], kend = grid->lend[KDIR];

    if (side == X1_BEG){ /* -- X1_BEG boundary -- */
        #pragma acc parallel loop collapse(3) present(box, d, grid)
        BOX_LOOP(box,i,j,k){
            double *x = grid->xC[IDIR], *xL = grid->xL[IDIR];
            double *y = grid->xC[JDIR], *yL = grid->xL[JDIR];
            double *z = grid->xC[KDIR], *zL = grid->xL[KDIR];
            double vj[NVAR];
            JetValues(vj, x[ibeg], y[j], z[k], grid);
            #pragma acc loop
            for (int nv = 0; nv < NVAR; nv++){
                d->Vc(i,j,k,nv) = vj[nv];
            }
        }
    }
}

```

The previous piece of code is executed *only* if you have selected *userdef* at the X2-beg boundary inside your pluto.ini.

Example #2:

As a second example, we discuss the user-defined boundary condition employed in the shock-cloud problem (Test_Problems/MHD/Shock_Cloud/). Here we want to prescribe, at the X1-end boundary, constant pre-shock values on both cell-centered quantities and staggered magnetic fields.

```

void UserDefBoundary (const Data *d, RBox *box, int side, Grid *grid)
{
    int i, j, k;
    RunConfig &run_config = *(d->run_config);

    if (side == X1_END){ /* -- select the boundary side -- */

        #pragma acc parallel loop collapse(3) present(d, box, run_config)
        BOX_LOOP(box,i,j,k) {
            d->Vc(i,j,k,RH0) = 1.0;
            d->Vc(i,j,k,VX1) = -11.2536;
            d->Vc(i,j,k,VX2) = 0.0;
            d->Vc(i,j,k,VX3) = 0.0;
            d->Vc(i,j,k,PRS) = 1.0;
            d->Vc(i,j,k,BX1) = 0.0;
            d->Vc(i,j,k,BX2) = run_config.inputParam[B_PRE];
            d->Vc(i,j,k,BX3) = run_config.inputParam[B_PRE];
        }

        #if (DIMENSIONS == 3) && (defined STAGGERED_MHD)
        int nx1_tot = grid->ntot[IDIR];
        int nx2_tot = grid->ntot[JDIR];
        int nx3_tot = grid->ntot[KDIR];
        int iend = grid->lend[IDIR];
        #pragma acc parallel loop collapse(3) present(d, run_config)
        for (k = 0; k < nx3_tot+1; k++){
            for (j = 0; j < nx2_tot; j++){
                for (i = iend+1; i < nx1_tot; i++){
                    d->Vs[BX3s](i,j,k) = run_config.inputParam[B_PRE];
                }
            }
        }
        #endif
    }
}

```

5.2.1 Internal Boundary

When `UserDefBoundary()` is called with `side==0` and `INTERNAL_BOUNDARY` has been turned to `YES` inside your `definitions.hpp`, the user has full control over the solution array. This feature can be used to adjust the value of selected cell-centered primitive (and conservative) variables inside a specific region of the computational domain rather than at boundaries. In this case, the `BOX_LOOP()` should be

used to loop over the (local) computational domain and a user-defined criterion (typically spatially- or variable-dependent) is used to modify the solution array in the selected zones.

A typical example may occur when a lower (or upper) threshold value should be imposed on physical variables such as density, pressure or temperature. For instance, the following piece of code sets a floor value for density (`run_config.small_rho`) and pressure (`run_config.small_prs`):

```
void UserDefBoundary (const Data *d, RBox *box, int side, Grid *grid)
{
    int ibeg = grid->lbeg[IDIR], iend = grid->lend[IDIR];
    int jbeg = grid->lbeg[JDIR], jend = grid->lend[JDIR];
    int kbeg = grid->lbeg[KDIR], kend = grid->lend[KDIR];
    RunConfig &run_config = *(d->run_config);

    if (side == 0){
        #pragma acc parallel loop collapse(3) present(d)
        for(int k = kbeg; k <= kend; k++){
            for(int j = jbeg; j <= jend; j++){
                for(int i = ibeg; i <= iend; i++){
                    bool convert = false;
                    double v[NVAR], u[NVAR];
                    if(d->Vc(i,j,k, RHO) < run_config.small_rho) {
                        d->Vc(i,j,k, RHO) = run_config.small_rho;
                        convert = true;
                    }

                    if(d->Vc(i,j,k, PRS) < run_config.small_prs) {
                        d->Vc(i,j,k, PRS) = run_config.small_prs;
                        convert = true;
                    }
                    if(convert){
                        int nv;
                        #pragma acc loop seq
                        NVAR_LOOP(nv) v[nv] = d->Vc(i,j,k, nv);
                        PrimToCons(v,u);
                        #pragma acc loop seq
                        NVAR_LOOP(nv) d->Uc(i,j,k, nv) = u[nv];
                    }
                }
            }
        }
    } /* if (side == 0) */
}
```

Please note that changing a primitive variable in an active zone requires recomputing the conservative variables in that zone as well (this is done by the call to `PrimToCons()`).

A more complex example consists of a time-independent region of space where variables are fixed in time and should not be evolved by the algorithm. If this is the case, you may additionally tell **gPLUTO** not to update the solution in the specified computational zones during the current time step by enabling the `FLAG_INTERNAL_BOUNDARY` flag. Conversion to conservative is not necessary in this case.

By default the Courant condition is not computed inside flagged zones. This is controlled by the macro `INTERNAL_BOUNDARY_CFL` which is set to `NO`, see Appendix B.3.

Example:

The following example (taken from `Test.Problems/HD/Stellar.Wind`) shows how to set up a radially symmetric spherical wind in cylindrical coordinates inside a small spherical region of radius 1 centered around the origin. This is achieved by prescribing fixed inflow values for density, pressure and velocity:

```
void UserDefBoundary (const Data *d, RBox *box, int side, Grid *grid)
{
    int i, j, k, nv;
    double r, r0, cs;
    double Vwind = 1.0, rho, vr;
    double *x1 = grid->xC[IDIR];
    double *x2 = grid->xC[JDIR];
    double *x3 = grid->xC[KDIR];
    double *param = d->run_config->inputParam;
    double gamma = d->run_config->gamma;

    if (side == 0){
        r0 = 1.0;
        cs = param[CS_WIND];
        BOX_LOOP(box,i,j,k){
            #if GEOMETRY == CARTESIAN
            r = sqrt(x1[i]*x1[i] + x2[j]*x2[j] + x3[k]*x3[k]);
            if (r <= r0){
                vr = tanh(r/r0/0.1)*Vwind;
                rho = Vwind*r0*r0/(vr*r*r);
                d->Vc(i,j,k,RHO) = rho;
                d->Vc(i,j,k,VX1) = Vwind*x1[i]/r;
                d->Vc(i,j,k,VX2) = Vwind*x2[j]/r;
            }
        }
    }
}
```

```

        d->Vc(i,j,k,VX3) = Vwind*x3[k]/r;
        d->Vc(i,j,k,PRS) = cs*cs/gamma*pow(rho,gamma);
        d->flag(i,j,k)   |= FLAG_INTERNAL_BOUNDARY;
    }
    #elif GEOMETRY == CYLINDRICAL
    r = sqrt(x1[i]*x1[i] + x3[k]*x3[k]);
    if (r <= r0){
        vr = tanh(r/r0/0.1)*Vwind;
        rho = Vwind*r0*r0/(vr*r*r);
        d->Vc(i,j,k,RH0) = rho;
        d->Vc(i,j,k,VX1) = Vwind*x1[i]/r;
        d->Vc(i,j,k,VX2) = 0.0;
        d->Vc(i,j,k,VX3) = Vwind*x3[k]/r;
        d->Vc(i,j,k,PRS) = cs*cs/gamma*pow(rho,gamma);
        d->flag(i,j,k)   |= FLAG_INTERNAL_BOUNDARY;
    }
    #endif
}
}
...
}

```

The symbol `|=` (a combination of the bitwise OR operator `|` followed by the equal sign) turns the `FLAG_INTERNAL_BOUNDARY` bit on in the 3D array `d->flag[] [] []`. This is used by the code to reset the right hand side of the conservative equations in the selected zones to zero. These computational cells are thus not evolved in time by **gPLUTO**.

Note: The `*box` structure should not be used here and staggered magnetic field variables should not be altered.

Another useful employment of the internal boundary is to mimic the presence of solid (rigid) bodies in the domain. However, in this case, internal boundary values should be reflected according to the reconstruction direction in order for the Riemman solver to yield zero flux at the interface with active zones. In this case, we recommend to enable the macro `INTERNAL_BOUNDARY_REFLECT` (see §B.3).

5.3 Body Forces

Body forces are introduced by enabling the `BODY_FORCE` flag in your definitions.hpp. The force is computed in terms of the acceleration vector $\mathbf{a}$:

$$\mathbf{a} = -\nabla\Phi + \mathbf{g}, \quad (5.5)$$

where Φ is the scalar potential and $\mathbf{g} = (g_1, g_2, g_3)$ is a three-component acceleration vector. Beware that non-inertial effects due to a rotating frame of reference (such as Coriolis and centrifugal forces) should *not* be specified here.

- The scalar potential can be employed when the `BODY_FORCE` flag is set to `POTENTIAL` in your definitions.hpp. In this case, $\mathbf{g} = 0$ and the function `BodyForcePotential()` should be used to prescribe the analytical form of $\Phi \equiv \Phi(x_1, x_2, x_3)$:

```
double BodyForcePotential(const State& state, const RunConfig& run_config)
```

where `state` is a state structure at the local zone coordinates, `run_config` is a reference to the `RunConfig` structure and the return value of the function gives the potential. In this way, **gPLUTO** employs a conservative discretization that conserves total energy+gravitational energy, see Eq. (6.1) and Eq. (6.3). The gravitational potential, however, must not change in time.

As an example, a spherically symmetric point-mass potential $\Phi = -1/r$ can be defined using

```

double BodyForcePotential(const State& state, const RunConfig& run_config)
{
    double x1 = state.x1;
    double x2 = state.x2;
    double x3 = state.x3;

    #if GEOMETRY == CARTESIAN
    return -1.0/sqrt(x1*x1 + x2*x2 + x3*x3);
    
```

```

#elif GEOMETRY == CYLINDRICAL
    return -1.0/sqrt(x1*x1 + x3*x3);
#elif GEOMETRY == SPHERICAL
    return -1.0/x1;
#endif
return -1.0/sqrt(x1*x1 + x2*x2 + x3*x3);
}

```

- The acceleration vector can be employed when the BODY_FORCE flag is set to *VECTOR* and the three components of *g* are prescribed using the function `BodyForceVector()`:

```
void BodyForceVector(const State& state, const RunConfig& run_config, double *g)
```

where

- *state*: a `State` structure containing coordinates and primitive quantities (e.g., *v*[*RHO*], *v*[*VX1*], etc...);
- *run_config*: reference to `RunConfig` structure;
- **g*: a three-component array (*g*[*IDIR*], *g*[*JDIR*], *g*[*KDIR*]) specifying the gravity vector *g* components along the coordinate directions;

As an example, let's consider again a point-mass source located at the origin of coordinates. Then one needs to define, depending on the geometry (= *CARTESIAN*, *CYLINDRICAL* or *SPHERICAL*),

```

void BodyForceVector(const State& state, const RunConfig& run_config, double *g)
{
    double gs, rs;
    double x1 = state.x1;
    double x2 = state.x2;
    double x3 = state.x3;

    #if GEOMETRY == CARTESIAN
        rs = sqrt(x1*x1 + x2*x2 + x3*x3); /* spherical radius in cart. coords */
    #elif GEOMETRY == CYLINDRICAL
        rs = sqrt(x1*x1 + x3*x3); /* spherical radius in cyl. coords */
    #elif GEOMETRY == SPHERICAL
        rs = x1; /* spherical radius in sph. coords */
    #endif

    gs = -1.0/rs/rs; /* spherical gravity */

    #if GEOMETRY == CARTESIAN
        g[IDIR] = gs*x1/rs;
        g[JDIR] = gs*x2/rs;
        g[KDIR] = gs*x3/rs;
    #elif GEOMETRY == CYLINDRICAL
        g[IDIR] = gs*x1/rs;
        g[JDIR] = gs*x2/rs;
        g[KDIR] = 0.0;
    #elif GEOMETRY == SPHERICAL
        g[IDIR] = gs;
        g[JDIR] = 0.0;
        g[KDIR] = 0.0;
    #endif
}

```

It is also possible to prescribe the body force in terms of a vector *and* a potential by setting, in your `definitions.hpp`, BODY_FORCE to (*VECTOR+POTENTIAL*).

A word of caution about using reflective, equatorial symmetric (or similar) boundary conditions: strictly speaking, gravity should be defined consistently with the antisymmetric behavior of the velocity component normal to the given boundary plane. More precisely, the normal component of *g* should be antisymmetric while the potential should be an even function about the boundary plane. Consider, for instance, a reflective (or equatorial symmetric) conditions at the lower and upper boundaries *z_b* and *z_e* in the *z* direction. Then one should have:

$$\begin{cases} g_z(x, y, z_b - z) = -g_z(x, y, z_b + z) \\ \Phi(x, y, z_b - z) = \Phi(x, y, z_b + z) \end{cases}, \quad \begin{cases} g_z(x, y, z_e + z) = -g_z(x, y, z_e - z) \\ \Phi(x, y, z_e + z) = \Phi(x, y, z_e - z) \end{cases}$$

If gravity does not satisfy this property then it must be imposed manually. As an example you can look at the `Test_Problems/MHD/Rayleigh_Taylor` or `Test_Problems/MHD/Shearing_Box` setups where reflective and equatorial symmetric boundaries are used in the *y*- and *z*- directions.

Note: Relativistic flows: Body forces may be used with relativistic modules, only *VECTOR* should be used.

5.4 The Analysis() function

The Analysis() function can be used to perform run-time data analysis/reduction in order to save intensive I/O for data post-processing. This function call frequency is set in pluto.ini, see §4.6.

Syntax:

```
void Analysis (Data *d, Grid *grid)
```

Arguments:

- *d: a pointer to the **gPLUTO** data structure as in §5.2
- *grid: a pointer to the Grid structure containing all the relevant grid information.

Example:

In the next example we show how to compute, at run-time, the total integrated kinetic energy and the maximum internal energy:

$$\langle E_{\text{kin}} \rangle \equiv \frac{1}{\Delta \mathcal{V}} \int \frac{1}{2} \rho v^2 dx dy dz = \frac{1}{\Delta \mathcal{V}} \sum_{i,j,k} \frac{1}{2} \rho_{i,j,k} v_{i,j,k}^2 \Delta \mathcal{V}_{i,j,k} \quad (\rho e)_{\text{max}} = \max_{i,j,k} \left(\frac{p}{\Gamma - 1} \right)$$

where $\Delta \mathcal{V}$ is the total volume of the computational domain, $\Delta \mathcal{V}_{i,j,k} = \Delta x_i \Delta y_j \Delta z_k$ is the volume of a single cell, (i, j, k) extend over the entire computational domain (a Cartesian 3D domain is used for simplicity). The output file name is averages.dat and it is written as a 4-column tabulated ascii file containing the current integration time, the time step, the volume-integrated kinetic energy and maximum internal energy for the required time level. The example works also for parallel computations and can be safely used at restart since the last position of the file is automatically searched for and subsequent writing is appended starting from the correct row.

```

void Analysis (Data *d, Grid *grid)
{
    int    i, j, k;
    double vol, scrh;
    double Ekin, Eth_max;

    static int first_call = 1;
    static RBox box;

    if (first_call == 1){
        RBoxDefine(grid->lbeg[IDIR],grid->lend[IDIR],
                  grid->lbeg[JDIR],grid->lend[JDIR],
                  grid->lbeg[KDIR],grid->lend[KDIR],&box);
        first_call = 0;
        #pragma acc enter data copyin(box)
    }

    /* ---- Set pointer shortcuts ---- */

    /* ---- Main loop ---- */

    Ekin = Eth_max = 0.0;
    #pragma acc parallel loop collapse(3) present(box, grid, d)\
        copy(Ekin,Eth_max) reduction(+:Ekin, max:Eth_max)
    BOX_LOOP(&box, i,j,k){
        double *dx = grid->dx[IDIR];
        double *dy = grid->dx[JDIR];
        double *dz = grid->dx[KDIR];
        double dV = dx[i]*dy[j]*dz[k]; /* Cell volume (Cartesian coordinates) */

        double vx2 = d->Vc(i,j,k,VX1)*d->Vc(i,j,k,VX1); /* x-velocity squared */
        double vy2 = d->Vc(i,j,k,VX2)*d->Vc(i,j,k,VX2); /* y-velocity squared */
        double vz2 = d->Vc(i,j,k,VX3)*d->Vc(i,j,k,VX3); /* z-velocity squared */

        double scrh = 0.5*d->Vc(i,j,k,RHO)*(vx2 + vy2 + vz2); /* cell kinetic energy */
        Ekin      += scrh*dV;

        scrh      = d->Vc(i,j,k,PRS)/(GAMMA_EOS - 1.0); /* cell internal energy */
        Eth_max   = MAX(Eth_max, scrh);
    }

    double vol;
    vol = grid->xend_glob[IDIR] - grid->xbeg_glob[IDIR]; /* Compute total domain volume */
    vol *= grid->xend_glob[JDIR] - grid->xbeg_glob[JDIR];
    vol *= grid->xend_glob[KDIR] - grid->xbeg_glob[KDIR];

    Ekin /= vol; /* Compute kinetic energy average */

    /* ---- Parallel data reduction ---- */

    #ifdef PARALLEL
    MPI_Allreduce (MPI_IN_PLACE, &Ekin, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
    MPI_Allreduce (MPI_IN_PLACE, &Eth_max, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
    MPI_Barrier (MPI_COMM_WORLD);
    #endif

    /* ---- Write ascii file "averages.dat" to disk ---- */

    if (prank == 0){
        char fname[512];
        static double tpos = -1.0;
        FILE *fp;

        sprintf (fname, "%s/averages.dat",RuntimeGet()->output_dir);
        if (g_stepNumber == 0){ /* Open for writing only when we're starting */
            fp = fopen(fname,"w"); /* from beginning */
            fprintf (fp, "#%7s%%12s%%12s%%12s\n", "t", "dt", "<Ekin>","Max(Eth)");
        }else{
            /* Append if this is not step 0 */
            if (tpos < 0.0){ /* Obtain time coordinate of to last written row */
                char sline[512];
                fp = fopen(fname,"r");
                while (fgets(sline, 512, fp)) {}
                sscanf(sline, "%lf\n",&tpos); /* tpos = time of the last written row */
                fclose(fp);
            }
            fp = fopen(fname,"a");
        }
        if (g_time > tpos){ /* Write if current time if > tpos */
            fprintf (fp, "%12.6e%%12.6e%%12.6e%%12.6e\n",g_time, g_dt,Ekin, Eth_max);
        }
        fclose(fp);
    }
}

```

6. Basic Physics Modules

In this chapter we describe the basic equation modules available in the **gPLUTO** code for the solution of the fluid equations under different regimes: HydroDynamics (HD), MagnetoHydroDynamics (MHD) and their relativistic extensions (RHD, and RMHD).

6.1 The HD Module

The HD module may be used to solve the Euler or the Navier-Stokes equations of classical fluid dynamics. The relevant source files and definitions for this module can be found in the `Src/HD` directory.

With the HD module, **gPLUTO** evolves in time following system of conservation laws:

$$\frac{\partial}{\partial t} \begin{pmatrix} \rho \\ \mathbf{m} \\ E_t + \rho\Phi \end{pmatrix} + \nabla \cdot \begin{pmatrix} \rho\mathbf{v} \\ \mathbf{m}\mathbf{v} + p\mathbf{l} \\ (E_t + p + \rho\Phi)\mathbf{v} \end{pmatrix}^T = \begin{pmatrix} 0 \\ -\rho\nabla\Phi + \rho\mathbf{g} \\ \mathbf{m} \cdot \mathbf{g} \end{pmatrix} \quad (6.1)$$

where ρ is the mass density, $\mathbf{m} = \rho\mathbf{v}$ is the momentum density, $\mathbf{v}$ is the velocity, p is the thermal pressure and E_t is the total energy density:

$$E_t = \rho e + \frac{\mathbf{m}^2}{2\rho}. \quad (6.2)$$

An equation of state provides the closure $\rho e = \rho e(p, \rho)$.

The source term on the right includes contributions from body forces and is written in terms of the (time-independent) gravitational potential Φ and the acceleration vector $\mathbf{g}$ (§5.3).

The right hand side of the system of Eqns (6.1) is implemented in the `RightHandSide()` function inside `Src/MHD/rhs.c`¹ employing a conservative discretization that closely follows the expression given in §A.1 for Cartesian, cylindrical and spherical geometries (without magnetic fields).

Primitive variables are defined by $\mathbf{V} = (\rho, \mathbf{v}, p)^T$, where $\mathbf{v} = \mathbf{m}/\rho$ while $p = p(\rho, \rho e)$ depends on the equation of state, see Chapter 7. The maps $\mathbf{U}(\mathbf{V})$ and its inverse are provided by the functions `PrimToCons()` and `ConsToPrim()`.

Primitive variables are generally more convenient and preferred when assigning initial/boundary conditions and in the interpolation algorithms.

¹HD and MHD share the same `RightHandSide()` function.

6.2 The MHD Module

The MHD module is suitable for the solution of ideal or resistive (non-relativistic) magnetohydrodynamical equations. Source and definition files are located inside the Src/MHD directory.

With the MHD module, **gPLUTO** solves the following system of conservation laws:

$$\begin{aligned}
 \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) &= 0 \\
 \frac{\partial \mathbf{m}}{\partial t} + \nabla \cdot \left[\mathbf{m} \mathbf{v} - \mathbf{B} \mathbf{B} + \mathbf{I} \left(p + \frac{\mathbf{B}^2}{2} \right) \right]^T &= -\rho \nabla \Phi + \rho \mathbf{g} \\
 \frac{\partial \mathbf{B}}{\partial t} + \nabla \times (c \mathbf{E}) &= 0 \\
 \frac{\partial (E_t + \rho \Phi)}{\partial t} + \nabla \cdot \left[\left(\frac{\rho \mathbf{v}^2}{2} + \rho e + p + \rho \Phi \right) \mathbf{v} + c \mathbf{E} \times \mathbf{B} \right] &= \mathbf{m} \cdot \mathbf{g}
 \end{aligned} \tag{6.3}$$

where ρ is the mass density, $\mathbf{m} = \rho \mathbf{v}$ is the momentum density, $\mathbf{v}$ is the velocity, p is the gas (thermal) pressure, $\mathbf{B}$ is the magnetic field² and E_t is the total energy density:

$$E_t = \rho e + \frac{\mathbf{m}^2}{2\rho} + \frac{\mathbf{B}^2}{2}. \tag{6.4}$$

where an additional equation of state provides the closure $\rho e = \rho e(p, \rho)$ (see Chapter 7). The source term on the right includes contributions from body forces and is written in terms of the (time-independent) gravitational potential Φ and the acceleration vector $\mathbf{g}$ (see §5.3).

In the third of Eq. (6.3), $\mathbf{E}$ is the electric field defined by the expression

$$c \mathbf{E} = -\mathbf{v} \times \mathbf{B} \tag{6.5}$$

Note that the speed of light c never enters in the equations but we keep it for the sake of completeness. In the ideal case, the energy flux takes the form:

$$\frac{\partial (E_t + \rho \Phi)}{\partial t} + \nabla \cdot [(E_t + p_t + \rho \Phi) \mathbf{v} - \mathbf{B} (\mathbf{v} \cdot \mathbf{B})] = \mathbf{m} \cdot \mathbf{g}. \tag{6.6}$$

The hyperbolic contributions from the divergences terms on the left-hand side of Eqns (6.3) are implemented in the `RightHandSide()` function inside `Src/MHD/rhs.c` employing a conservative discretization that closely follows the expression given in §A.1.1, §A.1.2 and §A.1.3 for Cartesian, cylindrical and spherical geometries.

The sets of conservative and primitive variables $\mathbf{U}$ and $\mathbf{V}$ are given by:

$$\mathbf{U} = \left(\rho, \mathbf{m}, E_t, \mathbf{B} \right)^T, \quad \mathbf{V} = \left(\rho, \mathbf{v}, p, \mathbf{B} \right)^T.$$

The maps $\mathbf{U}(\mathbf{V})$ and its inverse are provided by the functions `PrimToCons()` and `ConsToPrim()`.

6.2.1 Assigning Magnetic Field Components

Magnetic field components are initialized in your `Init()` function just like any other flow quantity. Depending on the value of `ASSIGN_VECTOR_POTENTIAL` in your `definitions.h`, two alternative initializations are possible:

1. By setting `ASSIGN_VECTOR_POTENTIAL` to `NO` (default) you can assign the component of magnetic field in the usual way by directly prescribing the values for `us[BX1]`, `us[BX2]` and `us[BX3]`.
2. When `ASSIGN_VECTOR_POTENTIAL` is set to `YES`, the vector potential $\mathbf{A}$ is used instead and the magnetic field is recovered from $\mathbf{B} = \nabla \times \mathbf{A}$. This option guarantees that the initial field has zero divergence in the discretization which is more appropriate for the underlying formulation (i.e., cell or face centered fields, §6.2.2).

²A factor of $1/\sqrt{4\pi}$ has been absorbed in the definition of magnetic field.

Note: In 2D, only the third component of A (that is us[AX3]) should be used. Likewise, the third component of the magnetic field (B_z) cannot be assigned through the vector potential and must be prescribed in the standard way, see the third example in Table 6.1.

Table 6.1 shows some examples of magnetic field initializations with and without using the vector potential.

Magnetic Field	Standard	Using Vector Potential
$B = (0, 5, 0)$ Cartesian, 2D	$v[BX1] = 0.0;$ $v[BX2] = 5.0;$ $v[BX3] = 0.0;$	$v[AX1] = 0.0;$ $v[AX2] = 0.0;$ $v[AX3] = -x1*5.0;$
$B = (0, 5, 0)$ Cylindrical,	$v[BX1] = 0.0;$ $v[BX2] = 0.0;$ $v[BX3] = 0.0;$	$v[AX1] = 0.0;$ $v[AX2] = 0.5*x1*5.0;$ $v[AX3] = 0.0;$
$B = (-\sin y, \sin 2x, 2)$ Cartesian, 2.5D	$v[BX1] = -\sin(x2);$ $v[BX2] = \sin(2.0*x1);$ $v[BX3] = 2.0;$	$v[AX1] = 0.0;$ $v[AX2] = 0.0;$ $v[AX3] =$ $\cos(x2)+0.5*\cos(2.0*x1);$ $v[BX3] = 2.0;$

Table 6.1: Examples of how the magnetic field may be initialized. Direct initialization (standard) is possible when ASSIGN_VECTOR_POTENTIAL is set to *NO*. Otherwise, the components of the vector potential are used (third column).

6.2.2 Controlling the $\nabla \cdot B = 0$ Condition

6.2.2.1 Eight-Wave Formulation

In the eight-wave formalism [Pow94, PRL⁺99, Jan00, Jan00, BKW07, WFK11] the magnetic field has a cell-centered representation. In the ideal case, additional source terms are added on the right hand side of Eqns (6.3):

$$\frac{\partial}{\partial t} \begin{pmatrix} \rho \\ m \\ E_t \\ B \end{pmatrix} + \nabla \cdot (...) = -\nabla \cdot B \begin{pmatrix} 0 \\ B \\ v \cdot B \\ v \end{pmatrix} \quad (6.7)$$

Contributions to $\nabla \cdot B$ are taken direction by direction in the right hand side function and are added to momentum, induction and energy equation. However, when EIGHT_WAVES_REDUCED is set to *YES* source terms are only added to the induction equation, as shown [Jan00, BKW07], see §B.3. This has shown to be more robust in some situations, including HLL-type Riemann solvers.

Note that the 8-wave method keeps $\nabla \cdot B = 0$ only at the truncation level and NOT to machine accuracy. More accurate treatments of the solenoidal condition can be achieved using the other two formulations. The 8-wave algorithm should be used in conjunction any Riemann solver with the exception of *h11d*.

6.2.2.2 Hyperbolic Divergence Cleaning

In [DKK⁺02] (see also [MT10, MTB10] for additional discussion), the divergence free constraint is enforced by solving a modified system of conservation laws, where the induction equation is coupled to a generalized Lagrange multiplier (GLM). Using the mixed GLM hyperbolic/parabolic correction, the induction equation and the solenoidal constraint are replaced, respectively, by

$$\begin{cases} \frac{\partial B}{\partial t} + \nabla \times (cE) + \nabla \psi = 0 \\ \frac{\partial \psi}{\partial t} + c_h^2 \nabla \cdot B = -\frac{c_h^2}{c_p^2} \psi \end{cases} \quad (6.8)$$

where $c_h = \text{CFL} \times \Delta l_{\min} / \Delta t^n$ is maximum speed compatible with the step size, $c_p = \sqrt{\Delta l_{\min} c_h / \alpha}$ and $\Delta l_{\min}$ is the minimum cell length. The free parameter α controls the rate at which monopole are damped [MT10] and its value is set by the user-defined constant GLM_ALPHA (default value 0.1). A number of tests suggests that the

optimal range can be found for $0.05 \lesssim \alpha \lesssim 0.3$. In the mixed formulation, divergence errors are transported to the domain boundaries with the maximal admissible speed and are damped at the same time. By default, ψ is set to zero in the initial and boundary conditions but the user is free to change it at a user-defined boundary by prescribing $d \rightarrow \nabla c(i, j, k, \text{PSI_GLM})$ (inside `UserDefBoundary()`) which has the usual cell-centered representation. The scalar multiplier is not written to disk except for the double format, §11, needed for restart. The advantage of this formulation (GLM-MHD) is that the equations retain a conservative form (no source terms are introduced), all variables (including magnetic fields) retain a cell-centered representation and standard 7-wave Riemann solvers (with a single value of the normal component of magnetic field) may be used.

A slightly different version, called the *extended GLM* formulation, that breaks momentum and energy conservation, has been found to be more robust in problems involving strongly magnetized media (see, for example, configuration # 11 in `Test.Problems/MHD/Blast`). At present, this switch has been successfully tested for the classical MHD module only. The extended form of the equations [DKK⁺02, MT10] can be enabled by adding the user-defined constant `GLM_EXTENDED` to `definitions.hpp` and setting its value to `YES`. For a complete description of the GLM- and Extended GLM-MHD formulation and its implementation in **gPLUTO** refer to [MT10, MTB10].

6.2.2.3 Constrained Transport (CT)

In this formulation [BS99, Ld04, GS05], two sets of magnetic fields are used:

- face-centered magnetic field ($B^{(s)}$ hereafter);
- cell-centered magnetic field (B hereafter).

The primary set is the first one, where the three components of the field are located at different spatial points in the control volume, that is

$$B_{x_f}^{(s)} \equiv B_{x_1, i - \frac{1}{2}, j, k} \quad , \quad B_{y_f}^{(s)} \equiv B_{x_2, i, j - \frac{1}{2}, k} \quad , \quad B_{z_f}^{(s)} \equiv B_{x_3, i, j, k - \frac{1}{2}} \quad (6.9)$$

see Fig. 6.1. In Cartesian coordinates, for instance, $B_{x_f}^{(s)}$ is located at x -faces whereas $B_{y_f}^{(s)}$ lives at x_2 -faces and so on, see the boxes and triangles in Fig. 5.1. With CT, the solenoidal condition is maintained at machine accuracy as long as field initialization is done using the vector potential, §6.2.1.

The staggered magnetic field is treated as an area-weighted average on the zone face and Stoke's theorem is used to update it:

$$\int \left(\frac{\partial B^{(s)}}{\partial t} + \nabla \times E \right) \cdot dS_d = 0 \quad \implies \quad \frac{dB_{x_d}^{(s)}}{dt} + \frac{1}{S_d} \oint E \cdot dl = 0 \quad (6.10)$$

Please note that the staggered components are initialized and integrated also on the boundary interfaces in the corresponding staggered direction. In other words, the interior values are

$$B_{x_f}^{(s)} : \quad \text{ibeg} \leq i \leq \text{iend} + 1, \quad \text{jbeg} \leq j \leq \text{jend}, \quad \text{kbeg} \leq k \leq \text{kend}$$

$$B_{y_f}^{(s)} : \quad \text{ibeg} \leq i \leq \text{iend}, \quad \text{jbeg} \leq j \leq \text{jend} + 1, \quad \text{kbeg} \leq k \leq \text{kend}$$

$$B_{z_f}^{(s)} : \quad \text{ibeg} \leq i \leq \text{iend}, \quad \text{jbeg} \leq j \leq \text{jend}, \quad \text{kbeg} \leq k \leq \text{kend} + 1$$

Thus $B_{x_f}^{(s)}$ is NOT a boundary value for $i = \text{ibeg}$, $\text{jbeg} \leq j \leq \text{jend}$, $\text{kbeg} \leq k \leq \text{kend}$ but it is considered part of the solution. Similar considerations hold for $B_{y_f}^{(s)}$ and $B_{z_f}^{(s)}$ components at the x_2 and x_3 boundaries, respectively.

The electric field E is computed at zone edges, see Fig. 6.1 by a proper averaging/reconstruction scheme (set by `CT.EMF_AVERAGE` inside your `definitions.h`) selected among

- *ARITHMETIC*: yields a simple arithmetic averaging [BS99] of the fluxes computed during the upwind steps. Despite its simplicity, this average procedure may suffer from insufficient dissipation in some circumstances ([GS05, Ld04]) and does not reduce to its one dimensional equivalent algorithm for plane parallel grid aligned flows.
- *CT_FLUX*: Similar to *ARITHMETIC* but it doubles the amount of dissipation at zone edges and it reduces to the one dimensional algorithm for grid-aligned flows;
- *CT_CONTACT*: It employs the face-to-edge integration procedures proposed by [GS05], where electromotive force derivatives are selected according to the sign of the contact mode. For grid-aligned flows, it reduces to the correct one-dimensional algorithm.
- *UCT_HLL*: two dimensional Riemann solver based on a four-state HLL flux function, see [DBL03, Ld04].
- *UCT_HLLD*: A multidimensional extension of the HLLD Riemann solver for the induction system. This is described in the paper by Mignone & Del Zanna (2020) [MZ20].
- *UCT_GFORCE*: A multidimensional extension of the GFORCE solver for the induction system. This is described in the paper by Mignone & Del Zanna (2020) [MZ20].

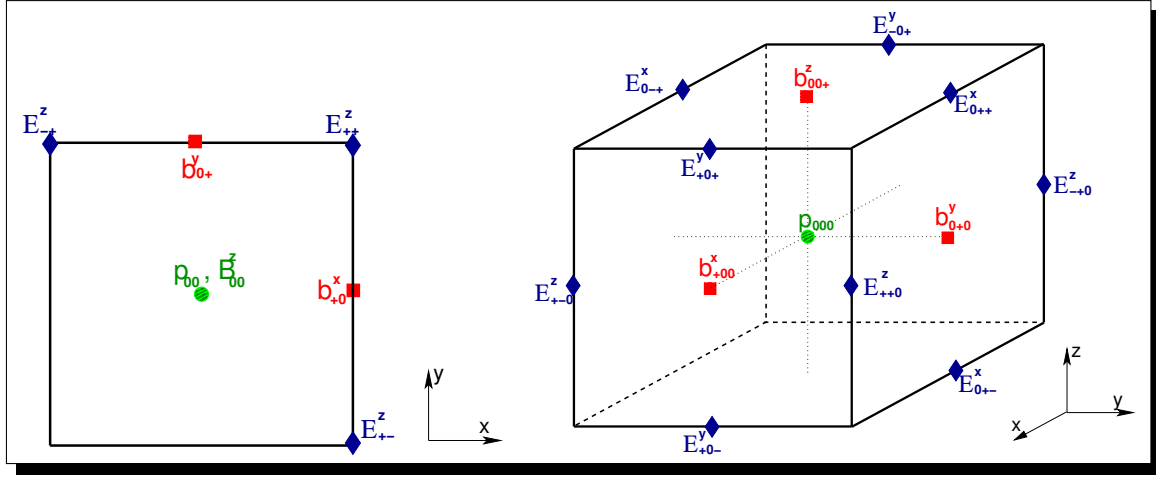


Figure 6.1: Collocation points in 2 D (left) and in 3D (right). Cell-centered quantities are given as green circles, face-centered quantities (magnetic field) as red squares and edge-centered values (electric field) as blue diamonds.

Options are described in the paper by [MZ20]

UCT averaging schemes employ only staggered field components in the definition of the fluid numerical fluxes in the corresponding inter-cell positions, in order to avoid the formation of magnetic monopoles [Ld04, MZ20]. Non-UCT schemes relies on electric field evaluation from the cell interface where fluxes give the following components:

$$\begin{pmatrix} 0 \\ -E_{x_3} \\ E_{x_2} \end{pmatrix}_{i-\frac{1}{2},j,k}, \quad \begin{pmatrix} E_{x_3} \\ 0 \\ -E_{x_1} \end{pmatrix}_{i,j-\frac{1}{2},k}, \quad \begin{pmatrix} -E_{x_2} \\ E_{x_1} \\ 0 \end{pmatrix}_{i,j,k-\frac{1}{2}}$$

during the x_1, x_2 and x_3 sweeps, respectively.

All algorithms, with the exception of the arithmetic averaging, reduce to the corresponding one dimensional scheme for grid aligned flows. The CT formulation works with any of the Riemann solvers.

Assigning Boundary Conditions. Within the CT framework, user-defined boundary conditions (b.c.) must be assigned on the staggered components as well. This is done in your `UserDefBoundary()` function using the `d→Vs[nv](i, j, k)` array, where `nv` gives the staggered component: `BX1s`, `BX2s` or `BX3s`.

Note: In **gPLUTO** we follow the convention that the cell “center” owns its left interface, e.g., ‘i’ means $i - \frac{1}{2}$. Thus:

$$\begin{aligned} B_{x_f}^{(s)} &\equiv d \rightarrow Vs[BX1s](i, j, k); \\ B_{y_f}^{(s)} &\equiv d \rightarrow Vs[BX2s](i, j, k); \\ B_{z_f}^{(s)} &\equiv d \rightarrow Vs[BX3s](i, j, k); \end{aligned}$$

where the subscripts x_f, y_f and z_f are defined in Eq. (6.9).

As the three staggered components have *different spatial locations* and the macros `BOXI_LOOP()`, `BOXJ_LOOP()` and `BOXK_LOOP()` should be used to correctly loop over the boundary ghost zones for each component, respectively. Thus, at the x_1 boundary, for instance, one needs to assign

$$\left. \begin{aligned} B_{y_f}^{(s)} &\quad \text{at } x_{1,i}, x_{2,j-\frac{1}{2}}, x_{3,k} \\ B_{z_f}^{(s)} &\quad \text{at } x_{1,i}, x_{2,j}, x_{3,k-\frac{1}{2}} \end{aligned} \right\} \quad \text{for } i = 0, \dots, \text{ibeg}$$

The component normal to the interface ($B_{x_f}^{(s)}$ in this case) should *NOT* be assigned since it is automatically computed by **gPLUTO** from the $\nabla \cdot \mathbf{B} = 0$ condition after the tangential components have been set.

Example:

The following example prescribes user-defined boundary conditions at the lower x_2 boundary for a MHD jet problem in cylindrical coordinates ($x_1 \equiv R, x_3 \equiv z$).

Inflow conditions are obtained by the function `GetJetVvalues()` for $R \leq 1$ while a symmetric counter-jet is assumed for $R > 1$:

```

if (side == X3_BEG){
    BOX_LOOP(box,i,j,k){
        x1 = grid->xC[IDIR];

        GetJetValues (x1[i], vjet, run_config);

        /* -- copy and reflect ambient medium values -- */

        NVAR_LOOP(nv) vout[nv] = d->Vc(i,j,2*kbeg-k-1,nv);
        vout[VX3] *= -1.0;
        #if PHYSICS == MHD
        vout[BX1] *= -1.0;
        vout[BX2] *= -1.0;
        #endif
        NVAR_LOOP(nv){
            fp = Profile(x1[i], nv, run_config);
            d->Vc(i,j,k,nv) = vout[nv] - (vout[nv] - vjet[nv])*fp;
        }
    }

    #ifdef STAGGERED_MHD
    x1 = grid->xL[IDIR];
    vjet[BX1] = 0.0;
    BOXI_LOOP(box,i,j,k){
        fp = Profile(fabs(x1[i]), BX1, run_config);
        vout[BX1] = -d->Vs[BX1s](i,j,2*kbeg - k - 1);
        d->Vs[BX1s](i,j,k) = vout[BX1] - (vout[BX1] - vjet[BX1])*fp;
    }
    #endif
}

```

Here STAGGERED_MHD is defined only in the MHD constrained transport and the boundary conditions are assigned on $B_{x_1}^{(s)} \equiv B_R^{(s)}$ only (i.e. the orthogonal component).

Note: In some circumstances it may be helpful to assign the electric field along the domain borders. **gPLUTO** allows the user to perform this operation by enabling the CT_EMF_BOUNDARY macro and then adding the function

```

#if CT_EMF_BOUNDARY == YES
void CT_EMF_Boundary(const Data *data, Grid *grid)
{
    int i,j,k;
    int ibeg = grid->lbeg[IDIR], iend = grid->lend[IDIR] + INCLUDE_IDIR;
    int jbeg = grid->lbeg[JDIR], jend = grid->lend[JDIR] + INCLUDE_JDIR;
    int kbeg = grid->lbeg[KDIR], kend = grid->lend[KDIR] + INCLUDE_KDIR;
    EMF *emf = data->emf;

    if (grid->lbound[IDIR] != 0){
        i = ibeg;
        for (k = kbeg; k <= kend; k++){
            for (j = jbeg; j <= jend; j++){
                emf->Ee[KDIR](i,j,k) = 0.0;
            }
        }
    }
}
#endif

```

to your init.cpp. Here $E_e[i,j,k]$ specifies the line-average electromotive force at cell edges. An example can be found in the 12th configuration of Test.Problems/RMHD/Jet.

6.2.3 Background Field Splitting

In situations where an external current-free background magnetic field is present (e.g. planetary magnetosphere, stellar dipole fields), it may be convenient to write the total magnetic field as $\mathbf{B}(x,t) = \mathbf{B}_0(x) + \mathbf{B}_1(x,t)$ where $\mathbf{B}_0$ is a background curl-free magnetic field and $\mathbf{B}_1(x,t)$ is a deviation. The background field must satisfy the following conditions:

$$\frac{\partial \mathbf{B}_0}{\partial t} = 0, \quad \nabla \cdot \mathbf{B}_0 = 0, \quad \nabla \times \mathbf{B}_0 = 0.$$

In this case one can show [Pow94] that the MHD equations reduce to:

$$\begin{aligned}\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) &= 0 \\ \frac{\partial \mathbf{m}}{\partial t} + \nabla \cdot (\mathbf{m} \mathbf{v} - \mathbf{B}_1 \mathbf{B} - \mathbf{B}_0 \mathbf{B}_1) + \nabla p_t &= \rho(-\nabla \Phi + \mathbf{g}) \\ \frac{\partial (E_{t1} + \rho \Phi)}{\partial t} + \nabla \cdot [(E_{t1} + p_t + \rho \Phi) \mathbf{v} - (\mathbf{v} \cdot \mathbf{B}_1) \mathbf{B}] &= \mathbf{m} \cdot \mathbf{g} \\ \frac{\partial \mathbf{B}_1}{\partial t} - \nabla \times (\mathbf{v} \times \mathbf{B}) &= 0\end{aligned}$$

where

$$p_t = p + \frac{\mathbf{B}_1^2}{2} + \mathbf{B}_1 \cdot \mathbf{B}_0, \quad E_{t1} = \frac{p}{\Gamma - 1} + \frac{1}{2} (\rho v^2 + \mathbf{B}_1^2)$$

Thus the energy depends only on $\mathbf{B}_1$, a feature that turns out to be useful when dealing with low-beta plasma. The sets of conservative and primitive variables are the same as the original ones, with $\mathbf{B} \rightarrow \mathbf{B}_1$, $E \rightarrow E_{t1}$.

To enable this feature, the macro `BACKGROUND_FIELD` must be turned to `YES` in your definitions.h. Initial and boundary conditions must be imposed on $\mathbf{B}_1$ while the function `BackgroundField()` can be added to your init.c to assign $\mathbf{B}_0$:

```
void BackgroundField (State &state, RunConfig &run_config)
```

Note that when writing output datafiles to disk, only the deviation $\mathbf{B}_1$ is written. Examples can be found in `Test_Problems/MHD/Rotor/`, `Test_Problems/MHD/Blast/` and `Test_Problems/MHD/Torus`.

Note: Not all Riemann solvers support background field splitting.

6.3 The RHD Module

The RHD module implements the equations of special relativistic fluid dynamics in 1, 2 or 3 dimensions. Velocities are always assumed to be expressed in units of the speed of light. The special relativistic module comes with 2 different equations of state, and it also works in curvilinear coordinates. Gravity in Newtonian approximation can also be incorporated. The relevant source files and definitions for this module can be found in the `Src/RHD` directory.

The special relativistic module evolves the conservative set $\mathbf{U}$ of state variables

$$\mathbf{U} = \left(D, m_1, m_2, m_3, E_t \right)^T$$

where D is the laboratory density, $m_{x1,x2,x3}$ are the momentum components, E_t is the total energy (including contribution from the rest mass). The evolutionary conservative equations are

$$\frac{\partial}{\partial t} \begin{pmatrix} D \\ \mathbf{m} \\ E_t \end{pmatrix} + \nabla \cdot \begin{pmatrix} D\mathbf{v} \\ \mathbf{mv} + p\mathbf{l} \\ \mathbf{m} \end{pmatrix}^T = \begin{pmatrix} 0 \\ \rho\mathbf{a} \\ \rho\gamma\mathbf{v} \cdot \mathbf{a} \end{pmatrix} \quad (6.11)$$

where $\mathbf{v}$ is the velocity, p is the thermal pressure. Primitive variables $\mathbf{V}$ always include the rest-mass density ρ , three-velocity $\mathbf{v} = (v_{x1}, v_{x2}, v_{x3})$ and pressure p . The relation between $\mathbf{U}$ and $\mathbf{V}$ is expressed by

$$D = \rho\gamma, \quad \mathbf{m} = \rho h\gamma^2\mathbf{v} = \rho h\gamma\mathbf{u}, \quad E_t = \rho h\gamma^2 - p,$$

where h is the specific enthalpy (see Chapter 7 for available equation of states). The expressions for the acceleration term $\mathbf{a}$ - in the case of gravity - are approximated as the Newtonian limit of general relativity.

Spatial reconstruction may be performed on the four-velocity rather than on the three-velocity by enabling the macro `RECONSTRUCT_4VEL` to `YES` manually in your `definitions.h` (see also Appendix B.3). Using the four-velocity in place of the three-velocity offers (in some circumstances) the advantage that the total velocity $|\mathbf{v}| = |\mathbf{u}|/\sqrt{1+\mathbf{u}^2}$ is always less than 1 by construction, for any $0 \leq |\mathbf{u}| < \infty$. This is not always the case when the three-velocity is used and precautionary measures are used to ensure that $|\mathbf{v}| < 1$.

6.4 The RMHD Module

The RMHD module implements the equations of (ideal) special relativistic magnetohydrodynamics in 1, 2 or 3 dimensions. Velocities are always assumed to be expressed in units of the speed of light. Source and definition files are located inside the `Src/RMHD` directory.

The RMHD module solves the following system of conservation laws:

$$\frac{\partial}{\partial t} \begin{pmatrix} D \\ \mathbf{m} \\ E_t \\ \mathbf{B} \end{pmatrix} + \nabla \cdot \begin{pmatrix} D\mathbf{v} \\ w_t\gamma^2\mathbf{v}\mathbf{v} - \mathbf{b}\mathbf{b} + \mathbf{I}p_t \\ \mathbf{m} \\ \mathbf{v}\mathbf{B} - \mathbf{B}\mathbf{v} \end{pmatrix}^T = \begin{pmatrix} 0 \\ \rho\mathbf{a} \\ \rho\gamma\mathbf{v} \cdot \mathbf{a} \\ \mathbf{0} \end{pmatrix} \quad (6.12)$$

where D is the laboratory density, $\mathbf{m}$ is the momentum density, E is the total energy (including contribution from the rest mass) while $\mathbf{f}_g$ is an acceleration term (see 6.3).

Primitive variables are similar to the RHD module but they also contain the magnetic field, $\mathbf{V} = (\rho, \mathbf{v}, p, \mathbf{B})$. The relation between $\mathbf{V}$ and $\mathbf{U}$ is

$$\begin{aligned} D &= \gamma\rho \\ \mathbf{m} &= w_t\gamma^2\mathbf{v} - b^0\mathbf{b} \\ E_t &= w_t\gamma^2 - b^0b^0 - p_t \end{aligned}, \quad \begin{cases} b^0 = \gamma\mathbf{v} \cdot \mathbf{B} \\ \mathbf{b} = \mathbf{B}/\gamma + \gamma(\mathbf{v} \cdot \mathbf{B})\mathbf{v} \\ w_t = \rho h + \mathbf{B}^2/\gamma^2 + (\mathbf{v} \cdot \mathbf{B})^2 \\ p_t = p + \frac{\mathbf{B}^2/\gamma^2 + (\mathbf{v} \cdot \mathbf{B})^2}{2} \end{cases}$$

Notice that the components of the momentum tensor may also be written as:

$$M^{ij} = w_t u^i u^j - b^i b^j = m^i v^j - \frac{b^i B^j}{\gamma} = m^i v^j - \left(\frac{B^i}{\gamma^2} + v^i \mathbf{v} \cdot \mathbf{B} \right) B^j$$

The available equations of state are *IDEAL* and *TAUB* already introduced for the RHD module (see [MM07] for the extension of this EOS to the RMHD equations).

The RMHD sub-menu offers some of the switches already discussed in the MHD module (§6.2) or in the RHD (§6.3) module. Divergence control is achieved using the same algorithms introduced for MHD, namely: 8-wave (§6.2.2.1), divergence cleaning (§6.2.2.2) and the constrained transport (§6.2.2.3).

Computation of the fast characteristic speeds can be performed by replacing the numerical solution of a quartic equation (see [MM07]) with the analytical solution of an approximate quadratic equations thus making computation faster. This is achieved by setting `RMHD_FAST_EIGENVALUES` to *YES* (as in [DZBL07]), see the Appendix B.3.

6.5 The Resistive RMHD (ResRMHD) Module

Note: This module is not part of the public code release, see “Terms & Conditions of Use” at the beginning of this guide

The ResRMHD module deals with the non-ideal relativistic MHD equations using the approaches discussed in [MMBD19]. Source and definition files are located inside the Src/ResRMHD directory.

The set of resistive relativistic equations arising from the time and space split of the covariant are, in vectorial form,

$$\begin{aligned}
 \frac{\partial D}{\partial t} + \nabla \cdot (D\mathbf{v}) &= 0, \\
 \frac{\partial \mathbf{m}}{\partial t} + \nabla \cdot (w\mathbf{u}\mathbf{u} + p\mathbf{I} + \mathbf{T}) &= 0, \\
 \frac{\partial \mathcal{E}}{\partial t} + \nabla \cdot \mathbf{m} &= 0, \\
 \frac{\partial \mathbf{B}}{\partial t} + \nabla \times \mathbf{E} &= 0, \\
 \frac{\partial \mathbf{E}}{\partial t} - \nabla \times \mathbf{B} &= -\mathbf{J},
 \end{aligned} \tag{6.13}$$

where $\mathbf{I}$ is the identity matrix and the fluid conserved variables are the density $D = \rho\gamma$ as measured in the laboratory frame, the total momentum density $\mathbf{m} = w\gamma\mathbf{u} + \mathbf{E} \times \mathbf{B}$, and the total energy density

$$\mathcal{E} = w\gamma^2 - p + \mathcal{P}_{\text{EM}}. \tag{6.14}$$

In the expressions above, $w = \varepsilon + p$ is the specific enthalpy and $\mathcal{P}_{\text{EM}} = (E^2 + B^2)/2$ denotes the EM energy density. Finally,

$$\mathbf{T} = -\mathbf{E}\mathbf{E} - \mathbf{B}\mathbf{B} + \frac{1}{2}(E^2 + B^2)\mathbf{I} \tag{6.15}$$

is the Maxwell’s stress tensor. Note that, unlike the MHD and RMHD modules, the electric field vector $\mathbf{E}$ is an independent variable and it is evolved through Ampere’s law (the last equation in 6.13).

The current density vector $\mathbf{J}$ comes from Ohm’s law ($\mathbf{J}' = \sigma\mathbf{E}'$ in the rest frame) which, in the lab frame, reads

$$\mathbf{J} = \frac{1}{\eta} \left[\gamma\mathbf{E} + \mathbf{u} \times \mathbf{B} - (\mathbf{E} \cdot \mathbf{u})\mathbf{v} \right] + q\mathbf{v}, \tag{6.16}$$

where $q = \nabla \cdot \mathbf{E}$ from Gauss’ law, so that the current is determined once the fluid velocity and the electromagnetic fields are known, for a given value of the resistivity η .

The ResRMHD sub-menu has several switches in common with the RMHD and RHD modules (§6.3, §6.4), including equations of state and divergence control methods (§6.2.2.2) such as the constrained transport (§6.2.2.3) which can be used also for the divergence of the electric field (see [MMBD19]). When employing constrained transport (CT), magnetic field always has a staggered representation while the electric field may be discretized at cell faces or at zone centers. This option is controlled by the macro `DIVE.CONTROL` (visible from the Python menu):

- **CONSTRAINED_TRANSPORT**: the electric field has a staggered formulation (default, according to [MMBD19]). Charge is computed at cell centers from staggered components.
- **DIV.CLEANING**: the electric field has a cell-centered representation, and charge is evolved explicitly in time. A Lagrangian multiplier (`PHI.GLM`) is introduced to couple Ampere’s law and charge conservation.
- **NO**: the electric field has a cell-centered representation, charge is computed simply from the electric field derivatives.

Table 6.2 summarizes the possible combination and the number of variables

Table 6.2: Summary of charge computation options

div(B)	div(E)	NVAR	Primary B	Primary E
NO	NO	11	Zone-center	Zone-center
CT	NO	11	Face-center	Zone-center
CT	CT	11	Face-center	Face-center
GLM	NO	12	Zone-center	Zone-center
GLM	GLM	13	Zone-center	Zone-center

The ResRMHD module works only with RK time-stepping (IMEX schemes) and supports only Cartesian coordinates.

6.5.1 Controlling Resistivity

The resistivity is defined through the function `Resistive_eta()`:

Syntax:

```
double Resistive_eta(const State& state, const RunConfig& run_config)
```

Arguments:

- `state`: the `State` structure;
- `run_config`: The `RunConfig` structure;

7. Equation of State

In the current implementation, **gPLUTO** allows the user to choose among different Equation of State (EoS). All equations of state are coded in `Src/eos.hpp`

The most common one describes a thermally ideal gas obeying the *thermal* Equation of State (EOS)

$$p = nk_B T = \frac{\rho}{m_u \mu} k_B T \quad (7.1)$$

where p is the pressure, n is the total particle number density, k_B is the Boltzmann constant, T is the temperature, ρ is the density, m_u is the atomic mass unit and μ is the mean molecular weight. The thermal EOS describes the thermodynamic state of a plasma in terms of its pressure p , density ρ , temperature T and chemical composition μ . Eq. (7.1) is written in CGS physical units. Using code units for p and ρ while leaving temperature in Kelvin, the thermal EOS is conveniently re-expressed as

$$p = \frac{\rho T}{\mathcal{K} \mu} \quad \Longleftrightarrow \quad T = \frac{p}{\rho} \mathcal{K} \mu \quad \left(\text{where } \mathcal{K} = \frac{m_u v_0^2}{k_B} \right) \quad (7.2)$$

where $\mathcal{K}$ is the `KELVIN` macro which depends explicitly on the value of `UNIT_VELOCITY`.

Another fundamental quantity is the (specific) internal energy e whose rate of change under a physical process is regulated by the first law of thermodynamics:

$$de = dQ - p d \left(\frac{1}{\rho} \right). \quad (7.3)$$

where Q represents the heat absorbed or released. The internal energy is a state function of the system and can also be related to temperature and density via the *caloric* equation of state [Tor97]

$$e = e(T, \rho). \quad (7.4)$$

The *thermal* and *caloric* equations of state given by Eq. (7.2) and (7.4) constitutes the basis for the consideration discussed in the next sections.

7.1 The ISOTHERMAL Equation of State

In an isothermal gas, the temperature is constant and the pressure is readily obtained as

$$p = \rho c_{\text{iso}}^2 \quad (7.5)$$

where c_{iso} (the isothermal sound speed) can be either a constant value or a spatially-varying quantity. This EOS is available only in the HD and MHD modules. No energy equation is present and the labels `ENG` and `PRS` are undefined. The value of a constant c_{iso} can be set using the variable `run_config.isoSoundSpeed` in your `init.cpp`, e.g.

```
run_config.isoSoundSpeed = 2.0; /* sets the sound speed to be 2 */
```

If not set, the default value is $c_{\text{iso}} = 1$.

In order to have a space-dependent isothermal speed of sound, one has to copy the source file `Src/eos.hpp` to your local working directory and make the appropriate modifications, see, e.g., `Test.Problems/HD/Disk_Planet` where $c_s = a/\sqrt{R}$.

7.2 The IDEAL Equation of State

For a calorically ideal gas, the ratio of specific heats Γ is constant and the internal energy can be written

$$\rho e = \frac{p}{\Gamma - 1}. \quad (7.6)$$

The value of Γ is stored in the global variable `g_gamma` and can be modified in your `Init()` function (default value 5/3).

For a relativistic flow, the constant- Γ EOS is more conveniently expressed through the specific enthalpy:

$$\rho h = \rho + \frac{\Gamma}{\Gamma - 1} p. \quad (7.7)$$

The ideal EOS is compatible with all physics modules, algorithms and Riemann solvers in the code.

7.3 The TAUB Equation of state

The Taub-Matthews (TM) equation of state is available to describe a relativistic perfect gas, for which the adiabatic exponent is a function of the temperature. The actual expression for the Synge gas [Syn57] is rather complex and **PLUTO** employs a quadratic approximation to the theoretical relativistic perfect gas EOS ($\Gamma \rightarrow 5/3$ in the low temperature limit, and $\Gamma \rightarrow 4/3$ in the high temperature limit), see [MPB05, MM07]:

$$\left(h - \frac{p}{\rho}\right) \left(h - 4\frac{p}{\rho}\right) = 1, \quad (7.8)$$

where h is the specific enthalpy related to the internal energy and pressure through

$$h = 1 + e + \frac{p}{\rho}.$$

8. Optically Thin Cooling

PLUTO can include time-dependent optically thin radiative losses in a fractional step formalism in which the hydrodynamical evolution and the source step are solved separately using Strang operator splitting:

$$S^{\Delta t/2} A^{\Delta t} S^{\Delta t/2}(U) \quad (8.1)$$

where $S^{\Delta t/2}$ and $A^{\Delta t}$ are the cooling source step (for half time step) and the hydro advance step (for a full step), respectively. This preserves 2^{nd} order accuracy in time if both the advection and source steps are at least 2^{nd} order accurate. During the cooling source step $S()$ **PLUTO** solves the internal energy and chemical reaction network equations

$$\begin{cases} \frac{\partial(\rho e)}{\partial t} &= -\Lambda(n, T, X) \\ \frac{\partial X}{\partial t} &= S(X, T) \end{cases} \quad (8.2)$$

where $\Lambda(n, T, X)$ is a cooling (or heating) term, X is an array of fractional abundances (typically ion or molecule number fractions) and S is a reaction source term. The right-hand side of Equations (8.2) is implemented in the function `Src/Cooling/<COOLING>/radiat.c`. The user can select one among different cooling modules by enabling the `COOLING` macro during the python script:

- **TABULATED**: only the equation for the internal energy with a tabulated cooling function $\Lambda(T)$ is provided. No chemical network, see §8.1;
- **SNEq**: cooling function for atomic hydrogen, $X = \{X_{HI}\}$, including ionization, recombination and collisionally excited emission lines, §8.2;

Cooling modules are implemented inside the `Src/Cooling` directory and require the three dimensional units (constants) to be correctly initialized. Dimensional constants are essential to scale data values to cgs physical units as explained in §5.1.1.

Equations (8.2) can be integrated in time using either an explicit 3rd-order Runge-Kutta Fehlberg method (suitable for non-stiff conditions); or with a 4th-order semi-implicit Rosenbrock method with a 3rd order embedded error estimation (more accurate but also more expensive, suitable in stiff regions where explicit time marching may potentially become unstable). A selective approach that combines the two methods can also be specified (see below): in this case stiffness is detected when i) the hydro time step largely exceeds both the cooling and chemical time scales and ii) when the initial fraction is far from equilibrium.

Control variables and macros are introduced to tune crucial parameters such as the maximum allowed cooling rate in each time step or the cutoff temperature. They can be either members of the `RunConfig` structure or macros to be included in your definitions.hpp:

- `run.config.maxCoolingRate`: limit the time step so that the maximum fractional thermal losses cannot exceed this value. In general `run.config.maxCoolingRate` $\in [0, 1]$; the default is 0.1.
- `run.config.minCoolingTemp`: sets the cut-off temperature below which cooling is artificially set to 0.
- `COOLING.TIME.STEPPING`: controls the time stepping method to evolve the system of ODE given by Eq. (8.2). Possible values are `ROS34`, `RKF23` or `SELECTIVE`, see also §B.3
- `COOLING.ODE.TOL`: controls the tolerance used to compute the error in the above mentioned integrators.
- `ROS34_MAXSTEPS`: sets the maximum number of sub-steps that the semi-implicit integrator can take. Exceeding number will cause a `ERR.TOO_MANY_ITERATIONS`.

8.1 Tabulated Cooling

The tabulated cooling module provides a way to solve the internal energy equation

$$\Lambda = n^2 \tilde{\Lambda}(T), \quad \text{with} \quad n = \frac{\rho}{\mu m_u} \quad (8.3)$$

when the cooling/heating function $\tilde{\Lambda}(T)$ is not known analytically but rather is available as a table sampled at discrete (not necessarily equidistant) points, i.e., $\tilde{\Lambda}_j \equiv \tilde{\Lambda}(T_j)$. In order to use this module, the user must provide a two-column ascii files in the working directory named `cooltable.dat` of the form

```

.      .
.      .
.      .
T(j)   Lambda(j)
.      .
.      .
.      .

```

with the temperature expressed in Kelvin and the cooling/heating function $\tilde{\Lambda}$ in $\text{ergs}\cdot\text{cm}^3/\text{s}$. An example of such file¹ can be found in `Src/Cooling/Tab/cooltable.dat`. As usual, the dimensionalization is done automatically by the cooling module, once `run_config.unit_density`, `run_config.unit_length` and `run_config.unit_velocity` have been defined in `Init()`.

Alternatively, the *TABULATED* cooling module can be used to provide a user-defined cooling function,

$$\Lambda = \Lambda(V), \quad (8.4)$$

where V is a vector primitive variables. The explicit dependence of Λ can be given by i) copying `Src/Cooling/Tab/radiat.c` into your local working directory and ii) make the appropriate changes.

¹Generated with Cloudy 90.01 for an optically thin plasma and solar abundances, thanks to T. Plewa.

8.2 Simplified Non-Equilibrium Cooling: SNEq

This module is implemented in the Src/Cooling/SNEq directory and introduces a new variable, with index X_HI used to label the fraction of neutrals x_{HI} :

$$x_{HI} = \frac{n_{HI}}{n_H} . \quad (8.5)$$

where n_H is the total (neutral + ionized) number density of hydrogen:

$$n_H = \left(\frac{\rho_0}{m_u} \frac{B_H}{A_H} \right) \rho \quad (8.6)$$

where ρ_0 is the unit density (see §5.1.1), B_H (=H.MASS.FRAC) and A_H are the hydrogen mass fraction and the atomic weight of hydrogen, respectively.

The fraction of neutrals can be assigned by setting

```
v[X_HI] = 0.2; /* for example */
```

in your Init() function. The fraction of neutrals is treated as a passive scalar during the hydro step while it is governed by the following ODE during the cooling step:

$$\frac{\partial x_{HI}}{\partial t} = S = n_e \left[- (c_r + c_i) x_{HI} + c_r \right] \quad (8.7)$$

together with the energy equation

$$\frac{\partial(\rho e)}{\partial t} = -\Lambda = -n_e n_H \left(\sum_{k=1}^{k=16} j_k + w_{i/r} \right) \quad (8.8)$$

where $n_e = n_H(1 - x_{HI} + x_Z)$ is the electron number density while x_Z ($= 10^{-3}$ by default) is the fractional number density of metals. In Eq. (8.8) the summation over k accounts for 16 different line emissions coming from some of the most common elements, $k = \text{Ly } \alpha, \text{H } \alpha, \text{HeI (584+623), CI (9850 + 9823), CII (156}\mu\text{), CII (2325}\text{\AA}), \text{NI (5200 } \text{\AA}), \text{NII (6584 + 6548 } \text{\AA}), \text{OI (63}\mu\text{), OI (6300 + 6363 } \text{\AA}), \text{OII (3727), MgII (2800), SiII (35}\mu\text{), SII (6717 + 6727), FeII (25}\mu\text{), FeII (1.6}\mu\text{)}$.

The coefficient j_k in (8.8) has dimensions of erg/sec cm³ and is computed from

$$j_k = \frac{\hbar^2 \sqrt{2\pi}}{\sqrt{k_B m_e m_e}} f_k q_{12} \frac{h\nu_k}{1 + n_e (q_{21} / A_{21})}$$

where k is the index of a particular transition, $f_k = n_k / n_H$ is the abundance for that particular species. Here

$$q_{12} = \frac{8.6 \cdot 10^{-6}}{\sqrt{T}} \frac{\Omega_{12}}{g_1} \exp\left(-\frac{h\nu_k}{k_B T}\right) , \quad q_{21} = \frac{8.6 \cdot 10^{-6}}{\sqrt{T}} \frac{\Omega_{21}}{g_2}$$

where $\Omega_{12} = \Omega_{21}$ is the collision strength and is tabulated.

In Eq. (8.8) $w_{i/r}$ represents the thermal energy lost by ionization and recombination:

$$w_{i/r} = c_i \times 13.6 \times 1.6 \cdot 10^{-12} x_{HI} + c_r \times 0.67 \times 1.6 \cdot 10^{-12} (1 - x_{HI}) \frac{T}{11590}$$

where c_r and c_i are the hydrogen ionization and recombination rate coefficients:

$$c_r = \frac{2.6 \cdot 10^{-11}}{\sqrt{T}} ; \quad c_i = \frac{1.08 \cdot 10^{-8} \sqrt{T}}{(13.6)^2} \exp\left(-\frac{157890.0}{\sqrt{T}}\right) .$$

9. Additional Modules

9.1 The FARGO Module

The FARGO module permits larger time steps to be taken in those computations where a (grid-aligned) supersonic or super-fast dominant background orbital motion exists, see [MFS⁺12].

The algorithm decomposes the total velocity into an average azimuthal contribution and a residual term,

$$\mathbf{v} = \mathbf{w} + \mathbf{v}' \quad (9.1)$$

where $\mathbf{w}$ is a background velocity field that must be solenoidal and not depend on the coordinate corresponding to the transport direction and $\mathbf{v}'$ is called the residual velocity. The MHD or HD equations are solved in two steps: i) a linear transport operator corresponding to the velocity $\mathbf{w}$ in the direction of orbital motion and ii) a standard nonlinear solver applied to the original equations written in terms of the residual velocity $\mathbf{v}'$.

The Courant condition is then computed only from the residual velocity, leading to substantially larger time steps. In [MFS⁺12] it has been shown that if the characteristic velocity of fluctuations are comparable in magnitude than the expected gain in cylindrical coordinates is, roughly,

$$\frac{\Delta t_F}{\Delta t_s} \approx \frac{\max_{ijk} \left[\frac{1}{\Delta R} + \frac{M+1}{R\Delta\phi} + \frac{1}{\Delta z} \right]}{\max_{ijk} \left[\frac{1}{\Delta R} + \frac{1}{R\Delta\phi} + \frac{1}{\Delta z} \right]}, \quad (9.2)$$

where Δt_F and Δt_s are the FARGO time step and the standard time step, respectively, whereas $M = |\mathbf{w}|/\lambda'$ and $\lambda' = |\mathbf{v}'_d| + c_{f,d}$ is the characteristic speed in the $\hat{\mathbf{e}}_d$ direction.

The discretization is fully conservative in both angular momentum and total energy. The MHD module works only with the Constrained Transport (CT) method to control divergence-free condition.

9.1.1 Using the Module

The FARGO module is implemented in the directory `Src/Fargo/` and can be enabled by invoking the python script with the `--with-fargo` option. It works in Cartesian, cylindrical and spherical coordinates with all time stepping scheme.

The background velocity and the residual are initially assigned in `Init()` function, e.g.

```
#ifdef FARGO
v[FARGO_W] = -SB_Q*SB_OMEGA*x; /* Background velocity */
v[VX2]      = dvy;             /* Residual */
#else
v[VX2] = -SB_Q*SB_OMEGA*x + dvy; /* Standard assignment */
#endif
```

Throughout the computation, **gPLUTO** will always use the residual velocity and any user-defined boundary condition must also prescribe $\mathbf{v}'$ (that is, the residual). During the evolution, $\mathbf{v}'$ and $\mathbf{w}$ may be recomputed every fixed number of time steps set by the macro `FARGO_NSTEP_AVERAGE` (default is 10). This operation is done by first averaging the total azimuthal velocity v_y or v_ϕ along the corresponding orbital direction and then updating the residual. However, if `FARGO_NSTEP_AVERAGE` is set to `-1`, **gPLUTO** will compute the background speed only at the beginning.

The linear transport step is implemented by the function `FARGO.ShiftSolution()` and the order of reconstruction used during this step is set by the constant `FARGO_ORDER` which, by default, is 3 (third-order PPM).

Output files are written using the total velocity (FARGO_OUTPUT_VTOT set to *YES*) or the residual (FARGO_OUTPUT_VTOT set to *NO*, default)¹. In addition, to ensure that restart operations are done correctly (only with double precision binary files and when FARGO_NSTEP_AVERAGE is set to a positive value), the orbital speed is also written to disk (both with active and ghost zones) with the same frequency used by .dbl datafiles. Datafiles are named as wFargo.nnnn.dbl where nnnn is a four-digit padded integer following the same rules used by standard binary output.

The default value of the various switches can be changed by adding the corresponding pre-processor `#define` directive to your definitions.hpp, see Appendix §B.3.

The FARGO module is typically used to model supersonic accretion disks and test problems can be found in the directory `Test.Problems/HD/Disk.Planet/` (configurations #2, #4 and #6) as well as in `Test.Problems/MHD/FARGO/Spherical.Disk/`. For more information see the test problem documentation at `Doc/test_problems.html`.

9.1.2 A Note on Parallelization

The FARGO-MHD algorithm is fully parallelized in all coordinate directions with the requirement that the number of zones per processor in the orbital direction must be larger than the expected transport shift denoted with m .

With a large number of processors ($\gtrsim 2048$), the resulting auto-decomposition mode may result in sub-domains that violate this condition and an error message is issued. To avoid this problem you can specify the parallel decomposition manually using the `-dec n1 [n2] [n3]` command line argument (§1.4.2) and ensure that not too many processors are used along the ϕ direction. As an example, suppose you wish to use 4096 processors but only 8 along the orbital direction (x_2). You may specify the domain decomposition by giving, say, 32, 8 and 16 in the three directions with

```
mpirun -np 4096 ./pluto -dec 32 8 16
```

¹Saving the total velocity may result in slightly different results when restarting, due to arithmetic roundoff in the subtraction step following restart.

9.2 The RADIATION Module

The RADIATION module couples the conservation laws of relativistic/nonrelativistic HD/MHD to a system of frequency- and angle-averaged radiation transport equations for the radiation energy density, E_r , and the three components of the radiation flux, F_r . We normalize the latter by c in such a way that all radiation fields are measured in energy density units. Assumptions and implementation details are summarized in [MM19] (relativistic version) and [MKFM21] (nonrelativistic version).

In the nonrelativistic modules (HD/MHD), and ignoring extra source terms for clarity (e.g., viscosity), the resulting system of equations takes the form

$$\begin{aligned}
 \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) &= 0 \\
 \frac{\partial \mathbf{m}}{\partial t} + \nabla \cdot \left[\mathbf{m} \mathbf{v} - \mathbf{B} \mathbf{B} + \mathbf{I} \left(p + \frac{B^2}{2} \right) \right]^T &= -\rho \nabla \Phi + \rho \mathbf{g} + \mathbf{G} \\
 \frac{\partial \mathbf{B}}{\partial t} + \nabla \times (c \mathbf{E}) &= 0 \\
 \frac{\partial (E_t + \rho \Phi)}{\partial t} + \nabla \cdot \left[\left(\frac{\rho v^2}{2} + \rho e + p + \rho \Phi \right) \mathbf{v} + c \mathbf{E} \times \mathbf{B} \right] &= \mathbf{m} \cdot \mathbf{g} + c G^0 - \nabla \cdot \mathbf{F}_{\text{Irr}} \\
 \frac{1}{\hat{c}} \frac{\partial E_r}{\partial t} + \nabla \cdot \mathbf{F}_r &= -G^0 \\
 \frac{1}{\hat{c}} \frac{\partial \mathbf{F}_r}{\partial t} + \nabla \cdot \mathbb{P}_r &= -\mathbf{G}
 \end{aligned} \tag{9.3}$$

(see Eq. (6.3)), while in the relativistic modules (RHD/RMHD), the resulting system is

$$\frac{\partial}{\partial t} \begin{pmatrix} D \\ \mathbf{m} \\ E_t \\ \mathbf{B} \\ E_r \\ \mathbf{F}_r \end{pmatrix} + \nabla \cdot \begin{pmatrix} D \mathbf{v} \\ w_t \gamma^2 \mathbf{v} \mathbf{v} - \mathbf{b} \mathbf{b} + \mathbf{I} p_t \\ \mathbf{m} \\ \mathbf{v} \mathbf{B} - \mathbf{B} \mathbf{v} \\ \mathbf{F}_r \\ \mathbb{P}_r \end{pmatrix}^T = \begin{pmatrix} 0 \\ \mathbf{G} \\ G^0 - \nabla \cdot \mathbf{F}_{\text{Irr}} \\ 0 \\ -G^0 \\ -\mathbf{G} \end{pmatrix} \tag{9.4}$$

(see Eq. (6.12)). In all cases, the components of the radiation pressure tensor, $\mathbb{P}_r$, are defined as a function of $(E_r, \mathbf{F}_r)$ via the M1 closure by [Lev84] as

$$P^{ij} = E_r \left(\frac{1 - \xi}{2} \delta^{ij} + \frac{3\xi - 1}{2} n^i n^j \right), \quad \xi = \frac{3 + 4f^2}{5 + 2\sqrt{4 - 3f^2}}, \tag{9.5}$$

where $\mathbf{n} = \mathbf{F}_r / |\mathbf{F}_r|$, $f = |\mathbf{F}_r| / E_r$, and δ^{ij} is the Kronecker delta. The radiation-matter interaction terms ($G^0, \mathbf{G}$) are defined by boosting into the Eulerian frame their comoving values, given by

$$(G^0, \mathbf{G})_{\text{comov}} = \rho \left[\kappa (E_r - a_R T^4), (\kappa + \sigma) \mathbf{F}_r \right]_{\text{comov}}. \tag{9.6}$$

In this equation, κ and σ are the frequency-averaged absorption and scattering coefficients, respectively, $a_R = 4\sigma_{\text{SB}}/c$ is the radiation constant, and T is the fluid's temperature.

The reduced speed of light, $\hat{c}$, is introduced in the nonrelativistic equations to reduce the computational cost of explicitly integrating the radiation transport terms. Additional speedup is achieved by splitting the integration of the radiation and MHD fields, and using substepping to evolve the radiation subsystem. The user can minimize the computational cost by setting $\hat{c}/c$ to the smallest value that does not affect the resulting dynamics (see [MKFM21]).

Heating by external (e.g., stellar) sources of irradiation not included in the $(E_r, \mathbf{F}_r)$ radiation field can be added as a user-defined source term depending on an external irradiation flux, F_{Irr} (see §9.2.1).

The gas energy is computed by applying the *IDEAL* EOS in the nonrelativistic modules and either the *TAUB* or *IDEAL* EOS in the relativistic modules. If magnetic fields are included, the divergence-free condition can be enforced by applying either Hyperbolic Divergence Cleaning or Constrained Transport (see §6.2.2.2 and §6.2.2.3).

9.2.1 Using the module

The RADIATION module can be switched on by setting the RADIATION switch to *YES* in the HD, MHD, RHD, or RMHD sub-menus. The module is implemented in `Src/Radiation/` and can be used in Cartesian, cylindrical and spherical coordinates. The nonrelativistic version is compatible with the `ROTATING_FRAME` option and usual nonideal options, such as `VISCOSITY`, which are applied during the HD/MHD step.

The radiation fields are initialized in `Init()` or `InitDomain()` using the indices `ENR`, `FR1`, `FR2`, and `FR3` (see Table 9.1). Although these indices can be used in any coordinate system, an alternative set can be used in curvilinear coordinates to avoid confusion (see columns 2 and 3 in Table 9.1). Boundary conditions for (E_r, F_r) can be defined either in `pluto.ini`, in which case they coincide with those assigned to (ρ, v) , or in `UserDefBoundary()` if the boundary is set as *userdef*.

Index	Cylindrical	Spherical	Quantity
ENR	-	-	radiation energy density
FR1	iFRR	iFRR	x_1 -radiation flux
FR2	iFRPHI	iFRTH	x_2 -radiation flux
FR3	iFRZ	iFRPHI	x_3 -radiation flux

Table 9.1: Additional array indices used for labeling primitive variables in the RADIATION module.

Three possible Riemann solvers can be selected for the explicit integration of radiation fluxes. The solver for the radiation subsystem is defined independently from the HD/MHD solver by setting the `RadSolver` tag in the `[Solver]` block of `pluto.ini`, as described in §4.4. Available values are *hllc*, *hll* and *tvd1f*. All reconstruction methods available in **gPLUTO** can be used. Additional integration options can be found in Table B.1.

In the nonrelativistic module, the radiation subsystem is advanced separately from the HD/MHD subsystem using operator splitting and a smaller time step. The initial radiation time step, CFL number, and allowed step growth can be set in the `[Radiation Parameters]` block of `pluto.ini`; see §4.3.

External irradiation can be switched on by defining `IRRADIATION` as *YES* in `definitions.hpp`. The divergence of the irradiation flux, $\nabla \cdot F_{\text{Irr}}$, must then be computed in a local file `irradiation.cpp`, stored in both primitive and conserved fields using the index `FIR`. A template for this file is provided in `Src/Radiation/irradiation.cpp` (see also the Disk-Planet test in `Test.Problems/Radiation/Nonrelativistic/HD_Disk_Planet/`). The term $\nabla \cdot F_{\text{Irr}}$ is only computed at startup if `IRRADIATION_UPDATE` is defined as *NO* in `definitions.hpp`; otherwise, it is computed at each hydro step by default.

9.2.1.1 Constants and units

The RADIATION module constants are stored in the `RunConfig` structure and should be assigned in `init.cpp` through the `RunConfig &run_config` argument of `Init()`. These members replace the global radiation constants used in older versions. The following members are used by both the relativistic and nonrelativistic versions:

- `run_config.rad_constant` (double, default: 1): the radiation constant, $a_R = 4\sigma_{\text{SB}}/c$.
- `run_config.rad_ideal_gas_constant` (double, default: 1): the ideal gas constant, c_I , determining the gas temperature in code units as $T = c_I p/\rho$.

In the nonrelativistic version, the following constants must also be specified:

- `run_config.rad_c` (double, default: 1): the speed of light, c .
- `run_config.rad_reduced_c` (double, default: 1): the reduced speed of light, $\hat{c}$.

If constant opacities are used (see §9.2.1.2), they must be defined as follows:

- `run_config.rad_absorption_coeff` (double, default: 0): the absorption opacity, κ .
- `run_config.rad_scattering_coeff` (double, default: 0): the scattering opacity, σ .

For example:

```
void Init (double *v, double x1, double x2, double x3,
          RunConfig &run_config)
{
    const double *inputParam = run_config.inputParam;

    #if RADIATION
        run_config.rad_absorption_coeff = inputParam[COEF_ABSORPTION];
        run_config.rad_scattering_coeff = inputParam[COEF_SCATTERING];
        run_config.rad_constant        = inputParam[CONST_RAD];
        run_config.rad_ideal_gas_constant = inputParam[CONST_IDEALGAS];
    #if RADIATION_NR
        run_config.rad_c = inputParam[RAD_C];
        run_config.rad_reduced_c = inputParam[REDUCED_C];
    #endif
    #endif
}
```

where RADIATION_NR is automatically set to YES when the nonrelativistic radiation module is enabled, and to NO otherwise.

All fields and constants must be defined by the user as nondimensional quantities, according to the scales introduced by the source terms. Using the notation of §5.1.1, CGS and nondimensional code quantities can be related as follows:

$$E_r = \frac{E_{r,cgs}}{\rho_0 v_0^2} \quad , \quad F_r = \frac{F_{r,cgs}/c}{\rho_0 v_0^2} \quad , \quad \kappa = \kappa_{cgs} \rho_0 L_0 \quad , \quad a_R = \frac{a_{R,cgs}}{\rho_0 v_0^2 / T_0^4} \quad , \quad c_I = \frac{c_{I,cgs}}{T_0 / v_0^2} \quad , \quad c = \frac{c_{cgs}}{v_0} \quad , \quad (9.7)$$

where T_0 is the chosen temperature unit. If $T_0 = 1$ K, c_I can be defined from the temperature conversion factor Kelvin() by specifying the molecular weight μ and setting `rad_ideal_gas_constant = mu*Kelvin(run_config)`.

9.2.1.2 Opacities

The opacity coefficients κ and σ in Eq. (9.6) can be defined either as constants (see §9.2.1.1) or as user-defined functions of the primitive fields. The latter option is selected by defining RADIATION_VAR_OPACITIES as YES in definitions.hpp and defining in init.cpp a function UserDefOpacities() of the form

```
void UserDefOpacities(double *v, double *abs, double *scat,
                     RunConfig &run_config) ,
```

where $*v$ is a pointer to a vector of primitive variables, while $*abs$ and $*scat$ are pointers used to store the resulting absorption and scattering coefficients. As an example, Kramers' opacity law $\kappa \propto \rho T^{-7/2}$ with $\sigma = 0$ can be defined as follows:

```
#if RADIATION_VAR_OPACITIES == YES
#pragma omp declare target
#pragma acc routine seq
void UserDefOpacities (double *v, double *abs, double *scat,
                      RunConfig &run_config)
{
    double T;

    T = GetTemperature(v[RHO], v[PRS], run_config);

    *scat = 0.0;
    *abs = v[RHO]*pow(T, -3.5);
}
#pragma omp end declare target
#endif
```

In this example, the gas temperature is computed as $T = c_I p / \rho$ by means of the built-in function GetTemperature():

```
double GetTemperature(double rho, double prs, RunConfig &run_config) .
```

Another built-in function, particularly useful to define initial conditions in local thermal equilibrium, is Blackbody():

```
double Blackbody(double temperature, const RunConfig &run_config) ,
```

which returns the value of $a_R T^4$ taking as input the value of T .

Note: To achieve good accuracy in both the optically thick and optically thin limits, the terms between brackets in Equation (9.6) are commonly defined as $[\kappa_P(E_r - a_R T^4), \chi_R F_r]$, where κ_P is the Planck-averaged absorption opacity and χ_R is the Rosseland-averaged total (absorption+scattering) opacity. To do this, κ must be defined as κ_P , and σ must be defined as $\chi_R - \kappa_P$.

9.2.1.3 A note on implicit solvers

In the RADIATION module, Implicit-Explicit (IMEX) schemes are used to integrate the evolution equations in such a way that fluxes are integrated explicitly, while the potentially stiff radiation-matter interaction terms are integrated implicitly. As summarized in Appendix B.3, six different implicit solvers are implemented, four of which are applicable in both the relativistic and nonrelativistic modules and two of which (RADIATION_NEWTON_NR_RAD and RADIATION_NEWTON_NR_GAS) can only be selected in the nonrelativistic case. The implicit solver can be selected in definitions.hpp by setting the variable RADIATION_IMPL to one of the values listed and described in Appendix B.3.

All of the included implicit solvers are based on iterations of either the radiation fields, (E_r, F_r) or the matter fields. Relativistic gas-based solvers iterate $(p, \gamma v)$, while the nonrelativistic gas-based solver RADIATION_NEWTON_NR_GAS iterates (p, v) . **For radiation-dominated problems ($E_r \gg E_{\text{gas}}$), we recommend using one of the methods based on iterations of the gas fields. Conversely, for matter-dominated problems ($E_r \ll E_{\text{gas}}$), we recommend using methods based on iterations of the radiation fields.** This choice ensures good convergence of both the radiation and matter fields; otherwise, small relative errors in the dominant fields can lead to large relative errors in the subdominant fields.

For efficiency, we recommend using either RADIATION_NEWTON_NR_RAD or RADIATION_NEWTON_NR_GAS when using the nonrelativistic modules.

Additional RunConfig members are:

- `run_config.rad_max_iter` (default: 100), the maximum number of iterations for the implicit solver.
- `run_config.rad_tolerance` (default: $1e-7$), the convergence tolerance .
- `run_config.small_enr` (default: $1e-16$), the value assigned to E_r if it becomes negative.

10. Particles

gPLUTO comes with a fully parallel module supporting different kinds of particles, enabling gas-particle hybrid numerical schemes. A specific particle type is selected directly from the Python script (§2.1.10) by setting `PARTICLES` to

- `PARTICLES_CR` (*under development*):
- `PARTICLES_LP`: Introduces Lagrangian, tracer-like particles embedding sub-grid electron physics [VMB⁺18]. Specifically designed to address the problem of non-thermal emission from highly energetic particles (typically electrons) embedded in a large-scale classical or relativistic MHD flow, §10.2.

10.0.1 Particle Storage and Memory Management

Lagrangian particles in **gPLUTO** are managed through the `ParticlesContainer` class, which replaces the legacy particle structure with a more flexible and performance-oriented design.

Structure-of-arrays layout. The current implementation adopts a *structure-of-arrays* (SoA) approach. Each physical quantity is stored in a dedicated container, and a particle is identified by a common index i across all arrays. Particle attributes are therefore accessed as `field(i)` rather than through a pointer to a structure. The full list of available particle fields is summarized in Table 10.1.

Blocked storage. Each field is implemented through a `BlockedContainer<T>`, which stores data in fixed-size memory chunks of length $L = 2^e$. This allows the container to grow or shrink dynamically without incurring the overhead of reallocating large contiguous memory blocks. All field containers are kept synchronized so that they always contain the same number of particles.

Dynamic particle management. The number of particles per processor may change during runtime due to injection, boundary removal, or inter-process communication. Memory is managed through coordinated operations acting on all fields simultaneously:

- `Reserve(n)` increases the container capacity;
- `Shrink(n)` removes trailing particles and releases memory.

Since particles are not stored in any particular order, removal is handled through a compaction procedure in which particles marked for deletion (“holes”) are replaced by valid particles from the end of the container (“fills”). This ensures that active particles remain densely packed in memory.

MPI exchange. When particles cross subdomain boundaries, they are packed into communication buffers and exchanged between MPI ranks using non-blocking operations. Received particles are inserted into available slots in the local container, followed by a compaction step to restore memory continuity.

Design considerations. This design provides:

- efficient memory access on GPUs through coalesced reads/writes;
- reduced overhead from dynamic allocation and deallocation;

- scalability to large particle counts with frequent insertion/removal;
- a unified abstraction compatible with both CPU and GPU execution.

Field	Particle	Description
<code>pos(i)</code>	All	A 3-element array (double precision) giving the particle coordinates. Stored in the <code>pos_</code> container. Supplied by the user.
<code>speed(i)</code>	All	A 3-element array (double precision) giving the particle velocity. For the CR module, this represents the particle <u>four-velocity</u> . Stored in the <code>speed_</code> container.
<code>posPrv(i)</code>	All	Previous particle position, used by time integration schemes. Stored in the <code>posPrv_</code> container.
<code>cell(i)</code>	All	Integer array giving the index of the grid cell hosting the particle. Updated during interpolation routines. Stored in the <code>cell_</code> container.
<code>id(i)</code>	All	Unique integer identifier assigned at particle creation. Stored in the <code>id_</code> container.
<code>tinj(i)</code>	All	Particle injection (creation) time. Set by the code. Stored in the <code>tinj_</code> container.
<code>color(i)</code>	All	Passive scalar used to tag or classify particles. Supplied by the user. Stored in the <code>color_</code> container.
<code>srFlag(i)</code>	All	Flag used for particle status and boundary handling (e.g. send/receive). Stored in the <code>srFlag_</code> container.
<code>nmicro(i)</code>	LP	Number density of micro-particles associated with the LP. Stored in the <code>nmicro_</code> container.
<code>density(i)</code>	LP	Local fluid density sampled by the particle. Stored in the <code>density_</code> container.
<code>pressure(i)</code>	LP	Local fluid pressure at the particle position. Stored in the <code>pressure_</code> container.
<code>lorG(i)</code>	LP	Fluid Lorentz factor at the particle location. Stored in the <code>lorG_</code> container.
<code>mag(i)</code>	LP	Magnetic field magnitude (or related quantity) sampled by the particle. Stored in the <code>mag_</code> container.
<code>cr(i)</code>	LP	Coefficient controlling radiative losses (synchrotron/IC). Stored in the <code>cr_</code> container.
<code>chi(i)</code>	LP	Array representing the particle energy distribution (number of micro-particles per unit volume per energy bin). Size set by <code>PARTICLES_LP_NEbins</code> . Stored in the <code>chi_</code> container.
<code>eng(i)</code>	LP	Array of energy bin values corresponding to <code>chi(i)</code> . Stored in the <code>eng_</code> container.
<code>cmp_ratio(i)</code>	LP+Shock	Shock compression ratio experienced by the particle. Stored in the <code>cmp_ratio_</code> container (if enabled).
<code>shk_gradp(i)</code>	LP+Shock	Pressure gradient during shock crossing. Stored in the <code>shk_gradp_</code> container.
<code>shkflag(i)</code>	LP+Shock	Flag indicating whether the particle is currently inside a shock. Stored in the <code>flag_</code> or shock-related containers.

Table 10.1: Particle container members in the current gPLUTO implementation. Each particle is identified by an index i , and its properties are accessed through structure-of-arrays containers. Labels “All”, “LP”, and “LP+Shock” indicate availability depending on the selected particle model.

10.1 Initial, Boundary and Injection conditions

10.1.1 Initial Conditions

Particle initialization and injection are performed through the function `Particles.Init()`, defined in `particles.init.cpp`. This function allows the user to specify the spatial distribution and particle properties at the time of initialization ($t = 0$) or during subsequent injection stages ($t > 0$).

In the current implementation, particles are initialized according to a *cell-based sampling strategy* - equivalent to a particle number density particle / cell) - controlled by the ratio

$$\frac{n}{m},$$

where n is the number of particles assigned to each block, while m is the number of grid cells forming a block. This formulation generalizes the traditional cell-by-cell initialization and allows both dense ($n > 1$) and sparse ($m > 1$) particle sampling. In your `pluto.ini` you may specify:

- 1/1: one particle per cell;
- 4/1: four particles per cell;
- 1/8: one particle every 8 cells.

In practice, when $m > 1$, the computational grid is effectively coarsened by a factor m for the purpose of particle injection, and particles are assigned per block rather than per individual cell.

The `Particles_Init()` function follows these steps:

- Define the (possibly coarsened) grid used for particle assignment;
- At the first call only, initialize the array `pc->insert(i,j,k)` specifying which cells host particles;
- Count the number of particles to be created and reserve memory using `pc->Reserve()`;
- Loop over the computational domain and distribute particles in selected cells using either a uniform or random distribution, while also initializing particle attributes (e.g., color, spectra, id).

The array `pc->insert(i,j,k)` is used to enable or disable particle creation in individual computational cells. By default, all cells in the domain are selected. Subsequent calls to `Particles_Init()` may redefine this array so that particles are created only in specific regions. This is particularly useful for modeling localized particle injection at later times.

The `BOX_LOOP()` macro iterates over the active domain and places particles in the selected cells using either `Particles_LoadUniform()`, which distributes particles evenly within each cell (or block), or `Particles_LoadRandom()`, which samples positions from an arbitrary probability distribution using an acceptance-rejection method.

Particles identifiers are assigned as:

```
pc->id(indexLoc) = progressive_cell + np + pc->idCounter;
```

where `np` is the local particle index within a cell (or block), `progressive_cell` is a cell-dependent offset, and `pc->idCounter` is a global counter tracking previously created particles. This construction guarantees globally unique and monotonically increasing particle identifiers, also across multiple initialization or injection stages.

10.1.2 Boundary Conditions

At present, boundary conditions used for the fluid also apply to particles. If a *userdef* value is given in the standard boundary block (§4.5), particles may be created or destroyed at the specified boundary and the user must code his/her boundary condition using the function `Particles_UserDefBoundary()` located in the file `particles_init.c`. By default, this function is called by **PLUTO** in the case of an *outflow* boundary which requires particles to be deleted once they cross an *outflow* boundary.

10.1.3 Injection

The function `Particles_Inject()` is used to create (inject) new particles inside a particular area of the computational domain at specific times during your simulation. Injection requires the user to specify the field members used during the initialization process.

In the example below (taken from `Test.Problems/LP/Jet2D`), injection takes place every Δt_{inj} (which is assigned through a user-defined parameter `inputParam[INJECT_FREQ]`) but only in a specific region of the domain ($|y| < r_j$, $x > 0.006$, $x < 0.01$):

```

void Particles_Inject(Data *data, Grid *grid)
{
    int i,j,k;
    ParticlesContainer *pc = data->pc;
    RunConfig &run_config = *(data->run_config);
    RBox box;

    RBoxDefine(0,grid->ntot[IDIR]-1,
               0,grid->ntot[JDIR]-1,
               0,grid->ntot[KDIR]-1, &box);

    double dtinj = run_config.inputParam[INJECTION_DT];
    double t = data->Dts->time;
    double tnext = data->Dts->time + data->Dts->dt;
    int time_to_inject = int(t/dtinj) != int(tnext/dtinj);
    if (time_to_inject){
        BOX_LOOP(box, i, j, k){
            double x1 = grid->xC[IDIR][i];
            double x2 = grid->xC[JDIR][j];
            double x3 = grid->xC[KDIR][k];
            double rjc = 0.01;

            pc->insert(i,j,k) = false;
            if ( (sqrt(x2*x2) < rjc) && (x1 >= 0.006) && (x1 <= 0.01) ){
                pc->insert(i,j,k) = true;
            }
        }
        Particles_Init(data, grid);
    }
}

```

Note that this function defines the `pc->insert()` array in order to reuse the `Particles_Init()` function to perform injection. Beware that **gPLUTO** calls this function at every time step and care must be taken in order to avoid overpopulating the domain with unwanted particles.

10.2 Lagrangian Particles

Note: This module is not part of the public code release, see “Terms & Conditions of Use” at the beginning of this guide

The Lagrangian particle (LP) module is enabled directly from the Python menu by setting **PARTICLES** to **PARTICLES_LP**. LPs follow the fluid streamlines such that its position coordinates x_p are evolved in time using the following equation -

$$\frac{dx_p}{dt} = v_p \equiv v_{\text{fluid}}(x_p) \quad (10.1)$$

where, the particle velocity v_p is obtained by interpolating the fluid velocity defined at the center of each cell to the particle’s position. LP can be used as simple tracer particles or to model sub-grid physical processes although *excluding* back-reaction on the fluid.

The main motivation behind developing a Lagrangian tracer particle module is to incorporate sub-grid physics that depends on the local dynamical property of the fluid. In particular, we have utilised the LP module to develop a novel framework incorporating sub-grid physics of particle acceleration and physical processes responsible for non-thermal emission from AGN jets and clusters. The brief description of using the module is given below and technical and numerical details in [VMB⁺18] and its application in continuum SED modelling for kilo-parsec AGN jets is demonstrated in [BAV⁺20].

10.2.1 Radiative Losses - LP Model

The main objective of the hybrid framework developed involving macro-particles is to model non-thermal emission from astrophysical flows. Each LP in this framework is regarded as *macro-particle* with a spectral distribution $\chi(E) = N(E)/n_0$, where $N(E)$ is the number density of real particles (leptons or hadrons) as a function of energy E and n_0 is the number density of the fluid interpolated at the grid position of the particle.

To enable the radiative losses module, the user has to modify the `definitions.h` in the user-defined constant section (§ 2.3) as


```

/* [Beg] user-defined constants (do not change this line) */
#define PARTICLES_LP_SPECTRA YES
/* [End] user-defined constants (do not change this line) */

```

If the previous flag is enabled, other structure members (which we list in Table ??) are included in the definition.

The non-dimensional spectral distribution defined for each *macro-particle* is updated in time by solving the relativistic cosmic ray transport equation based on local fluid conditions. The evolution takes into account radiative losses from adiabatic expansion, synchrotron cooling and losses due to Inverse Compton processes via CMB photons for a given redshift that is specified by `PARTICLES_LP_ICCMBZ`, whose value is 0.0 by default. Its value can be modified depending upon the problem by re-defining its value in `definitions.h` under the *user-defined constants* (see below).

In addition, the spectral distribution of macro-particles experiencing shocks is updated by considering the process of diffusive shock acceleration (DSA) for both quasi-parallel and quasi-perpendicular shocks, see [VMB⁺18]. There are two modes of spectral updates after the macro-particle crosses the shock viz.,

- **Non-Convolved** : A brute-force approach whereby the particle's spectra is reset to a power-law with the spectral power determined based on the shock compression ratio and the angle between the shock normal and the magnetic field direction. The lower and upper energy bounds of the power-law spectra or obtained as described in [VMB⁺18].
- **Convolved** : In this approach, the spectral update of the *macro-particle* considers history of the particle spectra and uses the update as prescribed by [MZM⁺99] and Mukherjee et. al. (in prep) accounting for multiple particles within a grid-cell to normalise the spectra.

The user can choose the spectra update post shock by setting the variable `PARTICLES_LP_CONV_SPECTRA`, which by default is set to YES for adopting the convolved approach.

10.2.2 Spectral Distribution Initialisation

The function `Particles_LP_InitSpectra()` should be used to prescribe the initial spectral profile for each macro-particle. For example, to prescribe an initial power-law spectra, the user needs to specify four input parameters as demonstrated in the code snippet below. These input parameters are the upper and lower bounds of the energy array, the power of the spectra, and the variable `pl->nmicro` that represents total integrated number density of initial non-thermal particles. This is a non-dimensional number, its physical value in cm^{-3} can be obtained using `pl->nmicro*(UNIT_DENSITY/CONST_amu)`.

The energy of these particles E is an array of equally spaced bins in logarithmic scale from $E_{\min}$ to $E_{\max}$. The energy bounds are to be specified in units of `PARTICLES_LP_SPEC_ENERGY`, whose value by default is set to be `CONST_me*CONST_c*CONST_c`. The user can modify this value by re-defining it in `definitions.h`. Additionally, the user also has a control in choosing the number of bins to resolve the energy array. This is specified by the variable `PARTICLES_LP_NEbins` whose value by default is set to 100.

The following piece of code, for instance, can be used to initialise all macro-particle distribution to a power-law spectra with $s = 9.0$, $E_{\min} = 100$, $E_{\max} = 10^8$ and `pl->nmicro =  $10^{-12}\text{cm}^{-3}$` :

```

/* ***** */
void Particles_LP_InitSpectra(Particle* pl)
/*!
 * Initialise spectra for each particle (only for LAGRANGIAN).
 * Specify here the initial distribution of N(E) with E for each particle
 *
 * \param [in] pl Pointer to the Particle structure.
 *
 * ***** */
{
    int i;
    double Emin, Emax, N_0, s;
    double lnEmin, lnEmax, dlnE, scrh;
    Emin = 1.0e2; /* In code units : For physical value multiply by PARTICLES_LP_SPEC_ENERGY */
    Emax = 1.0e8; /* In code units : For physical value multiply by PARTICLES_LP_SPEC_ENERGY */
    lnEmin = log10(Emin);
    lnEmax = log10(Emax);
    dlnE = (lnEmax - lnEmin)/((double)PARTICLES_LP_NEbins-1);

```

```

/* Value of N_tot in non-dimensional units : (nmicro_cm^-3)*(CONST_amu)/UNIT_DENSITY */
pl->nmicro = 1.0e-12*(CONST_amu/UNIT_DENSITY);

/* -----
0. Initialize structure members
----- */

pl->cmp_ratio   = 1.0;
pl->shkflag     = 0;
pl->Vshk_upst[RH0] = -1.0;
pl->Vshk_dnst[RH0] = -1.0;
pl->cr         = 0.0;
pl->shk_gradp   = -1.0;

/* -----
1. Initialize spectral distribution to power law.
   Chi in each bin is assigned as an average over the energy interval.
   chi_i = \int^{Eh}_{El} E^{-p} dE / (Eh - El)
----- */

alpha1 = 9.0;
scrhchi = (pl->nmicro)/(pow(Emax,1.0-alpha1)-pow(Emin,1.0-alpha1));

pl->eng[0] = Emin;
for (i = 0; i < PARTICLES_LP_NEbins; i++){
    scrhln = lnEmin + (i+1)*dlnE;
    pl->eng[i+1] = pow(10.0, scrhln);
    pl->chi[i] = scrhchi*( pow(pl->eng[i+1],1.0-alpha1)
                        -pow(pl->eng[i],1.0-alpha1));
    pl->chi[i] /= pl->eng[i+1] - pl->eng[i];
}
}

```

Few parameters are defined to provide finer control on the evolution of macro-particles and the associated non-thermal emission from them. These user defined constants are listed below in Appendix B.3.

As the spectral is evolved at every advection time for each macro-particle, we can use this information to compute the Inverse Compton and synchrotron emissivity along with polarisation (Stokes Parameters) from a single macro-particle at every instant. It is possible to output these emissivities using the `userdef_output.c`. For example, to output the synchrotron emissivity along with Stoke Q and Stoke U polarisation parameters for an observed frequency of 5 GHz and a given line of sight of 20° with respect to the z -axis along the $x - z$ plane (by default the y axis in the fluid frame and observer's frame are parallel), we have -

```

#if PARTICLES
void Emiss_Deposit(Particle *p, double *qd);
#endif

/* ***** */
void ComputeUserVar (const Data *d, Grid *grid)
/*
 *
 * PURPOSE
 *
 *   Define user-defined output variables
 *
 *
 * ***** */
{
    #if PARTICLES
        #if PARTICLES_LP_SPECTRUM == YES
            double ***Dq[3];
            Dq[0] = GetUserVar("J5GHz");
            Dq[1] = GetUserVar("Q5GHz");
            Dq[2] = GetUserVar("U5GHz");
            Particles_Deposit(d->PHead, Emiss_Deposit, Dq, 3, grid);
        #endif
    #endif
}

#if PARTICLES_LP_SPECTRA == YES
/* ***** */
void Emiss_Deposit(Particle *p, double *pr)
/*
 * The helper function for Particles_Deposit to deposit
 * emissivity density.
 *
 * ***** */
{
    #if PHYSICS == MHD || PHYSICS == RMHD

```

```
int i;
double thetaobs = 20.0; /* Angle of incidence in degree i.e., line of sight angle. */
double JnVals, StQVals, StUVals;
double frq = 5.0; /* in GHz */
Particles_LP_Sync_Emissivity(p, frq, thetaobs, &JnVals, &StQVals, &StUVals);
pr[0] = JnVals;
pr[1] = StQVals;
pr[2] = StUVals;
#endif
}
#endif
```

10.3 Particles Output and Visualization

Similarly to the fluid section (§11.1), particles output datafiles are named as particles.nnnn.ext where nnnn is a four-digit zero-padded integer counting the output number and ext is the corresponding file extension. At present, particles datafiles can be written using single, double-precision format (.flt or .dbl) and VTK format. Particles output is controlled by the [Particles] block in pluto.ini, see §4.7. The above name convention that is set as default is followed for particle entities *PARTICLES_DUST* and *PARTICLES_CR*. The particle entities *PARTICLES_LP* have been formulated to follow a different name convention so that the maximum file size of data can be controlled. For particles data with *PARTICLES_LP*, the data files are named as particles.nnnn_chxx.ext where nnnn is the four digit zero-padded integer counting the output number, xx is the two digit chunk number by default its value is 00 and ext is the corresponding file extension. In cases with large number of *macro-particles*, the filesize of particle datafiles can be huge and sometimes difficult to analyse and process. Thus, we have set a file size limit of 2GB and using this threshold we sub-divide the particle datafiles into several *chunks* and are denoted by chxx.

Float and double precision datafiles begin with an ASCII header file followed by raw binary data. The header file consists of lines beginning with a # character providing important information which can be used to read the file (for visualization purposes or for restart). For instance, the number of particles is specified by nparticles while the number and names of the particle structure fields that is written to disk is specified by nfields and field_names, respectively. Raw binary data follows immediately after the header section and contains a sequence of nparticles blocks each containing the selected structure fields. In the example below, we show the header file of a .flt file where only particle positions and velocities are being written:

```
# PLUTO 4.4 binary particle data file
# dimensions      2
# nflux           9
# dt_particles    1.000000e-05
# endianness      little
# nparticles      18432
# idCounter       0
# particletype    3
# precision       single
# time            0.000000e+00
# stepNumber      0
# nfields         6
# field_names     x1  x2  x3  vx1  vx2  vx3
<x1, x2, x3, vx1, vx2, vx3>_1
<x1, x2, x3, vx1, vx2, vx3>_2
<x1, x2, x3, vx1, vx2, vx3>_3
...
<x1, x2, x3, vx1, vx2, vx3>_nparticles
```

For Lagrangian Particle (LP) module, the particles are divided in more than one file, if the file size exceeds 1.9 GB. The particle files are named as: particles.0001_ch00.flt, particles.0001_ch01.flt and so on. Each such file is referred to as chunks. The total number of chunks for a given output step, the number of particles in a given chunk and the total number of particles distributed over all chunks for the given output step can be found from header entries:

```
# nparticles      1234933
# Totparticles    2000000
# nchunks         2
```

The above indicates that the chunk 0 file (particles.0001_ch00.flt) has nparticles=1234933. The rest of the particles are in chunk 1, with a total of Totparticles=2 million particles. There are a total 2 chunks (nchunks=2) for the given output step.

Note: The max size of a chunk file is set to 1.9 GB by default. This can be changed by editing the PARTICLES_LP_MAX_FILE_SIZE parameter in particles.h. It should be noted that in the rare occasion when a single processor has many particles such that the total memory of the particles to be written exceeds 2 GB, an error will occur. This is not generally expected in standard scenarios where particles will be distributed over several processors. Hence, the user change the size limit to more than 2GB, but at their own peril. This will be updated in future releases.

Members of the particle structure may be individually selected / deselected for a particular format by calling the function SetOutputVar() from your userdef_output.c in a similar fashion to the fluid part,

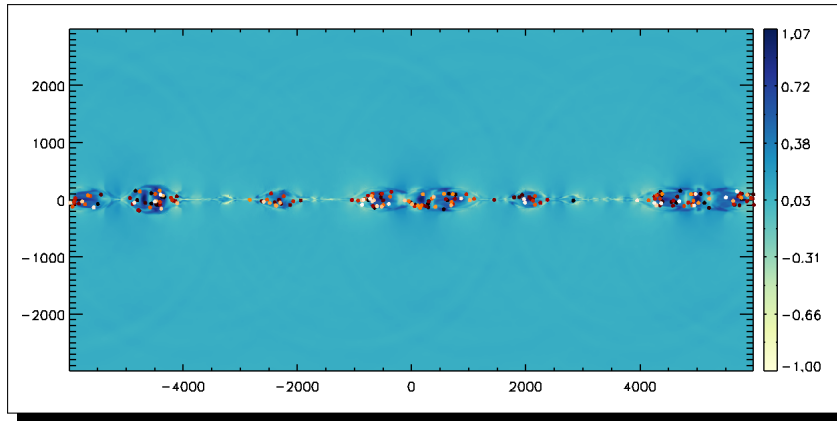


Figure 10.1: Composite image showing fluid density and particles coloured by energy

see §???. This function should be called inside the `ChangeOutputVar()` function. The following piece of code, for instance, disables writing of the `color` and `id` structure fields when writing single precision (particle) datafiles and `tinj` when writing VTK datafiles:

```
void ChangeOutputVar ()
{
  #if PARTICLES
    SetOutputVar ("color", PARTICLES_FLT_OUTPUT, NO);
    SetOutputVar ("id",    PARTICLES_FLT_OUTPUT, NO);
    SetOutputVar ("tinj",  PARTICLES_VTK_OUTPUT, NO);
  #endif
}
```

A complete list of structure field names may be found in `particles_set_output.c`.

Note: Structure fields *should not* be modified in the case of double-precision datafiles if they're used for restarting purposes. Single-precision and VTK datafile, on the other hand, contain only basic members and can be freely customized.

10.3.1 Visualization

Particle datafiles can be visualized in IDL, VisIt or pyPLUTO.

10.3.1.1 Visualization with IDL

The procedure `PARTICLES_LOAD` can be used to read binary or VTK particle datafiles. The following example can be used to display a composite image of both fluid density and particles coloured by energy (suitable for CR particles):

```
IDL> ; -- Display fluid data --
IDL> DISPLAY,x1=x1,x2=x2,alog10(rho),/VBAR, xrange=xrange, yrange=yrange
IDL>
IDL> ; -- Filter particles by energy --
IDL> Ek      = particles.vx1^2 + particles.vx2^2 + particles.vx3^2
IDL> indx    = WHERE(Ek GT MAX(Ek)*0.9)
IDL> particles = particles[indx]
IDL> Ek      = Ek[indx]

IDL> ; -- Filter particles spatially --
IDL> indx = WHERE(particles.x1 GT xrange[0] AND particles.x1 LT xrange[1])
IDL> particles = particles[indx]
IDL> Ek      = Ek[indx]

IDL> ; -- Sort particles --
IDL> particles = particles[sort(Ek)]
IDL> Ek      = Ek[sort(Ek)]

IDL> ; -- Overlay the most 200 energetic particles --
IDL> LOADCT,/SIL,3
IDL> nparts = 200
IDL> PARTICLES_OPLLOT, particles[0:nparts],symsize=1, color=Ek[0:nparts]
```

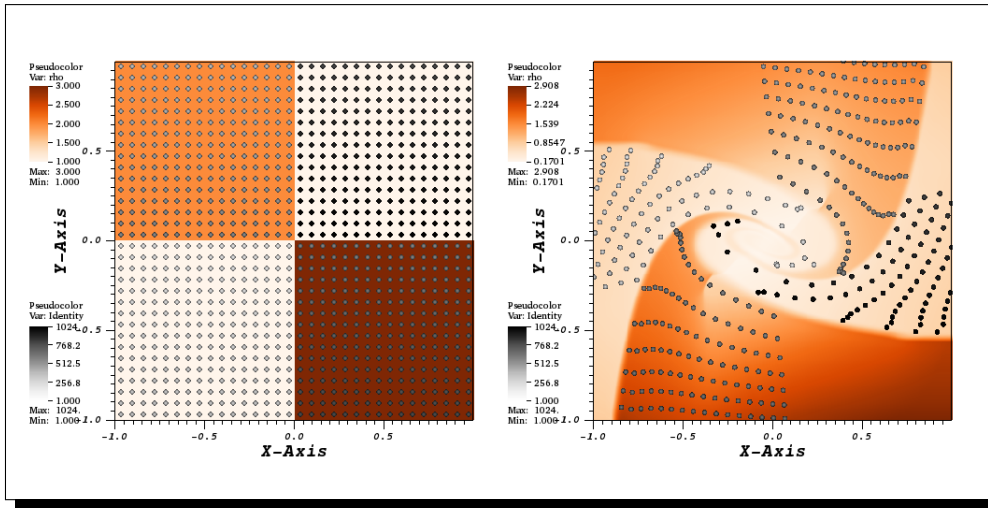


Figure 10.2: A composite image of both 2D fluid density (in color) and particles colored by identity from a binary *vtk* file using the Visit visualization tool. *Left* Initial state and final state (textitright) of the run. Note that LPs do not cross the contact discontinuity.

10.3.1.2 Visualization with VisIt

VisIt can be used to visualize particles datafile. If you're overlaying 2D images fluid with particles, the project operator should be applied. The figure 10.2 shows the evolution of fluid density for the 2D Riemann problem in background and particles over-layed that are colored with its identity which is a monotonically increasing set of integers.

10.3.1.3 Visualization with pyPLUTO

The module *LoadPart* can be used to read binary and VTK particle datafiles. The following example can be used to display a composite image of both 2D fluid density and particles colored by velocity magnitude from a binary *vtk* file:

```
import numpy as np
import pyPLUTO # Loading the relevant packages

data_path = pyPLUTO.find_example("Particles/LP/Riemann_2D") # Set the relative path to the data folder

Data_f = pyPLUTO.Load(path=data_path) # Loading the fluid and particle data into two pload objects
Data_p = pyPLUTO.LoadPart(path=data_path)

Image = pyPLUTO.Image(figsize=[9.5, 8], nwin=12) # Creating the image and the axes
Image.create_axes(right=0.8)
Image.create_axes(nrow=2, left=0.81, right=0.85, hspace=0.01)

# Plotting the data
pcm = Image.display(
    Data_f.rho, x1=Data_f.x1, x2=Data_f.x2, ax=0, aspect="equal"
)
indx = np.argsort(Data_p.vx1**2 + Data_p.vx2**2 + Data_p.vx3**2)

Image.colorbar(pcm, cax=1, clabel=r"$\rho$") # Plot the fluid colorbar

# Plot the particles
pcm = Image.scatter(
    Data_p.x1, Data_p.x2, ax=0, c=Data_p.id, cmap="Greys_r",
    title="Test 12 - LP Riemann 2D test", titlesize=23,
    ms=20, vmin=0, xlabel="x", ylabel="y",
)

Image.colorbar(pcm, cax=2, clabel="id") # Plot the particles colorbar

Image.savefig("test12_riemannlp.png") # Saving the image and showing the plot
# pyPLUTO.show()
```

The image obtained after running the above code for the Riemann 2D problem is shown in figure 10.3

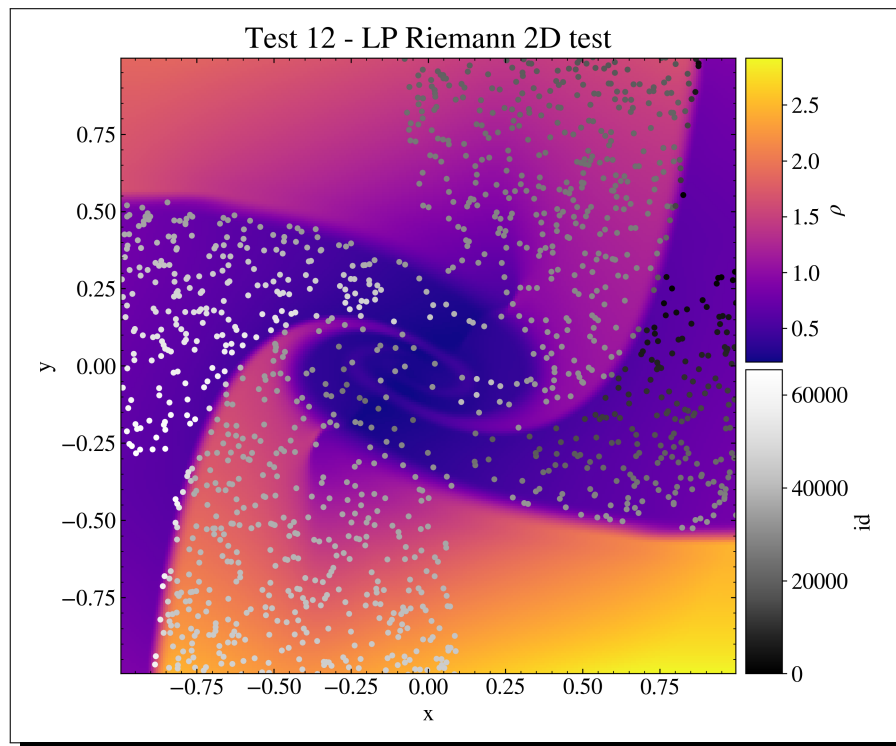


Figure 10.3: A composite image of both 2D fluid density (in color) and particles colored by velocity magnitude from a binary *vtk* file from Riemann 2D test problem

11. I/O and Visualization

In this Chapter we describe the data formats supported by the static grid version of **gPLUTO** and how they can be read and visualized with some popular visualization packages.

11.1 Output Data Formats

With the static version of **gPLUTO**, data can be dumped to disk in a variety of different formats. The majority of them is supported on serial as well as parallel systems. The available formats are classified based on their file extensions:

- *.dbl: double-precision (8 byte) binary data (serial/parallel);
- *.flt: single-precision (4 byte) binary data (serial/parallel);
- *.dbl.h5: double-precision (8 byte) HDF5 data (serial/parallel);
- *.flt.h5: single-precision (4 byte) HDF5 data (serial/parallel);
- *.vtk: VTK (legacy) file format using structured or rectilinear grids (serial/parallel);
- *.tab: tabulated multi-column ascii format (serial only);

Output files are named as base.nnnn.ext, where base is either "data" (when all variables are written to a single file) or the name of the corresponding variable (when each variable is written to a different file, see Table 11.1), nnnn is a four-digit zero-padded integer counting the output number and ext is the corresponding file extension listed above. By default, data files are written in the local working directory unless a different location has been specified using `output_dir` in your `pluto.ini`, see §4.6. There's no distinction between serial or parallel mode.

Base name	Variable	Single record size
rho	Density	$N_1 \times N_2 \times N_3$
prs	Pressure	$N_1 \times N_2 \times N_3$
vx1	x_1 velocity	$N_1 \times N_2 \times N_3$
vx2	x_2 velocity	$N_1 \times N_2 \times N_3$
vx3	x_3 velocity	$N_1 \times N_2 \times N_3$
bx1	x_1 mag. field	$N_1 \times N_2 \times N_3$
bx2	x_2 mag. field	$N_1 \times N_2 \times N_3$
bx3	x_3 mag. field	$N_1 \times N_2 \times N_3$
bx1s	x_1 stag. mag. field	$(N_1 + 1) \times N_2 \times N_3$
bx2s	x_2 stag. mag. field	$N_1 \times (N_2 + 1) \times N_3$
bx3s	x_3 stag. mag. field	$N_1 \times N_2 \times (N_3 + 1)$
enr	Rad. energy density	$N_1 \times N_2 \times N_3$
fr1	x_1 rad. flux	$N_1 \times N_2 \times N_3$
fr2	x_2 rad. flux	$N_1 \times N_2 \times N_3$
fr3	x_3 rad. flux	$N_1 \times N_2 \times N_3$
tr1	first tracer	$N_1 \times N_2 \times N_3$

Table 11.1: Base prefix for multiple data set. The size is in units of 4 (for the `flt` format) or 8 (for the `dbl` format) bytes.

The default setting is described separately for each output in the next subsections. For each format, it is possible to dump all or just some of the variables, and additional user-defined variables and may be written as well, see §11.2.

Each format has an independent output frequency and an associated log file (i.e. `dbl.out`, `flt.out`, `vtk.out` and so forth) keeping track of the dump history. Three additional files, `grid.out`, `sysconf.out` and `restart.out`, contain - respectively - grid, system-related and restart information, respectively.

Important: some visualization packages need the information stored in `*.out` files. We thus strongly recommend to backup these files together with the actual datafiles.

11.1.1 Binary Output: `dbl` or `flt` data formats

Binary data can be dumped to disk at a given time step as i) one single file containing all variables (by selecting `single_file` in `pluto.ini`) or ii) as a set of separate files for each variable (`multiple_files`). We recommend the second option for large data sets. The base name is set to `data` for a single data file containing all of the fields, or takes the name of the corresponding variable if multiple sets are preferred, see Table 11.1.

The corresponding log file (`dbl.out` or `flt.out`) is a multi-column ascii files of the form:

```

.      .      .      .      .      .      .      .      .
.      .      .      .      .      .      .      .      .
.      .      .      .      .      .      .      .      .
nout   t      dt      nstep single_file little var1 var2 ...
.      .      .      .      .      .      .      .      .
.      .      .      .      .      .      .      .      .

```

where `nout`, `t`, `dt` and `nstep` are, respectively, the file number, time, time step and integration step at the time of writing. The next column (`single_file/multiple_files`) tells whether a single-file or multiple-files are expected. The following one (`little/big`) gives the endianness of the architecture, whereas the remaining columns list the variable names and their order in this particular format.

Default: The default is to write ALL fields in `dbl` format, whereas to exclude staggered magnetic field components (if any) from the `flt` format.

11.1.2 HDF5 Output: `dbl.h5` or `flt.h5` data formats

HDF5 output format can be used in the static grid version if **gPLUTO** has been successfully compiled with the serial or parallel version of the HDF5 library, see §3.3. The file extension is `.h5` and output files are written in Pixie format, a single-block, rectilinear mesh file using HDF5, may be related to the Polar Ionospheric X-Ray Imaging Experiment.

The conventions used in writing `.dbl.h5` or `.flt.h5` files are the same ones adopted for the `.dbl` and `.flt` data formats. However, with HDF5, all variables are written to a single file.

Pixie files can be opened and visualized directly by different softwares, like *VisIt* and *Paraview*. Since we have found compatibility issues with some versions of these visualization softwares, each file comes along with a supplementary `.xmf` text file in XDMF format that describes the content of the corresponding HDF5 file and can be opened correctly by *VisIt* and *Paraview*, see §11.3.4.

Default: The default is to write ALL fields in `.dbl.h5` format, whereas to exclude staggered magnetic field components (if any) from the `.flt.h5` format.

11.1.3 Restart

Restart can be performed from double-precision binary data files (`.dbl`) by invoking the code with the command-line option `-restart n`, where `n` is the output file number. Alternatively, restart can also be performed from HDF5 data files (`.dbl.h5`) using the option `-h5restart n` (see §1.4.2), where `n` again denotes the output file number. Files must be located in the `output_dir` directory specified in your `pluto.ini`, together with the corresponding descriptors (`dbl.out` or `dbl.h5.out`)

In addition, **gPLUTO** requires the file `restart.out`, which is updated at each output of `.dbl` or `.dbl.h5` files, in order to recover the necessary runtime information.

Additional problem-dependent information can be stored using the array `data->restartParam[]` at any point in the code. This array can contain up to `MAX_RESTART_PARAMS` double-precision values and it is dumped in binary format to `restart.out`.

11.1.4 VTK Output: legacy vtk data format

VTK (from the Visualization ToolKit format) output follows essentially the same conventions used for the .dbl or .flt outputs. Single or multiple VTK files can be written by specifying either *single_file* or *multiple_files* in your pluto.ini and data values are always written using single precision with byte order set to big endian.

The mesh topology uses a rectilinear grid format for *CARTESIAN* or *CYLINDRICAL* geometry while a structured grid format is employed for *SPHERICAL* geometries. Data is written with the *CELL_DATA* attribute and grid nodes (or vertices) are used to store the mesh.

The following symbolic constants (§2.3) can be used to control some options of the output .vtk files:

- **VTK_TIME_INFO**: when set to *YES*, time information will be added to the header section of the .vtk file. Beware that standard VTK files do not have a specific construct for adding time information and, by doing so, this information will be available only to the VisIt visualisation software (see §11.3.4) which implements a convention where *CYCLE* and *TIME* values can be specified as *FieldData* in the file. If you're using Paraview or other visualisation software different from VisIt, enabling this option will most likely result in a software crash.

See also Table B.1.

If a VTK file is written to disk, the log file vtk.out is updated in the same manner as dbl.out or flt.out.

Default: By default, all variables except staggered magnetic field components (if any) are written.

11.1.5 ASCII Output: tab Data format

The *tab* format may be used for one dimensional data or relatively small two dimensional arrays in serial mode only. We warn that this output is not supported in parallel mode. The output consists in multi-column ascii files named data.nnnn.tab of the form:

```

.      .      .      .      .      .
.      .      .      .      .      .
.      .      .      .      .      .
x(i)  y(j)  var1(i,j)  var2(i,j)  var3(i,j)  ...
.      .      .      .      .      .
.      .      .      .      .      .
.      .      .      .      .      .

```

where the index j changes faster and a blank records separates blocks with different i index.

Default: By default, all variables except staggered magnetic field components (if any) are written.

11.1.6 The grid.out output file

The grid.out file contains information about the computational grid used during the simulation. It is an ASCII file starting with a comment-header containing the creation date, dimension and geometry of the grid:

```

# *****
# gPLUTO <version>
# Generated on <date>
#
# DIMENSIONS: <DIMENSIONS>
# GEOMETRY:   <GEOMETRY>
# X1: [ <x1_beg>, <x1_end>], <nx1> point(s), <ngh> ghosts
# X2: [ <x2_beg>, <x2_end>], <nx2> point(s), <ngh> ghosts
# X3: [ <x3_beg>, <x3_end>], <nx3> point(s), <ngh> ghosts
# *****

```

The rest of the file is made up of 3 sections, one for each dimension, giving the (interior) number of point followed by a tabulated multi-column list containing (from left to right) the point number, left and right cell interfaces:

```

nx1
.      .      .
.      .      .
.      .      .
<point number> <cell left edge> <cell right edge>
.      .      .
.      .      .
.      .      .

```

and similarly for the x_2 and x_3 directions.

11.2 Customizing your output

Output can be customized by editing the function `UserDefOutput()` in the source file `Src/userdef_output.c` in the **gPLUTO** distribution. We recommend to copy this file into your working directory and modify the default settings, if necessary. Changes can be made by i) introducing new additional variables and ii) altering the default attributes.

Adding new variables. New variables can be written to disk in any of the available formats previously described. The number and names of these extra variables is set in your `pluto.ini` initialization file under the label “uservar”. The function `UserDefOutput()` (located inside `Src/userdef_output.cpp`) tells **gPLUTO** how these variables are computed.

As an example, suppose we want to compute and write temperature ($T = p/\rho$) and the z component of vorticity ($\omega = \partial_x v_y - \partial_y v_x$). Then one has to set

uservar 2 T vortz

in your `pluto.ini` under the `[Static Grid Output]` block. This informs **gPLUTO** that 2 additional variables named `T` and `vortz` can be included in any of the available output format. They should be computed and linked to a specific output type through the function `UserDefOutput()`:

```
void UserDefOutput (const Data *d, Grid *grid)
{
    int i,j,k;
    int *ntot = grid->ntot;
    double *dx = grid->dx[IDIR]; // shortcut to dx
    double *dy = grid->dx[JDIR]; // shortcut to dy
    static Ary3D T, vortz;
    RBox box;

    RBoxDefine (grid->lbeg, grid->lend, &box);
    if (T.isAllocated() == NO){
        T.create (ntot[IDIR], ntot[JDIR], ntot[KDIR]);
        vortz.create (ntot[IDIR], ntot[JDIR], ntot[KDIR]);
    }

    LinkUserVar(&T, "T", DBL_OUTPUT);
    LinkUserVar(&vortz, "vortz", DBL_OUTPUT);

    BOX_LOOP(&box,i,j,k){
        T(i,j,k) = d->Vc(i,j,k,PRS)/d->Vc(i,j,k,RHO);
        vortz(i,j,k) = 0.5*(d->Vc(i+1,j,k,VX2) - d->Vc(i-1,j,k,VX2))/dx[i]
            - 0.5*(d->Vc(i,j+1,k,VX1) - d->Vc(i,j-1,k,VX1))/dy[j];
    }
}
```

The function allocates static memory area for the new arrays `T` and `vortz` and link them to a particular output with `LinkUserVar()`. The `BOX_LOOP()` macro performs a loop on the whole computational domain (boundary excluded) in order to compute `T(i,j,k)` and `vortz(i,j,k)`. Once **gPLUTO** runs, these two variables will automatically be written in the selected formats (`DBL_OUTPUT` in the example).

Changing Default Attributes To include/exclude a variable from a certain output type, add

```
SetOutputVar(var, type, YES/NO);
```

inside the function. Here “var” is a string containing the name of a variable listed in Table 11.1 or an additional one defined in your `pluto.ini`. The “type” argument can take any value among: `DBL_OUTPUT`, `FLT_OUTPUT`, `VTX_OUTPUT`, `DBL_H5_OUTPUT`, `FLT_H5_OUTPUT`, `TAB_OUTPUT`, `PPM_OUTPUT`, `PNG_OUTPUT`.

Note that the default for `.dbl` of `.dbl.h5` datasets should never be changed since restarting from a given file requires ALL variables being evolved in time.

11.3 Visualization

gPLUTO data files can be read with a variety of commercial and open source packages. In what follows we describe how **gPLUTO** data files can be read and visualized with IDL¹, VisIt², ParaView³, PyPLUTO (§11.3.3) and Gnuplot⁴. Table 11.2 show some of the visualization softwares supporting different output formats.

Table 11.2: Output data formats and supported graphic visualization packages.

File Format	Gnuplot	IDL	Paraview	PyPluto	Visit
.dbl	✓	✓	✓	✓	✓
.flt	✓	✓	✓	✓	✓
.vtk	-	✓	✓	✓	✓
.dbl.h5	-	✓	✓	✓	✓
.flt.h5	-	✓	✓	✓	✓
.hdf5	-	✓	✓	✓	✓
.tab	✓	-	-	✓	-

We recall that reading of .dbl or .flt files must be complemented by grid information which is stored in a separate file (grid.out). On the other hand, VTK and HDF5 files (.xmf / .h5 , .vtk or .hdf5) are “stand-alones” in the sense that they embed grid information and can be opened alone.

11.3.1 Visualization with Gnuplot

Gnuplot can be used to visualize relatively small or moderately large 1- or 2D datasets written with the tabulated (.tab) or binary data formats (.dbl or .flt)⁵. Gnuplot can be started at the command line by simply typing

```
> gnuplot
```

In the following we give a short summary of the available options while a more detailed documentation can be found in Doc/gnuplot.html.

Ascii Data Files. If you enabled the .tab output format in pluto.ini, you can plot 1D data from, e.g., the first output file by typing

```
gnuplot> plot "data.0001.tab" u 1:3 title "Density"
gnuplot> replot "data.0001.tab" u 1:5 title "Pressure" # overplot
```

Here the first column corresponds to the x coordinate, the second column to the y coordinate and flow data values start from the 3rd column. Fig. 11.1 shows the density and pressure profiles for the Sod shock tube problem (conf. #03 in Test_Problems/HD/Sod) using the previous commands.

Two-dimensional ascii datafiles can also be visualized using the splot command. Fig. 11.2 shows a simple contour drawing of the final solution of the Mach reflection test problem (remember to enable .tab output) using

```
gnuplot> set contour
gnuplot> set cntrparam level incremental 0.1,0.2,20 # Uniform levels from 0.1 to 20
gnuplot> set view map
gnuplot> unset surface
gnuplot> unset key
gnuplot> splot "data.0001.tab" u 1:2:3 w lines
```

¹<http://www.exelisvis.com/>

²<https://wci.llnl.gov/codes/visit/home.html>

³<http://www.paraview.org/>

⁴<http://www.gnuplot.info>

⁵Version 4.2 or higher is recommended.

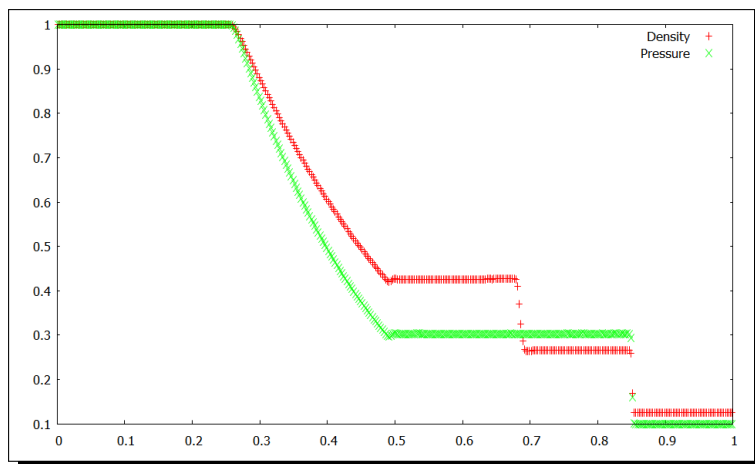


Figure 11.1: Density and pressure plots for the Sod shock tube using Gnuplot.

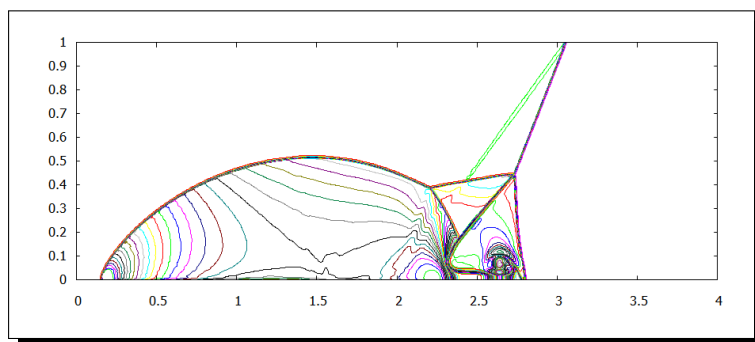


Figure 11.2: Density and pressure plots for the Sod shock tube using Gnuplot.

Binary Data Files. Starting with Gnuplot 4.2, raw binary files are also supported. In this case, grid information (being stored in separate files) must be supplied explicitly through appropriate keywords making the syntax a little awkward. To ease up this task, we recommend to enable the keyword `GNUPLUTO_HEADER` to `YES` (in your definitions.h) which causes **gPLUTO** to write a Gnuplot header file (`pluto.gp`) containing some useful information.

In addition, one can take advantage of the scripts provided with the code distribution in `Tools/Gnuplot`. For this, we recommend to define the `GNUPLUTO_LIB` environment variable (in your shell) which will be appended to the loadpath of Gnuplot:

```
> export GNUPLUTO_LIB=$PLUTO_DIR/Tools/Gnuplot # use setenv for tcsh users
```

You can also define the loadpath directly from Gnuplot:

```
gnuplot> set loadpath '<pluto_full_path>/Tools/Gnuplot'
```

A typical gnuplot session can be started as follows:

```
gnuplot> dtype = "dbl"      # Select the data type to read
gnuplot> load "pluto.gp"    # Read and store grid info, variable indices, and so forth
gnuplot> nvar = vx2         # Set the variable name
```

The 1st line selects the datatype (`dbl` in this case, which is also the default), the 2nd line invokes the `pluto.gp` script (created automatically by **PLUTO**) that initializes variables such as number of points (`nx1`, `nx2`, `nx3`), domain range (`x1beg`, `x1end` and `x2beg`, `x2end`), grid spacing (`dx1`, `dx2`), variable indices (e.g. `rho`, `vx1`, `vx2`, ... `Bx1`, `Bx2`, etc...).

For additional documentation and examples please refer to `Doc/gnuplot.html`.

11.3.2 Visualization with IDL

IDL (Interactive Data Language) is a vectorized programming language commonly used in the astronomical community for interactive processing of large amounts of data. The **gPLUTO** code distribution comes with a number of useful routines written in the IDL programming language to read and visualize data written with **gPLUTO**. Several functions and procedures for data visualization and analysis can be found in the `Tools/IDL` directory which we strongly advise adding to the IDL search path. IDL function documentation can be found in `Doc/idl.tools.html`.

The PLOAD Procedure The PLOAD procedure can be considered the main driver for reading data stored in one of the following formats: .dbl, .flt, .vtk, .dbl.h5, .flt.h5 or .hdf5. PLOAD requires the information stored in the corresponding data log file (e.g. grid.out, dbl.out, flt.out, etc...) to initialize common block variables and grid information shared by other functions and procedures in the Tools/IDL/ sub-directory. Because of this reason, it should always be called at the very beginning of an IDL session. For example:

```
IDL> PLOAD,3; Read variables {rho, vx1, ...} from 3rd output .dbl file

IDL> PLOAD,5,/FLOAT, VAR="prs"; Read pressure from 5th output .flt datafile

IDL> ; Read output 9 from the directory "Large_Data/", do not store it
IDL> ; into memory but use IDL file association (preferred for large datasets):
IDL> PLOAD,dir="Large_Data/",9,/ASSOC
```

By default, PLOAD tries to read binary data in double precision if dbl.out is present. To select a different format, a corresponding keyword must be supplied (e.g. /FLOAT, /H5 or /HDF5 or a combination of them, see Doc/idl.tools.html).

When PLOAD is called for the first time, it initializes the following four common blocks:

- PLUTO_GRID: contains grid information such as the number of points (nx1,nx2,nx3), coordinates (x1,x2,x3) and mesh spacing (dx1, dx2, dx3);
- PLUTO_VAR: the number (NVAR) and the names of variables being written for the chosen format. Variable names follow the same convention adopted in **gPLUTO**, e.g., rho, vx1, vx2, ..., bx1, bx2, prs, .. and so on;
- PLUTO_RUN: time stepping information such as output time (t), time step (dt) and total number of files (nlast).
- PLUTO_USERDEF: an optional common block which can be used to collect user-defined variables. User-defined variables written to disk can be loaded by introducing the MATCH_USERDEF_VARNAME procedure and manually adding the desired variable names. A typical example is the file userdef.pro (in your local working directory):

```
COMMON PLUTO_USERDEF, flag, vort; Define the common block

PRO MATCH_USERDEF_VARNAME, vpt, name, silent=silent
COMMON PLUTO_GRID
COMMON PLUTO_VAR
COMMON PLUTO_RUN
COMMON PLUTO_USERDEF

CASE name OF
  "flag": BEGIN flag = vpt & PRINT,"> Reading ",name & END
  "vort": BEGIN vort = vpt & PRINT,"> Reading ",name & END
ELSE:
ENDCASE

vpt = 0; Free memory
END
```

The previous example will add the variables flag and vort and allow PLOAD to read and store the content in the corresponding variable names. During an IDL session, both PLOAD and userdef.pro must be compiled first (e.g. IDL> .r pload followed by IDL> .r userdef).

PLOAD can be used inside a normal IDL script, after it has been invoked at least once (or compiled with .r pload).

The DISPLAY Procedure DISPLAY is a general-purpose visualization routine and the source code can be found in Tools/IDL/display.pro The DISPLAY procedure can be used to display the intensity map of a 2D variable to a graphic window, e.g.,

```
IDL> PLOAD,3 ; load the third data set in double precision
IDL> DISPLAY,x1=x1,x2=x2,alog10(rho), title='Density',/vbar
IDL> DISPLAY,x1=x1,x2=x2,vx1,title='X-Velocity',nwin=1
```

The second line displays the density logarithm and the third line displays the x_1 component of velocity in a new window.

Another example, shown in Fig. 11.3, shows how to visualize magnetic pressure and density in two different windows and overplot the velocity field. For more details, consult Doc/idl.tools.html.

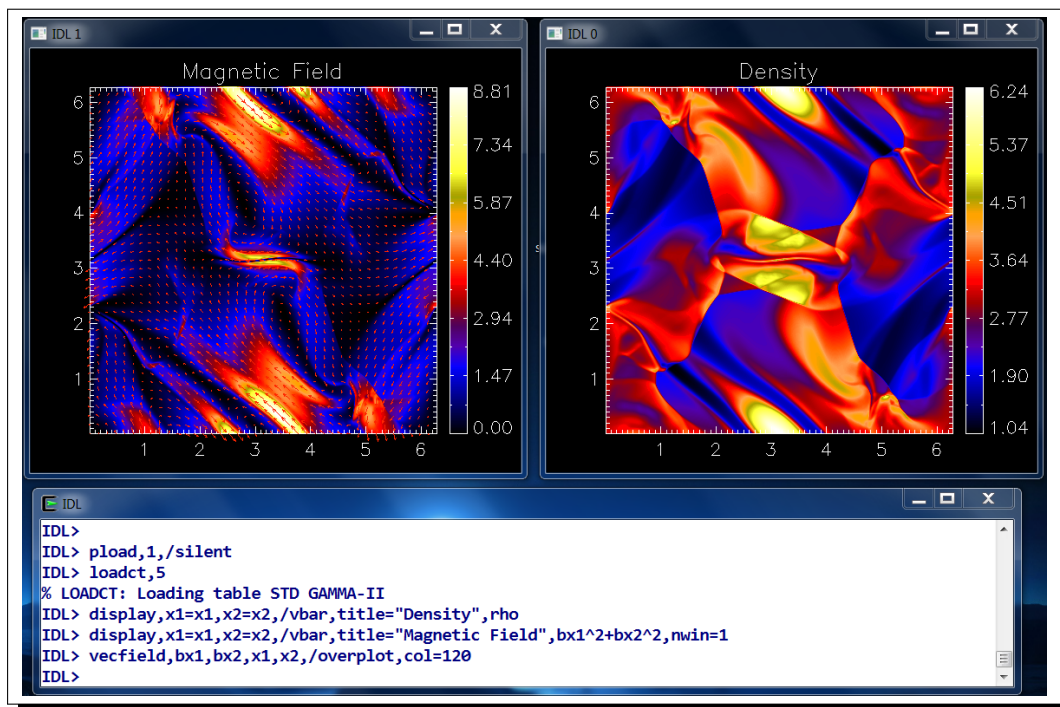


Figure 11.3:
An example of
visualization
in IDL
using the
`display.pro`
routine.

11.3.3 Visualization with PyPLUTO

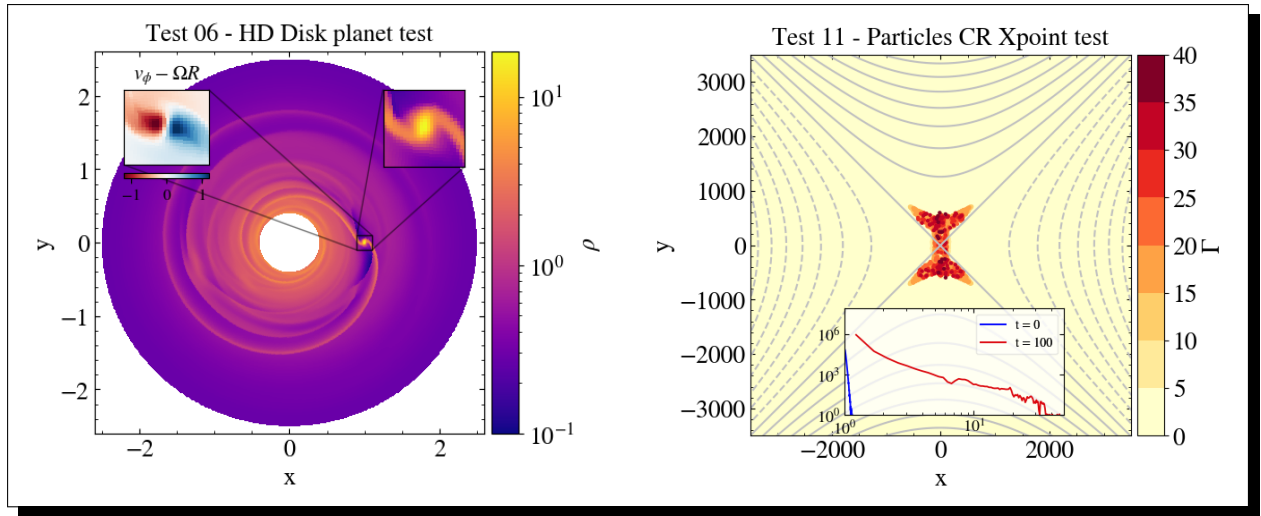


Figure 11.4: An example of visualization with the PyPLUTO tool.

All data formats can be visualized using the PyPLUTO Python package. The module is present in the Python Package Index and can be downloaded by typing

```
pip install py-pluto[gui] --upgrade
```

on the terminal. This tool is also included in the current code distribution in the directory `Tools/PyPLUTO/` and provides Python modules (Python version ≥ 3.11 is required) to load, visualize and analyze data. Details of the Installation and Getting Started can be found at the `Doc/PyPLUTO.html` page.

On successful installation, the user can load data in the following manner:

```
> ipython --pylab
In [1]: import pyPLUTO as pp
# for loading data.0010.dbl
In [2]: D = pp.Load(10,path=<path to data dir>)
# for loading data.0010.flt
In [3]: D = pp.Load(10,path=<path to data dir>, datatype='float')
```

Here, `D` is a `Load` object that has all the information regarding the variable names and their values which are stored as arrays. It also has the respective grid and time information. For example, `D.x1` is the numpy `x`-array, `D.rho` - is the numpy density array, `D.vx1` - is the numpy `vx1` array and so on. These numpy arrays can be easily visualized using `matplotlib`, python's plotting library. The `pyPLUTO` package also provides one class - `Image`. While the `Load` class contains some frequently used functions for analysis, the `Image` class contains all the routines for data plotting. Details about these classes along with their usage can be found in HTML document referred above.

Along with the code, an example folder with some sample `.py` files are provided for certain test problems. The source codes from these files along with their outputs are listed in the HTML documentation.

It is required to first run the respective test problem and generate the data files, after which the user can run the sample `.py` files as follows from the `Tools/PyPLUTO/Examples` folder:

```
> python samplefile.py
```

where the `samplefile.py` are listed in 11.3.

In addition to the standard package, PyPLUTO offers a GUI, which can be launched by typing

```
pypluto-gui
```

on the terminal. The GUI (shown in Fig. 11.5) supports only fluid output for all formats and geometry.

#	Test (# in (g)PLUTO tests)	Coordinates	Tools adopted
1	HD Sod (1)	Cartesian	None
2	MHD Orszag-Tang (2)	Cartesian	None
3	MHD Rayleigh-Taylor instability (1)	Cartesian	None
4	MHD Rotor (4)	Cylindrical	Contour lines
5	MHD Field loop (5)	Cartesian	Field lines
6	HD Disk-planet (6)	Cylindrical	Zoom
7	RMHD Kelvin-Helmholtz Instability (1)	Cartesian	Field lines
8	MHD Torus (4)	Spherical	Field lines
9	MHD Blast (9)	Cartesian	Legends
10	HD Riemann 2D (3)	Cartesian	Interactive
11	CR Xpoint (1)	Cartesian	Spectrum
12	LP Riemann2D (1)	Cartesian	Colorbars
13	HD Flow Past Cylinder (3)	Cylindrical	AMR

Table 11.3: Test problems present in the repository with a summary of their main features. The columns, from left to right, show the test number, the benchmark name, the grid coordinates and the specific tools adopted.

In Fig. 11.4 we report Exampled 6 and 11.

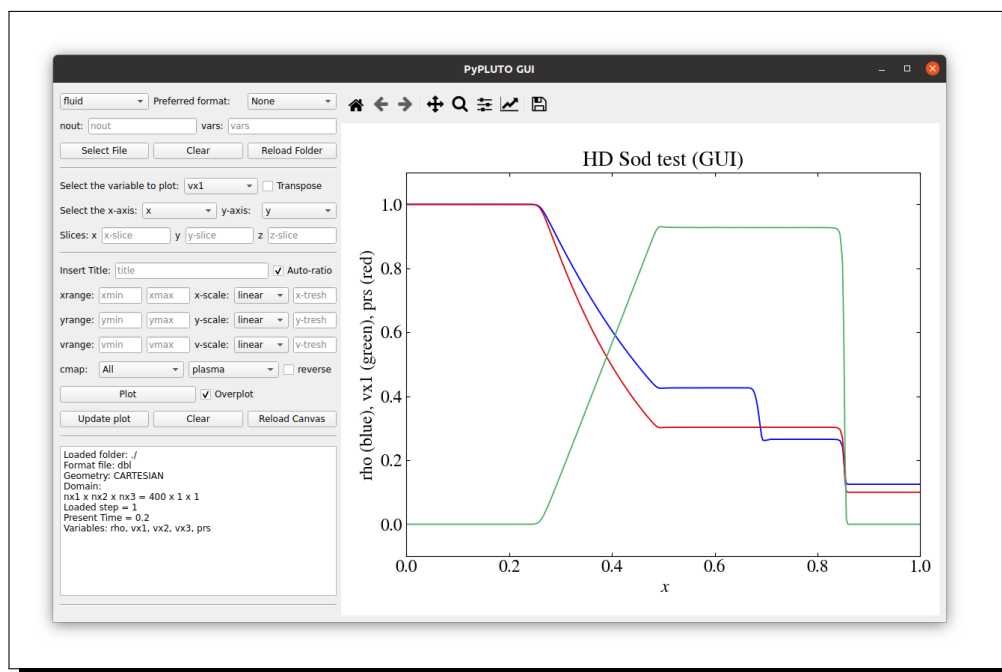


Figure 11.5: The PyPLUTO GUI.

11.3.4 Visualization with VisIt or ParaView

gPLUTO data written using VTK or HDF5 (both .h5 and .hdf5 files) formats can be easily visualized using either *VisIt* or *ParaView* available at <https://wci.llnl.gov/codes/visit/home.html> and <http://www.paraview.org/>, respectively. *VisIt* is an open source interactive parallel visualization and graphical analysis tool for

viewing scientific data. *ParaView* is an open source mutple-platform application for interactive, scientific visualization.

An example is shown in Fig. 11.6 for both software packages.

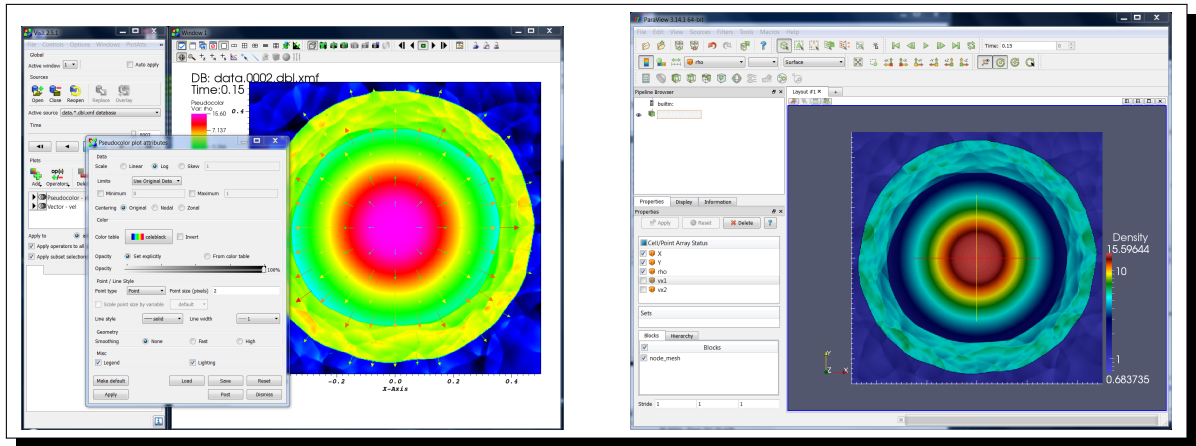


Figure 11.6: An example of visualization of an .xmf (.h5) data file using *VisIt* (left) or *ParaView* (right).

Visualization of HDF5 files. Both *VisIt* and *Paraview* interpret the cell-centered grid and data contained in the Pixie files as node-centered: as a consequence, the first and the last half cells in every direction are clipped from the images (e.g. a small sector around $\phi = 0$ is chopped from a periodic polar plot covering the 2π angle).

Therefore, for every .h5 file, **gPLUTO** writes also a .xmf text file in XDMF format that describes the content of the corresponding HDF5 file. The .xmf files can be directly opened by *VisIt* and *ParaView*, so as to provide the correct data centering and avoid the image clipping. Besides, we noticed that the Pixie reader crashes (e.g. using *ParaView* 3.14 - 3.98) or incorrectly reads the .h5 files (e.g. using *VisIt* versions after 2.6), while all versions of both *VisIt* and *Paraview* properly open the .xmf files. All the variables are read as scalar quantities.

Visualization of VTK files. **gPLUTO** writes .vtk files using a cell-centered attribute rather than point-centered (as in previous versions). Although this has not been found to be a problem for *VisIt*, many filters in *ParaView* (such as streamlines) may require to apply a *Cell Data to Point Data* filter.

A. Equations in Different Geometries

In this section we give the explicit form of the MHD and RMHD equations written in different systems of coordinates. Non-ideal terms such as viscosity, resistivity and thermal conduction are not included here. The discretizations used in the `Src/MHD/rhs.c` and `Src/RMHD/rhs.c` strictly follow these form. Equations for the non-magnetized version (HD and RHD) are obtained by setting the magnetic field vector $\mathbf{B} = \mathbf{0}$.

A.1 MHD Equations

A.1.1 Cartesian Coordinates

In Cartesian coordinates (x, y, z) , the conservative ideal MHD Equations (6.3) are discretized using the following divergence form

$$\begin{aligned}
 \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) &= 0 \\
 \frac{\partial m_x}{\partial t} + \nabla \cdot (m_x \mathbf{v} - B_x \mathbf{B}) + \frac{\partial p_t}{\partial x} &= \rho \left(g_x - \frac{\partial \Phi}{\partial x} \right) \\
 \frac{\partial m_y}{\partial t} + \nabla \cdot (m_y \mathbf{v} - B_y \mathbf{B}) + \frac{\partial p_t}{\partial y} &= \rho \left(g_y - \frac{\partial \Phi}{\partial y} \right) \\
 \frac{\partial m_z}{\partial t} + \nabla \cdot (m_z \mathbf{v} - B_z \mathbf{B}) + \frac{\partial p_t}{\partial z} &= \rho \left(g_z - \frac{\partial \Phi}{\partial z} \right) \\
 \frac{\partial}{\partial t} (E_t + \rho \Phi) + \nabla \cdot \left[(E_t + p_t + \rho \Phi) \mathbf{v} - \mathbf{B} (\mathbf{v} \cdot \mathbf{B}) \right] &= \rho \mathbf{v} \cdot \mathbf{g} \\
 \frac{\partial B_x}{\partial t} + \frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z} &= 0 \\
 \frac{\partial B_y}{\partial t} + \frac{\partial E_x}{\partial z} - \frac{\partial E_z}{\partial x} &= 0 \\
 \frac{\partial B_z}{\partial t} + \frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} &= 0
 \end{aligned} \tag{A.1}$$

where $\mathbf{v} = (v_x, v_y, v_z)$ and $\mathbf{B} = (B_x, B_y, B_z)$ are the velocity and magnetic field vectors, (E_x, E_y, E_z) are the components of the electromotive force $\mathbf{E} = -\mathbf{v} \times \mathbf{B}$, $\mathbf{g}$ is the body force vector and Φ is the gravitational potential.

A.1.2 Cylindrical Coordinates

In cylindrical coordinates (R, ϕ, z) , the conservative ideal MHD Equations (6.3) are discretized using the following divergence form

$$\begin{aligned}
\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) &= 0 \\
\frac{\partial m_R}{\partial t} + \nabla \cdot (m_R \mathbf{v} - B_R \mathbf{B} + p_t \hat{\mathbf{e}}_R) &= \rho \left(g_R - \frac{\partial \Phi}{\partial R} \right) + \frac{p_t + \rho v_\phi^2 - B_\phi^2}{R} \\
\frac{\partial (R m_\phi)}{\partial t} + \nabla \cdot [R (m_\phi \mathbf{v} - B_\phi \mathbf{B} + p_t \hat{\mathbf{e}}_\phi)] &= R \rho \left(g_\phi - \frac{1}{R} \frac{\partial \Phi}{\partial \phi} \right) \\
\frac{\partial m_z}{\partial t} + \nabla \cdot (m_z \mathbf{v} - B_z \mathbf{B} + p_t \hat{\mathbf{e}}_z) &= \rho \left(g_z - \frac{\partial \Phi}{\partial z} \right) \\
\frac{\partial}{\partial t} (E_t + \rho \Phi) + \nabla \cdot [(E_t + p_t + \rho \Phi) \mathbf{v} - \mathbf{B} (\mathbf{v} \cdot \mathbf{B})] &= \rho \mathbf{v} \cdot \mathbf{g}
\end{aligned} \tag{A.2}$$

In the previous equations $\mathbf{v} = (v_R, v_\phi, v_z)$ and $\mathbf{B} = (B_R, B_\phi, B_z)$ are the velocity and magnetic field vectors, (E_R, E_ϕ, E_z) are the components of the electric field, $\mathbf{g}$ is the body force vector and Φ is the gravitational potential. Note that curvature terms are present only in the radial component while the azimuthal component is discretized in angular momentum conserving form.

Depending on the numerical scheme, the induction equation keeps its standard curl-like representation (for constrained transport) or it is refactored in divergence form (for cell-centered schemes):

$$\begin{aligned}
\frac{\partial B_R}{\partial t} + \frac{1}{R} \frac{\partial E_z}{\partial \phi} - \frac{\partial E_\phi}{\partial z} + \frac{\partial \psi}{\partial R} &= 0 \\
\left\{ \begin{aligned} &\frac{\partial B_\phi}{\partial t} + \frac{\partial E_R}{\partial z} - \frac{\partial E_z}{\partial R} + \frac{1}{R} \frac{\partial \psi}{\partial \phi} \\ &\quad \Updownarrow \\ &\frac{\partial}{\partial t} \left(\frac{B_\phi}{R} \right) + \frac{\partial}{\partial z} \left(\frac{E_R}{R} \right) - \frac{1}{R} \frac{\partial}{\partial R} \left(R \frac{E_z}{R} \right) + \frac{1}{R} \frac{\partial}{\partial \phi} \left(\frac{\psi}{R} \right) \end{aligned} \right\} &= 0 \\
\frac{\partial B_z}{\partial t} + \frac{1}{R} \frac{\partial (R E_\phi)}{\partial R} - \frac{1}{R} \frac{\partial E_R}{\partial \phi} + \frac{\partial \psi}{\partial z} &= 0,
\end{aligned} \tag{A.3}$$

Note that the evolution of B_ϕ is written in covariant form by default (COVARIANT_BX2 set to YES). Alternatively, one may also consider the non-covariant formulation, including a source term (COVARIANT_BX2 set to NO)

$$\frac{\partial B_\phi}{\partial t} + \frac{\partial}{\partial z} (E_R) - \frac{1}{R} \frac{\partial}{\partial R} (R E_z) + \frac{1}{R} \frac{\partial \psi}{\partial \phi} = -\frac{E_z}{R} \tag{A.4}$$

where $E_z = -(v_R B_\phi - B_R v_\phi)$.

Terms in blue are the components of $\nabla \psi$ used in the GLM method only and rewritten as

$$\nabla \psi = \nabla \cdot (\psi \mathbf{l}) = \left(\frac{1}{R} \frac{\partial (R \psi)}{\partial R} - \frac{\psi}{R} \right) \hat{\mathbf{e}}_R + \left(\frac{1}{R} \frac{\partial \psi}{\partial \phi} \right) \hat{\mathbf{e}}_\phi + \left(\frac{\partial \psi}{\partial z} \right) \hat{\mathbf{e}}_z. \tag{A.5}$$

The divergence operator is defined as

$$\nabla \cdot \mathbf{F} = \frac{1}{R} \frac{\partial (R F_R)}{\partial R} + \frac{1}{R} \frac{\partial F_\phi}{\partial \phi} + \frac{\partial F_z}{\partial z}, \tag{A.6}$$

and discretized in a conservative way through the finite-volume formalism:

$$\nabla \cdot \mathbf{F} = \left(\frac{A_{i+\frac{1}{2}}^R F_{i+\frac{1}{2}}^R - A_{i-\frac{1}{2}}^R F_{i-\frac{1}{2}}^R}{\Delta \mathcal{V}} \right) + \left(\frac{A_{j+\frac{1}{2}}^\phi F_{j+\frac{1}{2}}^\phi - A_{j-\frac{1}{2}}^\phi F_{j-\frac{1}{2}}^\phi}{\Delta \mathcal{V}} \right) + \left(\frac{A_{k+\frac{1}{2}}^z F_{k+\frac{1}{2}}^z - A_{k-\frac{1}{2}}^z F_{k-\frac{1}{2}}^z}{\Delta \mathcal{V}} \right) \tag{A.7}$$

where $\Delta\mathcal{V} \equiv \Delta\mathcal{V}_{ijk} = R_i\Delta R_i\Delta\phi_j\Delta z_k$ is the cell volume while

$$A_{i-\frac{1}{2}}^R = R_{i-\frac{1}{2}} \Delta\phi \Delta z, \quad A_{j-\frac{1}{2}}^\phi = \Delta R \Delta z, \quad A_{k-\frac{1}{2}}^z = R_i \Delta R \Delta\phi, \quad (\text{A.8})$$

are the surface areas in the three directions.

A.1.3 Spherical Coordinates

In spherical coordinates (r, θ, ϕ) the ideal MHD equations (6.3) are discretized using the following divergence form

$$\begin{aligned} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) &= 0 \\ \frac{\partial m_r}{\partial t} + \nabla \cdot (m_r \mathbf{v} - B_r \mathbf{B} + p_t \hat{\mathbf{e}}_r) &= \rho \left(g_r - \frac{\partial \Phi}{\partial r} \right) + \frac{2p_t + \rho v_\theta^2 - B_\theta^2}{r} + \frac{\rho v_\phi^2 - B_\phi^2}{r} \\ \frac{\partial (rm_\theta)}{\partial t} + \nabla \cdot [r(m_\theta \mathbf{v} - B_\theta \mathbf{B} + p_t \hat{\mathbf{e}}_\theta)] &= r\rho \left(g_\theta - \frac{1}{r} \frac{\partial \Phi}{\partial \theta} \right) + \cot \theta (p_t + \rho v_\phi^2 - B_\phi^2) \\ \frac{\partial (r \sin \theta m_\phi)}{\partial t} + \nabla \cdot [r \sin \theta (m_\phi \mathbf{v} - B_\phi \mathbf{B} + p_t \hat{\mathbf{e}}_\phi)] &= r \sin \theta \rho \left(g_\phi - \frac{1}{r \sin \theta} \frac{\partial \Phi}{\partial \phi} \right) \\ \frac{\partial}{\partial t} (E_t + \rho \Phi) + \nabla \cdot [(E_t + p_t + \rho \Phi) \mathbf{v} - \mathbf{B} (\mathbf{v} \cdot \mathbf{B})] &= \rho \mathbf{v} \cdot \mathbf{g} \end{aligned} \quad (\text{A.9})$$

In the previous equations $\mathbf{v} = (v_r, v_\theta, v_\phi)$ and $\mathbf{B} = (B_r, B_\theta, B_\phi)$ are the velocity and magnetic field vectors, (E_r, E_θ, E_ϕ) are the components of the electric field, $\mathbf{g}$ is the body force vector and Φ is the gravitational potential. The divergence operator is defined as

$$\nabla \cdot \mathbf{F} = \frac{1}{r^2} \frac{\partial (r^2 F_r)}{\partial r} + \frac{1}{r \sin \theta} \frac{\partial (\sin \theta F_\theta)}{\partial \theta} + \frac{1}{r \sin \theta} \frac{\partial F_\phi}{\partial \phi}. \quad (\text{A.10})$$

Note that curvature terms are present in the radial and meridional components while the azimuthal component is discretized in angular momentum conserving form.

For the induction equation we have the following form(s):

$$\begin{aligned} \frac{\partial B_r}{\partial t} + \frac{1}{r \sin \theta} \frac{\partial (\sin \theta E_\phi)}{\partial \theta} - \frac{1}{r \sin \theta} \frac{\partial E_\theta}{\partial \phi} + \frac{\partial \psi}{\partial r} &= 0 \\ \left\{ \begin{aligned} &\frac{\partial B_\theta}{\partial t} + \frac{1}{r \sin \theta} \frac{\partial E_r}{\partial \phi} - \frac{1}{r} \frac{\partial (r E_\phi)}{\partial r} + \frac{1}{r} \frac{\partial \psi}{\partial \theta} \\ &\Downarrow \\ &\frac{\partial}{\partial t} \left(\frac{B_\theta}{r} \right) + \frac{1}{r \sin \theta} \frac{\partial}{\partial \phi} \left(\frac{E_r}{r} \right) - \frac{1}{r^2} \frac{\partial}{\partial r} \left[r^2 \left(\frac{E_\phi}{r} \right) \right] + \frac{1}{r} \frac{\partial}{\partial \theta} \left(\frac{\psi}{r} \right) \end{aligned} \right\} &= 0 \\ \left\{ \begin{aligned} &\frac{\partial B_\phi}{\partial t} + \frac{1}{r} \frac{\partial (r E_\theta)}{\partial r} - \frac{1}{r} \frac{\partial E_r}{\partial \theta} + \frac{1}{r \sin \theta} \frac{\partial \psi}{\partial \phi} \\ &\Downarrow \\ &\frac{\partial}{\partial t} \left(\frac{B_\phi}{r \sin \theta} \right) + \frac{1}{r^2} \frac{\partial}{\partial r} \left[r^2 \left(\frac{E_\theta}{r \sin \theta} \right) \right] - \frac{1}{r \sin \theta} \frac{\partial}{\partial \theta} \left[\sin \theta \left(\frac{E_r}{r \sin \theta} \right) \right] + \frac{1}{r \sin \theta} \frac{\partial}{\partial \phi} \left(\frac{\psi}{r \sin \theta} \right) \end{aligned} \right\} &= 0 \end{aligned} \quad (\text{A.11})$$

Terms in blue are the components of $\nabla \psi$ used in the GLM method only and discretized in the form of divergence plus a source term:

$$\nabla \psi = \nabla \cdot (\psi \mathbf{l}) = \left(\frac{1}{r^2} \frac{\partial (r^2 \psi)}{\partial R} - \frac{2\psi}{R} \right) \hat{\mathbf{e}}_r + \left(\frac{1}{r \sin \theta} \frac{\partial (\sin \theta \psi)}{\partial \theta} - \frac{\cot \theta \psi}{R} \right) \hat{\mathbf{e}}_\theta + \left(\frac{1}{r \sin \theta} \frac{\partial \psi}{\partial \phi} \right) \hat{\mathbf{e}}_\phi. \quad (\text{A.12})$$

The divergence operator $\nabla \cdot ()$ is discretized in conservative fashion by the following representation:

$$\nabla \cdot \mathbf{F} = \left(\frac{A_{i+\frac{1}{2}}^r F_{i+\frac{1}{2}}^r - A_{i-\frac{1}{2}}^r F_{i-\frac{1}{2}}^r}{\Delta \mathcal{V}_i} \right) + \left(\frac{A_{j+\frac{1}{2}}^\theta F_{j+\frac{1}{2}}^\theta - A_{j-\frac{1}{2}}^\theta F_{j-\frac{1}{2}}^\theta}{\Delta \mathcal{V}_i} \right) + \left(\frac{A_{k+\frac{1}{2}}^\phi F_{k+\frac{1}{2}}^\phi - A_{k-\frac{1}{2}}^\phi F_{k-\frac{1}{2}}^\phi}{\Delta \mathcal{V}_i} \right) \quad (\text{A.13})$$

where $\Delta \mathcal{V} = (r_{i+\frac{1}{2}}^3 - r_{i-\frac{1}{2}}^3) \Delta \mu \Delta \phi_k$ is the cell volume, $\mu = 1 - \cos \theta$ while

$$A_{i-\frac{1}{2}}^r = r_{i-\frac{1}{2}}^2 \Delta \mu \Delta \phi, \quad A_{j-\frac{1}{2}}^\theta = r_i \Delta r \sin \theta_{j-\frac{1}{2}} \Delta \phi, \quad A_{k-\frac{1}{2}}^\phi = r_i \Delta r \Delta \theta \quad (\text{A.14})$$

are the zone surfaces in the three directions.

A.2 (Special) Relativistic MHD Equations

A.2.1 Cartesian Coordinates

In Cartesian coordinates (x, y, z) , the relativistic MHD equations (6.12) take the form

$$\begin{aligned} \frac{\partial D}{\partial t} + \nabla \cdot (D\mathbf{v}) &= 0 \\ \frac{\partial m_x}{\partial t} + \nabla \cdot [(w + b^2)v_x \mathbf{v} - b_x \mathbf{b}] + \frac{\partial p_t}{\partial x} &= 0 \\ \frac{\partial m_y}{\partial t} + \nabla \cdot [(w + b^2)v_y \mathbf{v} - b_y \mathbf{b}] + \frac{\partial p_t}{\partial y} &= 0 \\ \frac{\partial m_z}{\partial t} + \nabla \cdot [(w + b^2)v_z \mathbf{v} - b_z \mathbf{b}] + \frac{\partial p_t}{\partial z} &= 0 \\ \frac{\partial E_t}{\partial t} + \nabla \cdot (\mathbf{m} - D\mathbf{v}) &= 0 \\ \frac{\partial B_x}{\partial t} + \frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z} &= 0 \\ \frac{\partial B_y}{\partial t} + \frac{\partial E_x}{\partial z} - \frac{\partial E_z}{\partial x} &= 0 \\ \frac{\partial B_z}{\partial t} + \frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} &= 0 \end{aligned} \quad (\text{A.15})$$

where $D = \gamma \rho$ is the lab density, $\mathbf{m} = (w + b^2)\mathbf{v} - \gamma(\mathbf{v} \cdot \mathbf{B})\mathbf{b}$ is the momentum density, w is the gas enthalpy, $b^2 = \mathbf{B}^2/\gamma^2 + (\mathbf{v} \cdot \mathbf{B})^2$, $\mathbf{v} = (v_x, v_y, v_z)$ is the velocity, $\mathbf{B} = (B_x, B_y, B_z)$ is the magnetic field in the lab frame, $\mathbf{b} = \mathbf{B}/\gamma + \gamma(\mathbf{v} \cdot \mathbf{B})\mathbf{v}$ is the covariant field, (E_x, E_y, E_z) are the components of the electromotive force $\mathbf{E} = -\mathbf{v} \times \mathbf{B}$.

A.2.2 Cylindrical Coordinates

In cylindrical coordinates (R, ϕ, z) , the RMHD Equations (6.12) are discretized using the following form

$$\begin{aligned} \frac{\partial D}{\partial t} + \nabla \cdot (D\mathbf{v}) &= 0 \\ \frac{\partial m_R}{\partial t} + \nabla \cdot [(w + b^2)v_R \mathbf{v} - b_R \mathbf{b}] + \frac{\partial p_t}{\partial R} &= + \frac{m_\phi v_\phi}{R} - \left(\frac{B_\phi}{\gamma^2} + (\mathbf{v} \cdot \mathbf{B})v_\phi \right) \frac{B_\phi}{R} \\ \frac{\partial (Rm_\phi)}{\partial t} + \nabla \cdot \left\{ R [(w + b^2)v_\phi \mathbf{v} - b_\phi \mathbf{b} + p_t \hat{\mathbf{e}}_\phi] \right\} &= 0 \\ \frac{\partial m_z}{\partial t} + \nabla \cdot [(w + b^2)v_z \mathbf{v} - b_z \mathbf{b}] + \frac{\partial p_t}{\partial z} &= 0 \\ \frac{\partial E_t}{\partial t} + \nabla \cdot (\mathbf{m} - D\mathbf{v}) &= 0 \end{aligned} \quad (\text{A.16})$$

In the previous equations $\mathbf{v} = (v_R, v_\phi, v_z)$ and $\mathbf{B} = (B_R, B_\phi, B_z)$ are the velocity and magnetic field vectors, (E_R, E_ϕ, E_z) are the components of the electromotive force $\mathbf{E} = -\mathbf{v} \times \mathbf{B}$.

For the magnetic field, divergence operators, etc, we follow §A.1.2. Note that curvature terms are present in the radial component while the azimuthal component is discretized in angular momentum conserving form. The corresponding divergence operators are

A.2.3 Spherical Coordinates

In spherical coordinates (r, θ, ϕ) the RMHD equations (6.12) are discretized using the following divergence form

$$\begin{aligned}
\frac{\partial D}{\partial t} + \nabla \cdot (D\mathbf{v}) &= 0 \\
\frac{\partial m_r}{\partial t} + \nabla \cdot \left[(w + b^2)v_r\mathbf{v} - b_r\mathbf{b} \right] + \frac{\partial p_t}{\partial r} &= f_{g,r} + \frac{m_\theta v_\theta + m_\phi v_\phi}{r} + \\
&\quad - \left(\frac{B_\theta}{\gamma^2} + (\mathbf{v} \cdot \mathbf{B})v_\theta \right) \frac{B_\theta}{r} - \left(\frac{B_\phi}{\gamma^2} + (\mathbf{v} \cdot \mathbf{B})v_\phi \right) \frac{B_\phi}{r} \\
\frac{\partial m_\theta}{\partial t} + \nabla \cdot \left[(w + b^2)v_\theta\mathbf{v} - b_\theta\mathbf{b} \right] + \frac{1}{r} \frac{\partial p_t}{\partial \theta} &= f_{g,\theta} - \frac{m_\theta v_r - \cot \theta m_\phi v_\phi}{r} \\
&\quad + \left(\frac{B_\theta}{\gamma^2} + (\mathbf{v} \cdot \mathbf{B})v_\theta \right) \frac{B_r}{r} - \cot \theta \left(\frac{B_\phi}{\gamma^2} + (\mathbf{v} \cdot \mathbf{B})v_\phi \right) \frac{B_\phi}{r} \\
\frac{\partial m_\phi}{\partial t} + \nabla^r \cdot \left[(w + b^2)v_\phi\mathbf{v} - b_\phi\mathbf{b} \right] + \frac{1}{r \sin \theta} \frac{\partial p_t}{\partial \phi} &= f_{g,\phi} \\
\frac{\partial E_t}{\partial t} + \nabla \cdot (\mathbf{m} - D\mathbf{v}) &= \mathbf{v} \cdot \mathbf{f}_g \\
\frac{\partial B_r}{\partial t} + \frac{1}{r \sin \theta} \frac{\partial (\sin \theta E_\phi)}{\partial \theta} - \frac{1}{r \sin \theta} \frac{\partial E_\theta}{\partial \phi} &= 0 \\
\frac{\partial B_\theta}{\partial t} + \frac{1}{r \sin \theta} \frac{\partial E_r}{\partial \phi} - \frac{1}{r} \frac{\partial (r E_\phi)}{\partial r} &= 0 \\
\frac{\partial B_\phi}{\partial t} + \frac{1}{r} \frac{\partial (r E_\theta)}{\partial r} - \frac{1}{r} \frac{\partial E_r}{\partial \theta} &= 0
\end{aligned} \tag{A.17}$$

Note that curvature terms are present in the radial and meridional components while the azimuthal component is discretized in angular momentum conserving form. The corresponding divergence operators are

$$\begin{aligned}
\nabla \cdot \mathbf{F} &= \frac{1}{r^2} \frac{\partial (r^2 F_r)}{\partial r} + \frac{1}{r \sin \theta} \frac{\partial (\sin \theta F_\theta)}{\partial \theta} + \frac{1}{r \sin \theta} \frac{\partial F_\phi}{\partial \phi} \\
\nabla^r \cdot \mathbf{F} &= \frac{1}{r^3} \frac{\partial (r^3 F_r)}{\partial r} + \frac{1}{r \sin^2 \theta} \frac{\partial (\sin^2 \theta F_\theta)}{\partial \theta} + \frac{1}{r \sin \theta} \frac{\partial F_\phi}{\partial \phi}
\end{aligned} \tag{A.18}$$

B. *Predefined Constants and Macros*

B.1 Predefined Physical Constants

gPLUTO has several predefined physical and astronomical constants in c.g.s. units which may be used anywhere in the code (see `pluto.h`):

```
#define CONST_AH      1.008          /**< Atomic weight of Hydrogen */
#define CONST_AHe     4.004          /**< Atomic weight of Helium */
#define CONST_AZ      30.0           /**< Mean atomic weight of heavy elements */
#define CONST_amu     1.66053886e-24 /**< Atomic mass unit. */
#define CONST_au      1.49597892e13  /**< Astronomical unit. */
#define CONST_c       2.99792458e10  /**< Speed of Light. */
#define CONST_eV      1.602176463158e-12 /**< Electron Volt in erg. */
#define CONST_G       6.6726e-8      /**< Gravitational Constant. */
#define CONST_h       6.62606876e-27 /**< Planck Constant. */
#define CONST_kB      1.3806505e-16  /**< Boltzmann constant. */
#define CONST_ly      0.9461e18      /**< Light year. */
#define CONST_mp      1.67262171e-24 /**< Proton mass. */
#define CONST_mn      1.67492728e-24 /**< Neutron mass. */
#define CONST_me      9.1093826e-28  /**< Electron mass. */
#define CONST_mH      1.6733e-24     /**< Hydrogen atom mass. */
#define CONST_Msun    2.e33          /**< Solar Mass. */
#define CONST_Mearth  5.9736e27      /**< Earth Mass. */
#define CONST_NA      6.0221367e23   /**< Avogadro Contant. */
#define CONST_pc      3.0856775807e18 /**< Parsec. */
#define CONST_PI      3.14159265358979 /**< \f$ \pi \f$. */
#define CONST_Rearth  6.378136e8     /**< Earth Radius. */
#define CONST_Rsun    6.96e10        /**< Solar Radius. */
#define CONST_sigma   5.67051e-5     /**< Stephan Boltmann constant. */
#define CONST_sigmaT  6.6524e-25     /**< Thomson Cross section. */
```

B.2 Predefined Function-Like Macros

gPLUTO comes with a number of pre-defined function-like macros to implement simple arithmetic operations such as maximum (**MAX**), minimum (**MIN**), or looping over a specific portion of the computational domain (e.g. **BOX_LOOP**). Please refer to the `Doc/Doxygen/html/macros_8h.html` page in the API reference guide (`Doc/Doxygen/html/index.html`).

B.3 Advanced Options

gPLUTO allows fine-tuning of a number of switches (macros) directly from your definitions.hpp in the user-defined constant section, see §2.3. These advanced options are described in Table B.1.

Table B.1: **gPLUTO** advanced options.

Name	Description
ACC_INIT	[<i>YES / NO</i>] When set to <i>YES</i> (Default), initial conditions are assigned directly on the GPU (device). Otherwise assignment takes place on the CPU and data transfer between host and device occurs later.
ASSIGN_VECTOR_POTENTIAL	[<i>YES / NO</i>] When set to <i>YES</i> , magnetic field components are initialized from the vector potential. In the constrained transport algorithm (CT, §6.2.2.3), this guarantees that the magnetic field has zero divergence. When set to <i>NO</i> , assignment proceeds in the usual way, see §6.2.1 for more details.
CHAR_LIMITING	[<i>YES / NO</i>] Set it to <i>YES</i> to perform reconstruction on characteristic variables rather than primitive. It is available for the HD, RHD and MHD modules. Although somewhat more expensive, reconstruction in characteristic variables is known to produce better quality solutions by suppressing unwanted numerical oscillation in proximity of strong discontinuities and leading to a better entropy enforcement.
COOLING_ODE_TOL	[<i>double</i>] Sets the tolerance used in the adaptive method / step size control during cooling. Default values is $2 \cdot 10^{-5}$. Smaller values will results in more sub-steps when <i>ROS34</i> is employed.
COOLING_TIME_STEPPING	[<i>RKF23 / ROS34 / SELECTIVE</i>] Controls the time-stepping method to solve Eq. (8.2) during the source step. Possible values are: <i>RKF23</i> (3 rd -order explicit Runge Kutta with step size control using a 2 nd -order embedded method, default method), <i>ROS34</i> (4 th -order semi-implicit Rosembrock method with adaptive step size control using a 3 rd -order embedded method), <i>SELECTIVE</i> : a cell-by-cell selective method (between the previous two) based on a stiffness detector.
COVARIANT_BX2	[<i>YES / NO</i>] Control the evolution of the ϕ or θ component of magnetic field in cylindrical coordinates, see Eq. A.3 or A.4. The default is <i>YES</i> .
CT_EMF_AVERAGE	[<i>string</i>] Control how the electromotive force (EMF) is integrated from the face center to the edges. This is discussed in more detailed in §6.2.2.3.
CT_EMF_BOUNDARY	[<i>YES / NO</i>] When set to <i>YES</i> , electric field can be prescribed through the user-supplied function <code>CT_EMF_Boundary()</code> , in order to provide fine control on the evolution of face centered magnetic field components in the CT formalis, see §6.2.2.3.
CT_ENERGY_CORRECTION	[<i>NO, 1</i>] This option is available only in the MHD and RMHD modules. The default is <i>NO</i> , implying that energy is not corrected after the conservative update. However, for low-beta plasma one may find useful to switch this option to <i>1</i> , as described in [BS99] (for MHD flows) and in [Mar15] (for relativistic MHD).
EIGHT_WAVES_REDUCED	[<i>YES/NO</i>] It controls in which equations Powell's eight wave source term should be included. When set to <i>NO</i> (default), introduces source terms into momentum, induction and energy equations ([PRL ⁺ 99]). When set to <i>YES</i> source terms are introduced only in the induction equation ([Jan00, BKW07, WFK11]).

Continued on next page

Table B.1 – Continued from previous page

Name	Description
EPS_PSHOCK_ENTROPY	[<i>double</i>] Sets the maximum shock strength above which fluid variables inside a given computational zone can be safely updated using the entropy equation (see §2.2.4 and the source file <code>Src/flag_shock.cpp</code>). It has effect only when <code>ENTROPY_SWITCH</code> has been enabled. A lower value will trigger the flattening procedure in more zones. Default is 0.05.
EPS_PSHOCK_FLATTEN	[<i>double</i>] Sets the minimum shock strength above which the <i>MULTID</i> shock flattening algorithm flags a zone to be inside a shock (see <code>Src/flag_shock.cpp</code>). It has effect only when <code>SHOCK_FLATTENING</code> is set to <i>MULTID</i> . A lower value will trigger the flattening procedure in more zones. Default is 5.0.
FALLBACK_MODE	[<i>NO/FALLBACK_RECOMPUTE</i>] Enable PLUTO fallback mechanism to handle the occurrence of unphysical values. When set to <i>FALLBACK_RECOMPUTE</i> the current time step is <i>discarded</i> and re-computed using a more robust (and diffusive) numerical setup in proximity of the flagged zones. (Default: <i>NO</i>).
FALLBACK_BUFFER	[$0 \leq \text{int} \leq n_{\text{ghost}}$] The size of the buffer (in number of cells) to be added around a troubled cell in all directions. To avoid divergence between serial and parallel execution this number should not exceed the number of ghost zones. (Default: 1).
FALLBACK_TTL_MAX	[<i>int</i>] Time to live counter controlling how many steps the fallback policy should persist on troubled cells. (Default: 2).
FALLBACK_SUBSTEPS	[<i>int</i>] Control how many sub-steps N_{sub} should be taken while redoing a given step: $\Delta t \leftarrow \Delta t / N_{\text{sub}}$. (Default: 1).
FARGO_NSTEP_AVERAGE	[<i>int</i>] Sets how often the orbital velocity should be recomputed in the FARGO transport step. Default is 10.
FARGO_ORDER	[<i>int</i>] Sets the spatial order of the reconstruction used during the linear transport step of the FARGO algorithm. The allowed values are 3 (third-order, PPM-like reconstruction) or 2 (second-order MUSCL-HANCOCK scheme). Default is 3.
FARGO_OUTPUT_VTOT	[<i>YES/NO</i>] Enable/disable writing of the total velocity to main output data (default value is <i>NO</i>).
GFORCE_OMEGA	[<i>double</i>] Define the ω_g parameter in Eq. (4.1). Note that one must have $0 \leq \omega_g < 1$ where $\omega = 0$ yields the Lax-Friedrichs Flux, $\omega_g = 1/2$ yields the FORCE flux, while $\omega_g = 1$ gives the Lax-Wendroff flux. The default is $\omega_g = 1/(1 + \text{CFL})$.
GLM_ALPHA	[<i>double</i>] Sets the value of the constant α used monopole damping in the GLM formalism, see §6.2.2.2. The default value is 0.1.
GLM_EXTENDED	[<i>YES/NO</i>] Enable the (E)xtended GLM form of the MHD equations, see §6.2.2.2. Default value is <i>NO</i> .
GNUPLOT_HEADER	[<i>YES/NO</i>] Enable writing of <code>pluto.gp</code> , a Gnuplot script file containing grid and variable index information useful for plotting purposes (see <code>Doc/gnuplot.html</code>). Default is <i>NO</i> .
INTERNAL_BOUNDARY	[<i>YES/NO</i>] When turned to <i>YES</i> , it allows to overwrite or change the solution array anywhere inside the computational domain. This is done inside the <code>UserDefBoundary()</code> function when <code>side==0</code> , see §5.2. This option is particularly useful when flow variables must be kept constant in time or to assign lower/upper threshold values to any physical quantity (e.g. density or pressure).

Continued on next page

Table B.1 – Continued from previous page

Name	Description
INTERNAL_BOUNDARY_CFL	[YES/NO] When turned to YES, compute the Courant condition by including also zones flagged with FLAG_INTERNAL_BOUNDARY. Exclude them otherwise. Default is YES.
INTERNAL_BOUNDARY_REFLECT	[YES/NO] When turned to YES, it will create symmetric (for density, pressure and transverse velocities) and antisymmetric (for the normal velocity) profiles about the interface separating a zone flagged as “internal boundary” from an active one. Any Riemann solver should then yield a zero mass flux across this interface, thus mimicking the presence of a solid body. Default is NO.
LIMITER	[lim] Sets the limiter(s) to be applied when RECONSTRUCTION is set to LINEAR. Here <i>lim</i> can be one of • FLAT_LIM: set slope to zero (1st order reconstruction); • MINMOD_LIM: use the minmod limiter (most diffusive); • VANALBADA_LIM: use the van Albada limiter function; • OSPRE_LIM: use the OSPRE limiter; • UMIST_LIM: use the umist limiter; • VANLEER_LIM: use the harmonic mean limiter of van Leer; • MC_LIM: use the monotonized central difference limiter (least diffusive); • DEFAULT: keep the default setting (defined in Src/States/set_limiter.c). where MINMOD_LIM is the most diffusive and MC_LIM is the least diffusive limiter. The TVD diagram for the various limiter functions is shown in Fig B.1. All of the above limiters employ a 3-point stencil.
MAPPED_GRID_FORCE_PERIODS	[YES/NO] When MAPPED_GRID is set to YES, it forces the grid size to respect periodicity conditions even if the transformation does not. This is done by adjusting ghost zones to fulfill periodicity.
MULTIPLE_LOG_FILES	[YES/NO] In parallel mode, controls if multiple log files (one per processor) or just processor #0's log should be dumped to disk. When set to NO (default), the function print() will work only for the root processor, while printLog() enables writing for all processors. If set to YES, each processor opens and write to a distinct log file (print() and printLog() are identical).
NGHOST_USR	[int] When positive, it allows the user to specify the number of ghost zones directly, thus overriding the PLUTO's default selection from get_nghost.cpp. Default is -1 (not effective).
PARALLEL_COMM_ASYNC	[YES/NO] Switch between asynchronous or synchronous MPI communication pattern. The default is YES allowing non-blocking MPI calls (better for performance, more complex implementation). When set to NO the code employs blocking communication by exchanging data one direction at a time (worse for performance, easier to implement).
PRNG	[...] Sets the pseudo random number generator used by the RandomNumber() function. Allowed values are PRNG_DEFAULT (employs standard C drand48() function), PRNG_ECUEYER (long period L'Ecuyer, adapted from Numerical Recipe) and PRNG_MT (64-bit version of Mersenne-Twister pseudo random number generator). Default is PRNG_DEFAULT.

Continued on next page

Table B.1 – Continued from previous page

Name	Description
RADIATION_DIFF_LIMITING	[YES/NO] [RADIATION Module only] Apply the limiter by [SNTZ13] to the radiation characteristic speeds, which can prevent excessive diffusion in high-opacity problems. Default is YES.
RADIATION_FULL_CONVERGENCE	[YES/NO] [RADIATION Module only] Add the relative variation of p to the error function in the implicit step if the iterated fields are (E_r, F_r) , and add the relative variation of E_r if the iterated fields are matter variables. Default is NO.
RADIATION_IMEX_SSP2	[YES/NO] [RADIATION Module only] Integrate the equations using the IMEX-SSP2(2,2,2) method by [PR05] instead of the IMEX1 method [MM19]. This option requires TIME_STEPPING to be RK2. Default is NO.
RADIATION_IMPL	[string] [RADIATION Module only] Implicit method used to integrate the radiation-matter interaction terms. Here, <i>string</i> can be one of • RADIATION_FIXEDPOINT_RAD: fixed-point method by [TO13] based on iterations of (E_r, F_r). This is the default method in the relativistic version. • RADIATION_FIXEDPOINT_GAS: fixed-point method based on iterations of $(p, \gamma v)$. • RADIATION_NEWTON_RAD: Newton method based on iterations of (E_r, F_r). • RADIATION_NEWTON_GAS: Newton method based on iterations of $(p, \gamma v)$. Descriptions of these methods are given in [MM19] and [MKFM21]. In addition, two faster implicit methods are available in the nonrelativistic module: • RADIATION_NEWTON_NR_GAS: nonrelativistic Newton method from [MK22], based on iterations of (p, v). This is the default method for nonrelativistic HD/MHD. • RADIATION_NEWTON_NR_RAD: nonrelativistic Newton method analogous to RADIATION_NEWTON_NR_GAS, but based on iterations of (E_r, F_r).
RADIATION_NEQS	[int] [RADIATION Module only] Number of equations solved during the implicit step (only for RADIATION_IMPL solvers available in both the relativistic and nonrelativistic modules). By default, a 4×4 system is solved. The size of this system can be reduced if DIMENSIONS < 3, provided there is no transport of matter and radiation in one or more of the excluded directions. For example, in a 1D problem where $v^y = v^z = F_r^y = F_r^z = 0$ the radiation subsystem is reduced to 2 equations corresponding to the evolution of E_r and F_r^x , in which case the implicit problem can be reduced by setting this value as 2. Default is 4.
RADIATION_VAR_OPACITIES	[YES/NO] [RADIATION Module only] Define in init.cpp the opacity coefficients as functions of the primitive fields through the function UserDefOpacities(). Default is NO.
RECONSTRUCT_4VEL	[YES/NO] Use the four-velocity $u = \gamma v$ instead of the three velocity when reconstructing left and right states in the RHD, RMHD, and ResRMHD modules. Default is NO.
RING_AVERAGE	[int] When > 1, it enables the ring average technique (see [ZSL ⁺ 19]) to remove the CFL restriction imposed by the clustering of cells in the azimuthal direction near the pole/axis. For instance, when RING_AVERAGE is set to 8, zones in the 1 st anular ring around the pole are averaged in groups of 8 to yield a larger effective $\Delta\phi$. The 2 nd ring will cluster zones into group of 4 and so forth. Only powers of 2 are admissible. This option works also with constrained transport MHD but it does not allow parallelization along the azimuthal direction (ϕ).
RING_AVERAGE_REC	[int] Sets the reconstruction order for zone-centered variables inside the chunk (averaged cells). Allowed values are FLAT, LINEAR, PARABOLIC and MP5. Default is MP5.

Continued on next page

Table B.1 – Continued from previous page

Name	Description
RING_AVERAGE_REC_PRIM	[YES/NO] Reconstruct primitive variable instead of conservative ones. This still yields a conservative averaging and yields generally a more robust scheme. Default is <i>NO</i> .
RING_AVERAGE_CT_REC	[int] Same as RING_AVERAGE_REC but for magnetic flux. Default is <i>FLAT</i> .
RING_AVERAGE_FOURIER	[YES/NO] Apply Fourier reductions for non-Cartesian vectors, as in Eq. (15) of [ZSL ⁺ 19]
RMHD_FAST_EIGENVALUES	[YES/NO] If set to YES, use approximate (and faster) expressions when computing the fast magnetosonic speed of the RMHD equations, see Sect. 3.3 of [DZBL07]. Solutions of quartic equation is avoided and replaced with upper bounds provided by quadratic equation. Default is <i>NO</i> .
RMHD_REDUCED_ENERGY	[YES/NO] Used in the RMHD module (§6.4) to evolve the total energy minus the mass density contribution (see [MM07]). This is more advisable in order to avoid numerical errors in the non-relativistic limit and catastrophic cancellation problems. Default is <i>YES</i> .
ROE_FIX_DEGENERACY	[YES/NO] Fix degeneracy in the Bmag = 0 limit, when less than 3 dimensions are included. This is useful to remove noisy behavior in the z-component when the problem is strictly planar. Default is <i>NO</i> .
ROS34_MAXSTEPS	[int] Sets the maximum number of sub-steps that the Rosembrock semi-implicit method can take while integrating the cooling and chemical network Eq. (8.2). If more steps are required, too many iterations will result in an error.
SHOCK_FLATTENING	[NO/MULTID] Provides additional dissipation in proximity of strong shock, see §2.4
VTK_TIME_INFO	[YES/NO] Enable writing of time information to .vtk output files. Notice that this information is useful only when reading data files with VisIt and may give problems with other visualisation softwares, §11.1.4. Default value is <i>NO</i> .
WARNING_MESSAGES	[-1, ..., 3] Issue a warning message when numerical problems are encountered. Depending on the level of verbosity, the following values are allowed: -1: ignore any error and continue simulation (dangerous, but slightly faster; used in scaling test) 0: Only errors leading to code crash (QuitPLUTO()) are printed; 1: Print the total number of errors during each step (default) without the reporting issuing the error; 2: Report cumulative error information as they occur in each function, at any RK stage; 3: Same as 2 but with extensive zone-by-zone error diagnostic;

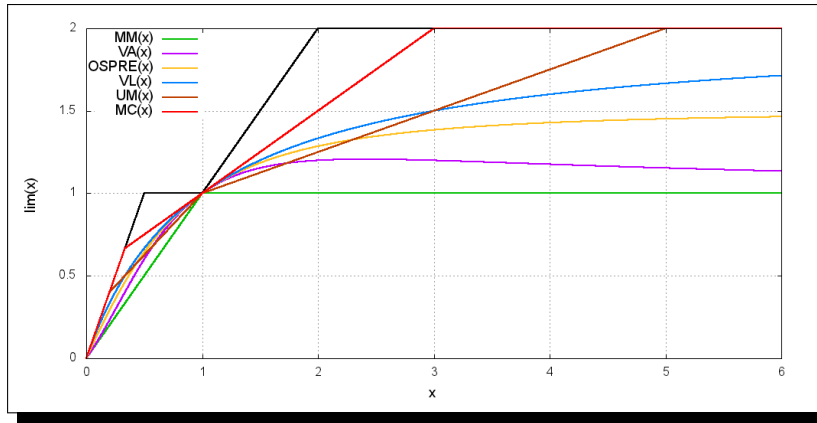


Figure B.1: Second-order TVD limiter functions used by **gPLUTO** as functions of the left to right slope ratio $x = \Delta V_{i-\frac{1}{2}} / \Delta V_{i+\frac{1}{2}}$. Larger values of $\lim(x)$ indicate larger compressive behavior. In this sense, the minmod limiter ($MM(x)$) and the monotonized central limiter ($MC(x)$) are the least and most compressive, respectively.

Bibliography

- [BAV⁺20] Nikhil Borse, Sriyash Acharya, Bhargav Vaidya, Dipanjan Mukherjee, Gianluigi Bodo, Paola Rossi, and Andrea Mignone, *Numerical study of MHD instabilities in AGN jets and their impact on non-thermal emission*, arXiv e-prints (2020), arXiv:2009.13540.
- [BCCD08] R. Borges, M. Carmona, B. Costa, and W. S. Don, *An improved weighted essentially non-oscillatory scheme for hyperbolic conservation laws*, *Journal of Computational Physics* **227** (2008), 3191–3211.
- [Bec92] J. M. Beckers, *Analytical linear numerical stability conditions for an anisotropic three-dimensional advection-diffusion equation*, *SIAM Journal on Numerical Analysis* **29** (1992), no. 3, 701–713.
- [BKW07] François Bouchut, Christian Klingenberg, and Knut Waagan, *A multiwave approximate riemann solver for ideal mhd based on relaxation. i: theoretical framework*, *Numer. Math.* **108** (2007), no. 1, 7–42.
- [BS99] D. S. Balsara and D. S. Spicer, *A Staggered Mesh Algorithm Using High Order Godunov Fluxes to Ensure Solenoidal Magnetic Fields in Magnetohydrodynamic Simulations*, *Journal of Computational Physics* **149** (1999), 270–292.
- [BS03] T. J. M. Boyd and J. J. Sanderson, *The Physics of Plasmas*, January 2003.
- [ČT09] M. Čada and M. Torrilhon, *Compact third-order limiter functions for finite volume methods*, *Journal of Computational Physics* **228** (2009), 4118–4145.
- [CW84] P. Colella and P. R. Woodward, *The Piecewise Parabolic Method (PPM) for Gas-Dynamical Simulations*, *Journal of Computational Physics* **54** (1984), 174–201.
- [DBL03] L. Del Zanna, N. Bucciantini, and P. Londrillo, *An efficient shock-capturing central-type scheme for multi-dimensional relativistic flows. II. Magnetohydrodynamics*, *A&A* **400** (2003), 397–413.
- [DKK⁺02] A. Dedner, F. Kemm, D. Kröner, C.-D. Munz, T. Schnitzer, and M. Wesenberg, *Hyperbolic Divergence Cleaning for the MHD Equations*, *Journal of Computational Physics* **175** (2002), 645–673.
- [DZBL07] L. Del Zanna, O. Zanotti, N. Bucciantini, and P. Londrillo, *ECHO: a Eulerian conservative high-order scheme for general relativistic magnetohydrodynamics and magnetodynamics*, *A&A* **473** (2007), 11–30.
- [GAH07] M. González, E. Audit, and P. Huynh, *HERACLES: a three-dimensional radiation hydrodynamics code*, *A&A* **464** (2007), no. 2, 429–435.
- [GS05] T. A. Gardiner and J. M. Stone, *An unsplit Godunov method for ideal MHD via constrained transport*, *Journal of Computational Physics* **205** (2005), 509–539.
- [Jan00] P. Janhunen, *A Positive Conservative Method for Magnetohydrodynamics Based on HLL and Roe Methods*, *Journal of Computational Physics* **160** (2000), no. 2, 649–661.
- [Kle98] W. Kley, *On the treatment of the Coriolis force in computational astrophysics*, *A&A* **338** (1998), L37–L41.
- [Ld04] P. Londrillo and L. del Zanna, *On the divergence-free condition in Godunov-type schemes for ideal magnetohydrodynamics: the upwind constrained transport method*, *Journal of Computational Physics* **195** (2004), 17–48.
- [Lev84] C. D. Levermore, *Relating Eddington factors to flux limiters.*, *JQSRT* **31** (1984), no. 2, 149–160.
- [Mar15] José-María Martí, *On the correction of conserved variables for numerical RMHD with staggered constrained transport*, *Computer Physics Communications* **191** (2015), 100–105.
- [MBM⁺07] A. Mignone, G. Bodo, S. Massaglia, T. Matsakos, O. Tesileanu, C. Zanni, and A. Ferrari, *PLUTO: A Numerical Code for Computational Astrophysics*, *ApJS* **170** (2007), 228–242.
- [MFS⁺12] A. Mignone, M. Flock, M. Stute, S. M. Kolb, and G. Muscianisi, *A conservative orbital advection scheme for simulations of magnetized shear flows with the PLUTO code*, *A&A* **545** (2012), A152.

- [Mig07] A. Mignone, *A simple and accurate Riemann solver for isothermal MHD*, Journal of Computational Physics **225** (2007), 1427–1441.
- [Mig14] ———, *High-order conservative reconstruction schemes for finite volume methods in cylindrical and spherical coordinates*, Journal of Computational Physics **270** (2014), 784–814.
- [MK05] T. Miyoshi and K. Kusano, *A multi-state HLL approximate Riemann solver for ideal magnetohydrodynamics*, Journal of Computational Physics **208** (2005), 315–344.
- [MK22] J. D. Melon Fuksman and H. Klahr, *No Self-shadowing Instability in 2D Radiation Hydrodynamical Models of Irradiated Protoplanetary Disks*, ApJ**936** (2022), no. 1, 16.
- [MKFM21] J. D. Melon Fuksman, H. Klahr, M. Flock, and A. Mignone, *A Two-moment Radiation Hydrodynamics Scheme Applicable to Simulations of Planet Formation in Circumstellar Disks*, ApJ**906** (2021), no. 2, 78.
- [MM07] A. Mignone and J. C. McKinney, *Equation of state in relativistic magnetohydrodynamics: variable versus constant adiabatic index*, MNRAS**378** (2007), 1118–1130.
- [MM19] J. D. Melon Fuksman and A. Mignone, *A Radiative Transfer Module for Relativistic Magnetohydrodynamics in the PLUTO Code*, ApJS**242** (2019), no. 2, 20.
- [MMBD19] A. Mignone, G. Mattia, G. Bodo, and L. Del Zanna, *A constrained transport method for the solution of the resistive relativistic MHD equations*, MNRAS**486** (2019), no. 3, 4252–4274.
- [MPB05] A. Mignone, T. Plewa, and G. Bodo, *The Piecewise Parabolic Method for Multidimensional Relativistic Fluid Dynamics*, ApJS**160** (2005), 199–219.
- [MT10] A. Mignone and P. Tzeferacos, *A second-order unsplit Godunov scheme for cell-centered MHD: The CTU-GLM scheme*, Journal of Computational Physics **229** (2010), 2117–2138.
- [MTB10] A. Mignone, P. Tzeferacos, and G. Bodo, *High-order conservative finite difference GLM-MHD schemes for cell-centered MHD*, Journal of Computational Physics **229** (2010), 5896–5920.
- [MUB09] A. Mignone, M. Ugliano, and G. Bodo, *A five-wave Harten-Lax-van Leer Riemann solver for relativistic magnetohydrodynamics*, MNRAS**393** (2009), 1141–1156.
- [MZ20] Andrea Mignone and Luca Zanna, *Systematic construction of upwind constrained transport schemes for MHD*, J. Chem. Phys.(2020).
- [MZM⁺99] M. Micono, N. Zurlo, S. Massaglia, A. Ferrari, and D. B. Melrose, *Diffusive shock acceleration in extragalactic jets*, A&A**349** (1999), 323–333.
- [MZT⁺12] A. Mignone, C. Zanni, P. Tzeferacos, B. van Straalen, P. Colella, and G. Bodo, *The PLUTO Code for Adaptive Mesh Computations in Astrophysical Fluid Dynamics*, ApJS**198** (2012), 7.
- [Pow94] K. G. Powell, *Approximate Riemann solver for magnetohydrodynamics (that works in more than one dimension)*, Tech. report, March 1994.
- [PR05] L. Pareschi and G. Russo, Journal of Scientific computing **25** (2005), no. 1, 129–155.
- [PRL⁺99] K. G. Powell, P. L. Roe, T. J. Linde, T. I. Gombosi, and D. L. De Zeeuw, *A Solution-Adaptive Upwind Scheme for Ideal Magnetohydrodynamics*, Journal of Computational Physics **154** (1999), 284–309.
- [Roe81] P. L. Roe, *Approximate Riemann Solvers, Parameter Vectors, and Difference Schemes*, Journal of Computational Physics **43** (1981), 357–372.
- [SH97] A. Suresh and H. T. Huynh, *Accurate Monotonicity-Preserving Schemes with Runge Kutta Time Stepping*, Journal of Computational Physics **136** (1997), 83–99.
- [SNTZ13] A. Skadowski, R. Narayan, A. Tchekhovskoy, and Y. Zhu, *Semi-implicit scheme for treating radiation under M1 closure in general relativistic conservative fluid dynamics codes*, MNRAS**429** (2013), no. 4, 3533–3550.
- [Syn57] John Lighton Synge, *The relativistic gas*, North-Holland Publishing Company ; Interscience Publishers, Amsterdam; New York, 1957.
- [TO13] H. R. Takahashi and K. Ohsuga, *A Numerical Treatment of Anisotropic Radiation Fields Coupled with Relativistic Resistive Magnetofluids*, ApJ**772** (2013), no. 2, 127.
- [Tor97] Eleuterio F. Toro, *Riemann solvers and numerical methods for fluid dynamics*, 2 ed., Springer-Verlag Berlin Heidelberg, 1997.
- [TT06] E. F. Toro and V. A. Titarev, *MUSTA fluxes for systems of conservation laws*, Journal of Computational Physics **216** (2006), no. 2, 403–429.

- [VMB⁺18] Bhargav Vaidya, Andrea Mignone, Gianluigi Bodo, Paola Rossi, and Silvano Massaglia, *A Particle Module for the PLUTO Code. II. Hybrid Framework for Modeling Nonthermal Emission from Relativistic Magnetized Flows*, *ApJ***865** (2018), no. 2, 144.
- [WFK11] K. Waagan, C. Federrath, and C. Klingenberg, *A robust numerical scheme for highly compressible magnetohydrodynamics: Nonlinear stability, implementation and tests*, *Journal of Computational Physics* **230** (2011), no. 9, 3331–3351.
- [YC09] N. K. Yamaleev and M. H. Carpenter, *A systematic methodology for constructing high-order energy stable WENO schemes*, *Journal of Computational Physics* **228** (2009), 4248–4272.
- [ZSL⁺19] Binzheng Zhang, Kareem A. Sorathia, John G. Lyon, Viacheslav G. Merkin, and Michael Wiltberger, *Conservative averaging-reconstruction techniques (Ring Average) for 3-D finite-volume MHD solvers with axis singularity*, *Journal of Computational Physics* **376** (2019), 276–294.